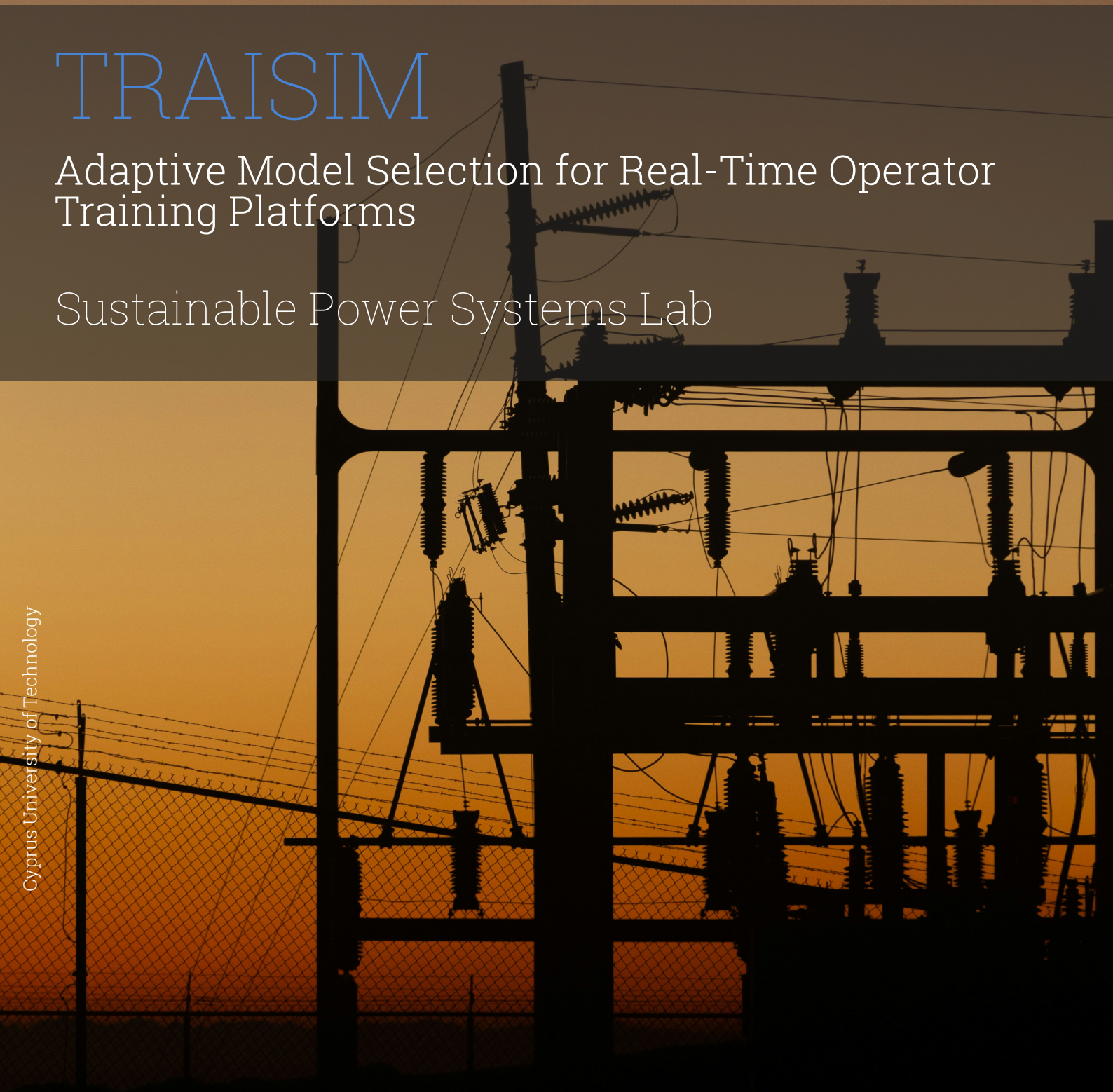
TRAISIM

Adaptive Model Selection for Real-Time Operator Training Platforms

Sustainable Power Systems Lab



Cyprus University of Technology

Cyprus University of Technology

Sustainable Power Systems Lab

RTE

CRESYM

TRAISIM

Adaptive Model Selection for Real-Time Operator Training Platforms

by

Sustainable Power Systems Lab

Contributors

Stefanos Eleftheriadis, SPS-Lab, CUT
Mohammad Hashemnezhad, SPS-Lab, CUT
Savvas Panagi, SPS-Lab, CUT
Petros Aristidou, SPS-Lab, CUT

Reviewers

Thibaut Vermeulen, RTE
Joseph Geethu, CRESYM

Funder:	CRESYM
Funding ID:	TRAISIM
Project Duration:	Jan, 2025 - Dec, 2027
Report Date:	Tuesday 3rd November, 2026
Cover Credits:	Michael Chen (Unsplash)
Style Credits:	TU Delft Report Style (modified by Daan Zwaneveld)



SPS
Sustainable Power Systems Lab



Cyprus University of Technology

Preface

This technical report represents the third deliverable of the TRAISIM project, a collaborative initiative between the Sustainable Power Systems Lab (Cyprus University of Technology), RTE, and the CRESYM consortium.

The work presented herein focuses on the development of an Adaptive Model Selection (AMS) framework, designed to dynamically adjust the level of detail in power system simulations for operator training platforms. By combining detailed dynamic modeling with AI-driven decision-making, this framework ensures that simulations remain both accurate and capable of real-time performance, addressing the trade-off between computational load and model fidelity.

The report lays the foundation for further investigations into intelligent, scenario-driven system representations, aiming to enhance the effectiveness, efficiency, and responsiveness of operator training exercises. The full Python coding scripts implementing the AMS methodology are included as part of the project deliverables.

Sustainable Power Systems Lab
Lemesos, November 2026

Summary

This report presents Deliverable 3 of the TRAISIM project: an Adaptive Model Selection (AMS) framework for operator training platforms. Full-detail dynamic models reproduce the phenomena that trainees must experience, but they often violate the real-time budget of a SCADA-compatible training loop. Coarse fixed reductions restore timing at the cost of trajectory fidelity. AMS addresses that trade-off by predicting, from the pre-disturbance operating point and the contingency description alone, which network components will be dynamically active, and by retaining full node-breaker detail only where that activity matters.

Activity is quantified by a sliding-window variance KPI on bus voltage magnitudes and generator apparent-power trajectories. Two successive formulations are developed and evaluated. The first maps group-wise normalised KPIs to ordered severity classes and trains two Graph Attention Network (GAT) predictors with CORAL ordinal heads on the French transmission system. On the held-out test set, node-level exact agreement reaches 93.26% for voltage and 79.47% for apparent power, with underestimation held at 0.01%. At substation scope, accuracy rises to 94.26%. When the predicted maps drive AMS at retention threshold $\epsilon = 1$, representative contingencies achieve speedups of about 2.4–$2.6\times$ with voltage Dynamic Time Warping (DTW) errors of order 10^{-5}, well below the solver tolerance, whereas a static Regional neighbourhood baseline is faster but incurs DTW errors of order 10^{-2}.

The second formulation removes the global normalisation step, selects physically aware cuts on the raw KPI scale, and replaces GAT–CORAL with two task-specific pair-aware Graph Isomorphism Networks with Edge features (GINE). Nordic 32 is used for cut selection, prediction diagnostics, and architecture comparison; the French system tests industrial-scale transfer and AMS. On Nordic 32, held-out balanced accuracy rises from 0.859 / 0.883 (XGBoost) to 0.893 / 0.921 with a pair-aware MLP, and to about 0.91–0.94 with edge-aware graph encoders; GINE attains 0.914 / 0.934 at about 1.4 ms per scenario, the lowest inference cost among those encoders. On the French test set, within-one accuracies remain 0.930 and 0.917 despite strong inactive dominance. Across 6768 French scenarios, AMS Prediction at $\epsilon = 5$ reduces continuous and discrete model size by about 72% and 66%. When the critical keep-set is recovered, voltage DTW stays near 10^{-5} with speedups up to $3.16\times$; underprediction of control-critical substations remains the residual risk and motivates a deliberately conservative bias toward overprediction.

Together, the results support the project hypothesis that graph-based activity prediction can configure hybrid-detail training models that are both interactive and physically faithful. The accompanying Python implementations are part of the deliverable.

Contents

Preface i

Summary ii

1 Introduction 3
- 1.1 Why Operator Training is More Critical Today . . . 4
- 1.2 Real-Time Performance and Model Detail Trade-Offs . . . 4
- 1.3 Project Objective . . . 5
- 1.4 Hypothesis and Proposed Approach . . . 6

2 Literature Review and Gap Analysis 8
- 2.1 Methods for predicting system dynamics using ML methods . . . 8
- 2.2 Model reduction methods . . . 16
- 2.3 Key Performance Indicators in Dynamic Simulation . . . 21
- 2.4 Gap Analysis . . . 24

3 Proposed Methodology 25
- 3.1 Methodology Overview . . . 25
  - 3.1.1 Methodology 1: Model Performance Prediction . . . 25
  - 3.1.2 Methodology 2: Dynamic Activity Prediction . . . 26
  - 3.1.3 High Level Comparison of Proposed Methodologies . . . 27
  - 3.1.4 Selected Methodology and Component-Level Dynamic Activity Assessment . . . 27
  - 3.1.5 First Approach: GAT-Based Prediction . . . 28
  - 3.1.6 Second Approach: Pair-Aware GINE-Based Prediction . . . 30
  - 3.1.7 Shared Substation-Level Simplification . . . 34
- 3.2 Data requirements . . . 34
- 3.3 Implementation . . . 34
  - 3.3.1 Operating points and contingencies . . . 35
  - 3.3.2 Full-detail simulation and KPI labelling . . . 38
  - 3.3.3 First approach (GAT–CORAL) . . . 38
  - 3.3.4 Second approach (pair-aware GINE) . . . 38
  - 3.3.5 Adaptive Model Simplification and comparison . . . 38

4 Results 39
- 4.1 KPI calculation . . . 39
- 4.2 Results of the First Approach . . . 40
  - 4.2.1 Class construction and label distributions . . . 40
  - 4.2.2 Prediction outcomes on the test set . . . 41
  - 4.2.3 Simplification testing . . . 43
- 4.3 Results of the Second Approach . . . 44
  - 4.3.1 Physically aware cut selection and class construction . . . 44
  - 4.3.2 Prediction outcomes on Nordic 32 . . . 48
  - 4.3.3 Architecture comparison on Nordic 32 . . . 51
  - 4.3.4 Scalability on the French transmission system . . . 52
  - 4.3.5 Adaptive simplification and real-time performance . . . 55

5 Conclusion 58

References 59

Appendices 63

A Implementation Code Listings **64**
A.1 Operating point definition and extension 64
A.2 Full-detail simulation driver 65
A.3 KPI Extraction, Disconnection, and Action Flags 66
A.4 Implementation of the First Approach: GAT-Based Prediction 67
A.5 Implementation of the Second Approach: Pair-Aware GINE-Based Prediction 72
A.6 Simplified cases, retention, and comparison to the benchmark 81
A.7 Switch modelling and root-function bookkeeping 82

List of Figures

1.1 Overview of a modern operator training platform [32] . . . 3
1.2 Accuracy–Computational Speed Trade-off (left: computational time, right: voltage response). Model 1 uses a simplified topology designed for fast, event-based simulations, while Model 3 represents a high-fidelity configuration with full switch tracking and detailed system dynamics. [32] . . . 5
1.3 Training platform with the proposed Adaptive Topology Selections module. . . . 6

2.1 Division of the power system into a study area and an external system for measurement-based model reduction [63]. . . . 16

3.1 Offline process of proposed methodology 1 . . . 25
3.2 Online process of proposed methodology 1 . . . 26
3.3 Offline process of proposed methodology 2 . . . 26
3.4 Online process of proposed methodology 2 . . . 27
3.5 Graph construction principle used to map the power system into a homogeneous graph representation. . . . 29
3.6 Graph attention forward architecture for node-level ordinal classification. . . . 29
3.7 Architecture of the pair-aware GINE model for component-level dynamic activity prediction. 32
3.8 AMS methodology diagram with parallel voltage and apparent-power models and substation-level conversion. . . . 35

4.1 Sliding-window variance KPI for a line-disconnection case. . . . 39
4.2 Example trajectories for different activity classes. . . . 40
4.3 Class distribution of the first-approach datasets. . . . 41
4.4 Predicted versus true substation activity classes. . . . 42
4.5 Apparent-power test distance histogram. . . . 42
4.6 Approach 1: Apparent-power Pred–True per node example. . . . 42
4.7 Voltage test distance histogram. . . . 43
4.8 Approach 1: Voltage Pred–True per node example. . . . 43
4.9 Solver time and voltage response for Full, Regional, and AMS. . . . 44
4.10 Nordic 32 cut selection: usability versus physical score for voltage (left) and apparent power (right). . . . 45
4.11 French-system cut selection: usability versus physical score for voltage (left) and apparent power (right). . . . 45
4.12 Nordic 32 raw KPI histograms with the selected class cuts. . . . 46
4.13 Nordic 32 raw KPI distributions by assigned class. . . . 46
4.14 Morphological severity φ versus assigned class on Nordic 32. . . . 47
4.15 Representative Δ trajectories for classes 0–4. . . . 47
4.16 Class distributions after physically aware cut selection. . . . 48
4.17 Nordic 32 voltage: test distance histogram. . . . 49
4.18 Nordic 32 voltage: row-normalised confusion matrix. . . . 49
4.19 Approach 2: Nordic 32 voltage Pred–True per node example. . . . 50
4.20 Nordic 32 apparent power: test distance histogram. . . . 50
4.21 Nordic 32 apparent power: row-normalised confusion matrix. . . . 51
4.22 Approach 2: Nordic 32 apparent power Pred–True per node example. . . . 51
4.23 Validation selection score on Nordic 32 (5-fold mean$\pm$std). . . . 52
4.24 French voltage: test distance histogram. . . . 53
4.25 French voltage: row-normalised confusion matrix. . . . 53
4.26 Approach 2: French voltage Pred–True per node example. . . . 54

4.27 French apparent power: test distance histogram. 54
4.28 French apparent power: row-normalised confusion matrix. 55
4.29 Approach 2: French apparent power Pred–True per node example. 55
4.30 Solving time and voltage for Under, Over, and Well scenarios ($\epsilon = 5$). 56

List of Tables

2.1 Summary of related works on prediction of dynamics approaches 9
2.2 Summary of related works on measurement-based approaches 17
2.3 Representative Voltage Stability Indicators from the Literature 23
2.4 Transient Stability and Coherency Indicators 23
2.5 Participation-Factor-Based Indicators 23
2.6 Norm-Based Performance Measures in Model Reduction 23

3.1 French-system operating points used in the implementation. 36
3.2 Grouped French contingency counts by operating-point set. 36
3.3 Nordic operating points used in the implementation. 37
3.4 Grouped Nordic contingency counts by operating-point set. 37

4.1 First-approach ordinal classes on the normalised KPI scale. 40
4.2 Prediction outcomes at node and substation level 41
4.3 System-size reduction after topology simplification. 43
4.4 Real-time performance and voltage fidelity ($\epsilon = 1$). 44
4.5 Examples of Nordic 32 voltage cut candidates. 45
4.6 Selected KPI cuts for Nordic 32 and the French system. 46
4.7 Nordic 32 architecture comparison on the held-out test set. Inference is the median (mean$\pm$std), in ms per scenario. Train wall is Optuna plus the final retrain. 52
4.8 Held-out test metrics of the pair-aware GINE on the French system. 52
4.9 Average system size after substation-level simplification on the French test set. 56
4.10 Real-time performance and voltage fidelity across scenarios and systems. 57

List of Acronyms

L_{mn} Line Stability Index. 21, 23

AI Artificial Intelligence. 8, 15
ANFIS Artificial Neuro-Fuzzy Interface System. 16, 18
ARX AutoRegressive eXogenous. 16–18

BCS Baja California Sur. 10

CF Connectivity Factor Index. 21, 23
CF/SF System Separation Status. 21, 23
CNN Convolutional Neural Network. 10, 12, 13

DL Deep Learning. 13
DRL Deep Reinforcement Learning. 13
DSA Dynamic Stability Assessment. 12
DST Deep Spatio-Temporal. 14
DT Decision Tree. 19, 20

EEAC Extended Equal-Area Criterion. 12
EI Eastern Interconnection. 17, 18
EMS Energy Management System. 12
EMT PF boundary EMT Boundary via Participation Analysis. 21, 23
EPGS Electrical Power Grid Stability. 12

FDR Frequency Disturbance Recorder. 16–19
FVSI Fast Voltage Stability Index. 21, 23

GAN Generative Adversarial Network. 13
GATv2 Graph Attention Network. 14
GCN Aggregates Graph Convolutional Network. 13
GEDF Graph Embedding Dynamic Features. 14
GEDF-SCL Graph Embedding Dynamic Feature-based Supervised Contrastive Learning. 14
GRU Gated Recurrent Unit. 8, 14

HDLM Hierarchical Deep Learning Machine. 12

IBR Inverter-Based Resource. 13

KAN Kolmogorov–Arnold Network. 14

LQP Line Stability Factor. 21, 23

LSTM Long Short-Term Memory. 12, 13
LVSI Line Voltage Stability Index. 21, 23

ML Machine Learning. 8, 13, 24, 26, 27

NB Naïve Bayes. 19
NEPOOL New England Power Pool. 17
NPCC Northeast Power Coordinating Council. 13, 17–20

PMU Phasor Measurement Unit. 9, 16–20

RACER Risk-Aware Control for Extreme Resilience. 15
Res-BiTCN Residual Bidirectional Temporal Convolutional Network. 14
RGCN Recurrent Graph Convolutional Network. 13
RNN Recurrent Neural Network. 12, 18

SAE Stacked Autoencoder. 13
SF Separation Factor Index. 21, 23
SVDKR Stochastic Variational Deep Kernel Regressor. 14
SVM Support Vector Machine. 9, 12, 19, 20

TF Transfer Function. 17, 18
Time-variant PF Time-variant Participation Factor. 21, 23
TSA Transient Stability Assessment. 12–14, 19, 20
TSI Transient Stability Index. 19, 21, 23
TSP Transient Stability Prediction. 12–14, 19
TTEDNN Temporal and Topological Embedding Deep Neural Network. 10, 14

WAMS Wide-Area Measurement Systems. 19
Wavelet PF Wavelet-based Participation Factor. 21, 23
WECC Western Electricity Coordinating Council. 19

1

Introduction

Operator training has long been a key aspect of power system operations, traditionally focused on standard procedures, safety practices, and routine operational tasks. As power systems have become more complex and technology-driven, training must adapt to prepare operators for rapidly changing conditions and the challenges of real-time decision-making.

Modern operator training platforms address these challenges by providing realistic simulation environments that replicate the behaviour of the power system in a controlled, interactive setting. Trainees can observe, analyse, and respond to dynamic scenarios, practising decision-making and coordination skills in a safe and fully simulated operational context. The architecture of such platforms, illustrated in Figure 1.1, integrates several key components that work together to recreate the operational environment:

- **Physical Simulator:** Reproduces the dynamic behaviour of generators, loads, and network elements in real time.
- **Information Flow Model:** Ensures accurate and timely data exchange between components.
- **Orchestrator:** Coordinates interactions among the simulator, operator inputs, and automated control systems.
- **Game Master (AI):** Introduces realistic events such as faults, disturbances, or unexpected operational challenges.
- **Human-Machine Interface (HMI):** Provides operators with a control-room-like interface for practice and decision-making.
- **Trainer:** Supervises exercises, monitors performance, and adjusts scenarios dynamically.

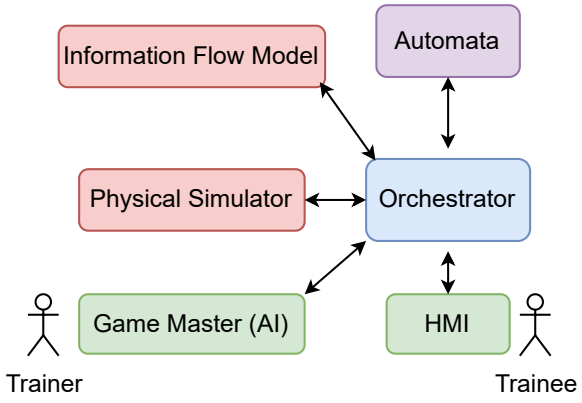


Figure 1.1: Overview of a modern operator training platform [32]

This framework enables operators at all experience levels to practise complex operational strategies in a safe, controlled environment, strengthening both technical proficiency and situational awareness that directly translate to real-world operations. The following section examines how the evolving characteristics of power systems have impacted the demands on operator training.

1.1. Why Operator Training is More Critical Today

Traditionally, electrical power systems were relatively simple and predictable. Large centralized power plants, primarily using synchronous machines, were connected through a stable, hierarchical transmission network. Power flowed in a largely unidirectional manner from generators to consumers, and system dynamics evolved slowly. Operators could maintain frequency, voltage, and power balance using established procedures, and the risk of cascading failures was low. Under these conditions, traditional training methods based on steady-state analysis and standard operating procedures were sufficient for developing operational skills.

In contrast, modern power systems have evolved into complex cyber-physical infrastructures that incorporate a significant share of renewable energy sources (RES), such as wind and solar. Unlike conventional synchronous generators, these units connect via power electronics and do not inherently provide rotational inertia. System dynamics are now fast, highly non-linear, and sensitive to network conditions, often occurring on millisecond timescales. Additionally, modern grids incorporate wide-area measurement systems, high-speed communication networks, and automated control platforms, making system behaviour more complex and less predictable.

The combination of conventional and converter-based technologies results in a system with mixed dynamic characteristics. Operators must manage rapid fluctuations, uncertainties, and interactions across multiple devices and control layers to ensure seamless operation. Consequently, effective operator training is more critical than ever; it must reflect the fast, coupled dynamics of modern systems and operate in real-time, allowing trainees to interact with unfolding scenarios. Advanced training platforms are crucial for equipping operators with the skills necessary to maintain stability, reliability, and safety in modern power systems.

1.2. Real-Time Performance and Model Detail Trade-Offs

Building on the need for advanced operator training platforms, a critical requirement is that the dynamic simulations run in *real time*. In this study, we follow the literature in defining *real time* in terms of the platform's ability to match the refresh rate of modern SCADA systems. Specifically, the end-to-end computational pipeline—including the physical simulator, information-flow layer, orchestrator, and HMI, as shown in Figure 1.1—must provide updated measurements and control states at intervals compatible with SCADA, typically around 2–5 s. Achieving this end-to-end refresh requires that internal simulation components operate at a faster rate than the overall platform refresh. In particular, the physical simulator should use an internal integration step that is sufficiently small to capture fast dynamics and provide timely data to the rest of the chain. For example, the setup described in [3] adopted a 1 s simulation step for the physical simulator while targeting an overall platform refresh compatible with the 2–5 s SCADA window.

Meeting real-time performance, however, becomes increasingly challenging as the overall system size grows. The size of the differential–algebraic equation (DAE) system depends on both the extent of the network—i.e., the number of buses, lines, and components included—and the level of detail used to model individual devices such as generators, loads, converters, and switches. As shown in [32], this level of detail is a key factor determining real-time performance. Higher-fidelity models produce larger DAE systems that better capture the dynamic behaviour and local interactions of the network but require greater computational resources. Conversely, simplified models reduce the computational load and enable faster execution but may fail to reproduce certain dynamic events or system responses with sufficient accuracy.

This trade-off between accuracy and computational speed is illustrated in [32] through two representative system configurations:

- **Model 1:** Designed for fast event-based simulations with simplified topology. This configuration

includes limited substation-level detail and omits non-essential switches, allowing for rapid and real-time execution suitable for general operator training exercises.

- **Model 3:** Provides a maximum-detail configuration where all switches are tracked (breakers and disconnectors), resulting in a larger number of model parameters and higher computational requirements. This setup is used for switching scenarios, including all possible combinations of routing between feeders and busbar sections, capturing the full dynamics of the system.

Figure 1.2 illustrates the implications of the accuracy–real-time trade-off using the *headroom per time step* metric. The headroom is defined as

$$dt = T_{\text{sync}} - T_{\text{sol}},$$

where T_{sync} denotes the real-time interval allocated to each simulation step (e.g., the SCADA/HMI refresh period) and T_{sol} is the actual computational time required to complete the numerical solution for that step. In this study, $T_{\text{sync}} = 1\,\text{s}$, consistent with the configuration adopted in [32]. Positive values of dt indicate faster-than-real-time execution with available timing margin, whereas negative values correspond to real-time violations. In practice, sustained real-time operation requires a positive buffer $dt \geq dt_{\text{min}} > 0$ to accommodate orchestration overheads and operating-system jitter.

These effects are illustrated in Figure 1.2. The left panel reports the per-step computation time for the different models, showing that Model 3 intermittently exceeds the available real-time budget, while Model 1 consistently satisfies the real-time constraint. The right panel presents the corresponding voltage trajectories, highlighting that Model 3 captures finer dynamic behavior that may be lost in simplified configurations. This illustrates the inherent trade-off between numerical fidelity and achieving reliable T_{sync}-level accuracy in real-time simulation.

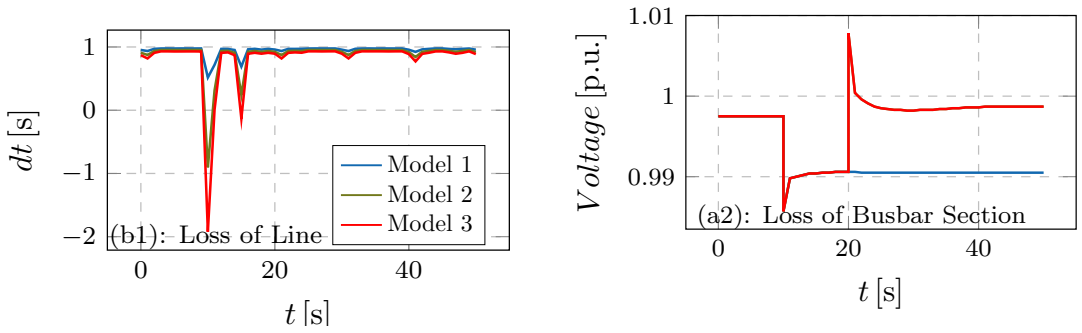


Figure 1.2: Accuracy–Computational Speed Trade-off (left: computational time, right: voltage response). Model 1 uses a simplified topology designed for fast, event-based simulations, while Model 3 represents a high-fidelity configuration with full switch tracking and detailed system dynamics. [32]

The challenges illustrated by the trade-off between model detail and real-time performance highlight a key limitation of conventional training platforms: a fixed system representation cannot simultaneously capture all relevant dynamics while ensuring interactive, real-time simulation. Addressing this limitation requires a flexible approach that can adapt the level of detail in the system model according to the objectives and requirements of each training scenario. The following section describes the project objective, which focuses on developing such an adaptive framework.

1.3. Project Objective

Building on this need for adaptive system representation, the primary objective of this project is to develop a framework that can **dynamically adjust the system model** to meet the requirements of specific training scenarios. Depending on the scenario, study area, types of dynamic phenomena, and the system operating point, the model can selectively include or simplify parts of the system. Critical regions are modelled in full detail, while less relevant areas are aggregated or simplified. Similarly, the level of detail in dynamic models of generators, loads, and other devices can be adjusted according to the phenomena of interest. By tailoring the system representation in this way, the platform provides realistic, timely feedback to trainees while maintaining computational efficiency.

This adaptive model selection strategy offers several key benefits:

1. **Realistic and targeted operator experience:** Operators interact with fast, complex, and sometimes unpredictable dynamics under conditions that closely mimic real operations. The platform can focus on critical regions of the system while simplifying less relevant areas, ensuring trainees experience the phenomena that matter most for the exercise.
2. **Maintained platform responsiveness:** By adjusting the level of detail where necessary, the simulation remains interactive and responsive. Trainees experience events unfolding in real time, which is essential for developing situational awareness and effective decision-making skills.

Overall, this adaptive model selection approach strikes a balance between realism and computational efficiency. It provides a flexible and effective training environment, enabling operators to develop the skills required to safely and efficiently operate complex, converter-dominated power systems.

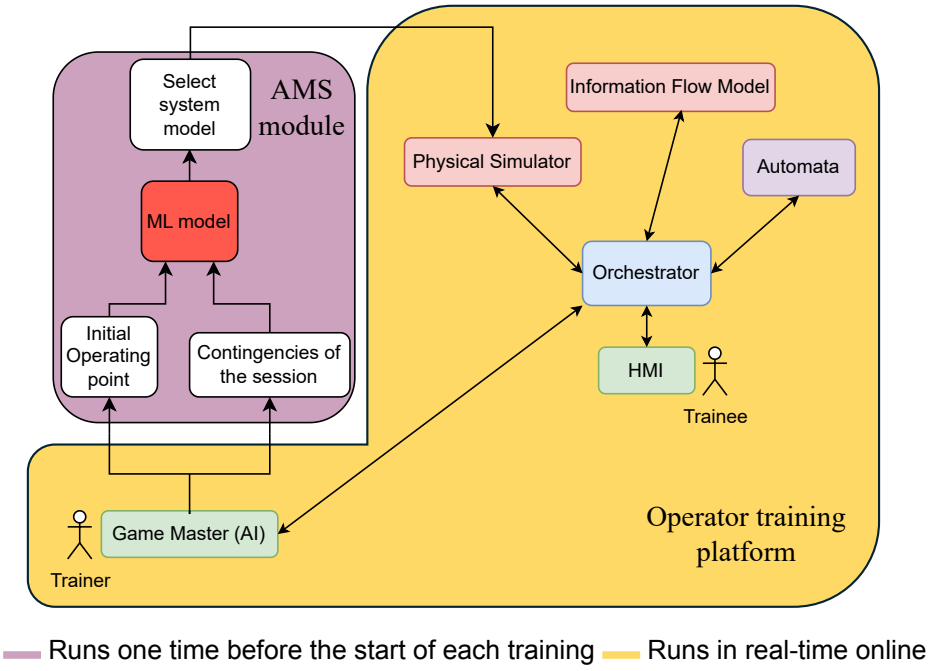


Figure 1.3: Training platform with the proposed Adaptive Topology Selections module.

1.4. Hypothesis and Proposed Approach

The central hypothesis of this project is that Machine Learning (ML) or other AI-based techniques can be leveraged to adaptively determine an optimal network representation for operator training simulations. The idea is to construct a model that, given a set of inputs describing the system's operating point and the specific training scenario provided by the orchestrator and the game master, outputs a system model that maximizes simulation accuracy while ensuring real-time computational constraints are met.

To train such a model, data will be generated from dynamic simulations of multiple scenarios using the full system model under varying load conditions, event types, and initial system operation points. Insights from electrical distance metrics will be incorporated to identify the range of influence of specific events, helping the model understand which parts of the network are critical for accurately reproducing system dynamics. Additionally, clustering the training data by event type or dynamic behaviour can improve the model's generalization capability, ensuring robust performance across a wide variety of training scenarios.

As shown in Figure 1.3, the proposed addition to the training platform is the *Adaptive Model Selection (AMS) module*. This module operates prior to the start of each training session, predicting the optimal level of detail for the system model and implementing it directly on the physical simulator. By doing

so, it creates an intelligent, adaptive framework that automatically selects the level of detail for each region and component of the system, balancing simulation accuracy with real-time computational efficiency. This approach extends the adaptive topology methodology introduced in the previous section, combining detailed dynamic modeling with AI-driven decision-making to support realistic, efficient, and responsive operator training.

2

Literature Review and Gap Analysis

To develop an adaptive and computationally efficient dynamic-simulation framework, it is essential to understand how existing research simplifies system representations without losing key dynamic behaviour. The literature reviewed here provides the theoretical and practical foundations for this goal. Measurement-based model-reduction techniques show how boundary data can construct simplified yet representative system equivalents, while data-driven methods demonstrate how dynamic responses can be predicted without full-order models. Together, these approaches highlight principles that enable flexible, scenario-dependent system representations, which underpin the framework developed in this work.

2.1. Methods for predicting system dynamics using ML methods

In parallel with model reduction approaches, machine-learning-based methods focus on predicting power-system dynamic behaviour directly from data, without relying on explicit full-order simulations. Existing studies show that system responses can be inferred from early post-disturbance measurements, including voltage and angle trajectories [16, 47, 46, 31], or from pre-disturbance steady-state operating conditions [33, 1, 30, 43].

Within the post-disturbance group, recurrent models treat early generator trajectories as sequential inputs rather than as static feature vectors. Okasha et al. propose a GRU-based classifier that predicts the transient-stability state in real time from rotor-angle, rotor-speed, and rotor-acceleration measurements, and they show that combining these three signals improves prediction reliability on the IEEE 39-bus system [31].

Applications include transient trajectory forecasting, stability classification, and estimation of key dynamic metrics using neural networks, ensemble learning techniques, and hybrid physics–ML models. More recent literature also extends this perspective toward risk-aware, AI-enabled operational architectures that connect prediction with control and resilience management under extreme events [45]. A comparative overview of these prediction-oriented methods, learning paradigms, and application domains is provided in Table 2.1. While these approaches significantly reduce computational effort and enable rapid assessment, their performance remains dependent on data quality, operating-condition coverage, and generalization capability. Nevertheless, they provide important insights into fast dynamic-response estimation and motivate hybrid strategies that combine data-driven prediction with physics-based simulation.

Table 2.1: Summary of related works on prediction of dynamics approaches

Reference	Topic / Focus	Techniques Used	Test System / Case Study	Main Benefits	Limitations
[16]	Develop a method that uses voltage phasors measured after a disturbance to determine the system's transient stability status and to signal ahead of time when the power system is nearing a transiently unstable state.	• Train SVM classifier to predict the transient stability status after a fault. • Use as inputs for the SVM, samples collected from the PMUs. • Training Data were generated through offline dynamic simulations.	Results shown on IEEE 39-bus New England system.	• Uses post-disturbance voltage phasors, making it suitable for PMU-based stability assessment. • Does not require identifying the fault location or type. • Low computational burden, enabling real-time classification. • Simple feature set based on voltage trajectories, making implementation practical. • SVM classifier achieves high prediction accuracy in tested scenarios. • Provides early indication of transient instability compared to traditional methods.	• Requires extensive PMU coverage – limited by data availability and quality. • Limited generalization – validated only on a specific test system. • Sensitive to load behavior and variability – certain load dynamics and stochastic variations can reduce reliability. • No contingency identification – treats all fault types uniformly. • Binary classification only – does not provide stability margins. • Requires large offline training datasets and periodic retraining. • Scalability to very large systems not fully demonstrated.

Reference	Topic / Focus	Techniques Used	Test System / Case Study	Main Benefits	Limitations
[47]	Develop a method that uses TTEDNNs to forecast transient stability with the early transient dynamics.	• Use of Grid-informed graph of the power system to capture the topology and the electrical parameters. • Use Graph Convolutional Network (GCN) to extract spatial/topological features. • Use Temporal Convolutional Network (TCN) to extract temporal features from early transient dynamics. • Combine spatial and temporal embeddings in a deep neural network (TTEDNN) to predict transient stability. • Train the model using simulated perturbation scenarios from IEEE test systems. • Use of transfer learning to test whether the trained model can generalize to more complex or higher-order dynamic power system models	Results shown on IEEE 39-bus & IEEE 118-bus systems.	• Captures both temporal and spatial features of the grid • Fast prediction using early post-perturbation dynamics • Does not require fault location or type • Improved accuracy with spatio-temporal deep learning • Transfer learning enables generalization to complex models • Grid-informed approach respects system topology and parameters	• Depends on high-quality measurements of early post-perturbation dynamics • Trained on simulated data; performance on real-world disturbances may vary • Black-box model; reasoning behind predictions is not easily interpretable • Generalization to very large or different system topologies not fully tested • Sensitive to noise or errors in input data
[33]	Develop a framework that predicts transient stability status through a CNN using steady-state information.	• Use as inputs steady-state information from power flow analysis. • Use of the layer freezing technique during the training to improve performance • The Dataset was built through simulations of three-phase faults under a range of loading operating conditions.	Results shown on BCS system.	• Predicts transient stability using only steady-state measurements • Enables fast, real-time assessment with a trained CNN • Does not require full dynamic simulations for each scenario • Improved accuracy with layer-freezing strategy in CNN training • Potential for generalization across varied operating conditions	• Trained on simulated data, may differ from real-world performance • Binary classification only (stable vs unstable) • Relies on selected steady-state inputs, may miss other influences • Limited to three-phase faults, cannot generalize to other fault types

Reference	Topic / Focus	Techniques Used	Test System / Case Study	Main Benefits	Limitations
[1]	Predicting the transient stability margin of a power system using only pre-disturbance (steady-state) system information.	• Uses steady-state information from power flow analysis as inputs. • Multi-swing based strategy to overcome false identification issues. • Dataset built from simulated power system models.	Results shown on IEEE 39-bus system.	• Avoids false identifications common in prior methods. • Enables fast, near real-time assessment. • Uses only steady-state data, no need for full dynamic simulation. • Scalable and practical for online evaluation.	• Trained on simulated data; real-world performance may differ. • Uses only steady-state inputs; may miss some transient dynamics. • Limited generalization to different system topologies and fault types. • Margin prediction is approximate compared to full time-domain simulation.
[30]	Framework for transient-stability assessment of power systems using machine-learning and static features.	• Use of machine learning classifier (unspecified) to assess stability. • Inputs: static (pre-disturbance) features of the grid model. • Training data generated from dynamic simulations of faults and stability outcomes.	Results shown on IEEE 39-bus test system augmented with renewable energy resources	• Enables fast (near real-time) stability assessment using only static data. • Reduces reliance on full dynamic time-domain simulation at runtime. • Potentially scalable once trained.	• Trained on simulated rather than real-world disturbances—generalisation risk. • Static features may fail to capture full dynamic behaviour post-fault. • Performance may degrade for large system topology changes or unseen fault types.
[53]	Predicting dynamic stability of power grids via static features and machine learning.	• ML model trained on grid static features to forecast stability after line outages/faults. • Focus on splitting asynchronous fragments due to line failures.	Case studies on grid models subject to line failure and subsequent asynchronous splitting scenarios (specific bus systems not enumerated).	• Uses only static feature set — simpler than full dynamic time-domain simulation. • ML approach can provide quick assessment of dynamic stability threats.	• Generalisation to diverse fault types or topologies may be limited. • Absence of full dynamic state variables may reduce accuracy under complex post-fault behaviour. • Black-box nature of ML may reduce interpretability.
[19]	ML-based power system stability assessment for modern grids with high renewables.	• Proposes ML framework for power system stability/control in high-RES context.	Demonstrated through simulation of power system models (details of bus/test□system not clearly specified).	• Addresses modern grid challenges (renewables, reduced inertia) via ML. • Potentially enables faster online assessment/control.	• Relies on simulated training data; real-world validation unclear. • May not capture full dynamics of emerging grid devices. • Details on test system and performance metrics somewhat limited.

Reference	Topic / Focus	Techniques Used	Test System / Case Study	Main Benefits	Limitations
[25]	Two-stage transient stability assessment using ensemble learning and cost-sensitivity.	• Stage 1: coarse classification of stability. • Stage 2: refined assessment via ensemble ML, with cost□sensitivity to imbalanced classes.	Application on power-system models for TSA benchmarks (not specified).	• Improves classification accuracy for transient stability via two-stage approach. • Incorporates cost-sensitive learning to address class imbalance (stable vs unstable may be imbalanced).	• Focused on classification only (rather than margin or quantitative stability). • Simulation only; real-world deployment not shown. • Test system details limited in the paper excerpt.
[43]	Rotor angle stability prediction model using steady-state data (pre-contingency) for DSA in the EMS.	• Framework uses EEAC theory to extract physically interpretable features. • SVM classifier is used for prediction. • Uses time-series power flow data partitioned by month to consider system topology changes.	Simulated data from a real-world system implemented in the Korean EMS. Focus on a critical line fault event.	• Addresses limitations of high-resolution post-fault data by using readily available steady-state data. • High effectiveness and real-time response capability for EMS integration. • Provides physically interpretable attributes via EEAC theory.	• Performance relies on the quality of feature extraction from steady-state data. • Requires partitioning by month, indicating complexity in handling topology changes.
[34]	Predicting EPGS using a novel ensemble-based framework to capture dynamic and non-linear grid characteristics.	• Novel Stacked Machine Learning Approach (Ensemble Learning). • Integrates ten distinct classifiers and corresponding regression models. • Uses a meta-model to combine predictions for final stability assessment.	Evaluated using a four-grid star network for feature analysis and several synthetic test grids.	• Achieves high predictive accuracy (up to 99%) by leveraging the strengths of multiple models. • Provides a robust and scalable solution for complex, non-linear grid dynamics.	• The stacked ensemble structure adds complexity to training and deployment. • The reliance on multiple classifiers increases the computational overhead.
[61]	Online identification of post-contingency transient stability; Time-adaptive TSA.	• LSTM network. • Proposes a temporal self-adaptive scheme; first use of RNN/LSTM for time-adaptive TSA.	Results shown on New England 10-machine system.	• Achieves high assessment accuracy with significantly reduced system response time. • Extracts both spatial and temporal data dependency.	• Reliance on comprehensive training data. • Potential generalization issues under unforeseen dynamics.
[64]	Online TSP, covering both quantitative (margin) and qualitative (status) prediction.	• HDLM using a two-level CNN regression model. • Uses anti-noise graphical transient characterization to transform trajectories into 2-D images.	Results shown on IEEE 39-bus system.	• Efficiently achieves both stability status and margin prediction. • Reliable and adaptive online TSP almost immediately after fault clearance.	• Requires transformation of transient trajectories into 2-D image inputs. • Complex two-level CNN regression model structure.

Reference	Topic / Focus	Techniques Used	Test System / Case Study	Main Benefits	Limitations
[59]	Machine Learning-Based Prediction of Distribution Network Voltage and Optimal Sensor Allocation.	• ML based prediction and optimization techniques. • Focuses on efficient voltage prediction and optimal sensor placement.	Results shown on IEEE 39-bus and Polish 2383-bus systems.	• Optimizes sensor placement for reduced cost and enhanced monitoring. • Enables accurate voltage prediction in complex distribution systems.	• Focuses on distribution voltage stability rather than bulk system transient stability. • Limited detail in abstract regarding specific ML model.
[18]	TSA adaptable to power system topology variations; Multi-task learning (classification and critical generator identification).	• RGCN framework. • GCN (for topology) and LSTM (for temporal features).	Results shown on IEEE 39-bus system and IEEE 300-bus systems.	• Addresses poor adaptability of models to topology variation. • Joint training improves efficiency and provides two critical results.	• Complexity of training a multi-task graph-based deep learning model. • Requires synchronized spatio-temporal feature handling.
[4]	A unified online prediction model for both small-signal stability and transient rotor angle stability.	• DL model. • Uses voltage phasor measurements as input.	Results shown on 3 systems: • New-England 10-machine, 39-bus • IEEE 16-machine, 68-bus • IEEE 50-machine, 145-bus	• Unifies small-signal stability and TSA prediction into a single model for comprehensive assessment. • Uses readily available voltage phasor measurements.	• Requires extensive time domain simulations to generate the large training dataset. • Model complexity in handling two distinct stability types.
[39]	TSA considering the challenges from increased integration of renewable energy sources (reduced inertia).	• SAE for feature extraction/dimensionality reduction. • Voting Ensemble classifier for final prediction.	Results shown on IEEE 39-bus (New England).	• Uses SAE for advanced feature engineering, improving TSA accuracy. • Ensemble method enhances robustness and reliability. • Addresses challenges of reduced system inertia.	• Complexity of the SAE preprocessing step. • Performance depends heavily on the feature representation learned by the SAE.
[65]	TSP Control for systems with high penetration of IBRs using DRL.	• DRL algorithms. • Formulates control policies for stability without prior system knowledge.	Results shown on real-world Guangdong Power Grid in South China.	• Produces near-optimal control actions fast without pre-existing structural knowledge. • Addresses challenges of reduced inertia and unforeseen uncertainties.	• Training DRL algorithms for large-scale Power Systems is complex and time-consuming. • Challenges related to finding near-optimal policies and stability guarantees.
[40]	Online TSA via Data Augmentation to address limited sample size and data imbalance.	• GAN based model to generate instability samples. • Uses a CNN supervised model to drive the GAN.	Results shown on 4 systems: • IEEE 39 bus system • NPCC 140 bus system • IEEE 145 bus system • IEEE 300 bus system	• Overcomes data imbalance and limited sample size issues. • Generates realistic yet rare instability cases for training.	• Complexity of training the combined GAN and CNN model. • Ensuring the fidelity and realism of synthetic instability data is challenging.

Reference	Topic / Focus	Techniques Used	Test System / Case Study	Main Benefits	Limitations
[46]	Fast TSP using early transient dynamics, incorporating power grid structure information (topological embedding).	• TTEDNN model. • Uses a Grid-informed Adjacency Matrix to incorporate structural information.	Results shown on IEEE 39-bus and IEEE 118-bus power systems.	• Accurate and efficient TSP using early dynamics. • Robust prediction performance and transfer capability to complex dynamics.	• Requires preprocessing to create the Grid-informed Adjacency Matrix. • Complexity of the deep neural network structure.
[49]	Bayesian Post-Fault Power System Dynamic Trajectory Prediction (e.g., generator rotor angles) for corrective control and TSA.	• SVDKR model. • Bayesian multi-output prediction model.	Results shown on IEEE 39-bus and the Illinois 200-bus systems.	• Provides dynamic trajectory prediction essential for corrective control. • Includes uncertainty quantification (Bayesian).	• High computational load associated with Bayesian deep kernel methods. • Complexity in training the SVDKR model.
[26]	TSP that is adaptable to changing power grid topologies (e.g., due to N-m-1 contingencies).	• GEDF-SCL model. • Uses GEDF to vectorize topology.	Power grids of varying topologies generated based on the IEEE 39-bus system model.	• Achieves high accuracy and excellent adaptability when the grid topology changes. • Overcomes a major limitation of traditional ML TSA models.	• Generating the GEDF adds a layer of complexity to the input data processing. • Contrastive Learning requires careful selection of positive and negative samples.
[62]	Rapid and accurate TSA for large-scale, complex power grids using spatio-temporal feature extraction.	• DST Feature Extraction Network. • Combines GATv2, Res-BiTCN, and KAN for classification.	Results shown on New England 39-bus system and NPCC 140-bus system.	• Significantly improved accuracy and robustness over single-model approaches. • KAN reduces model parameters while maintaining approximation efficiency. • Effectively handles class imbalance using weighted loss.	• Highly complex, multi-layered deep learning architecture. • Increased computational resources needed for training.
[31]	Real-time transient-stability prediction from post-disturbance generator measurements using a GRU-based recurrent model.	• Three stacked GRU layers with a sigmoid output layer for binary stability classification. • Uses temporal sequences of generator rotor angle, rotor speed, and rotor acceleration. • Training dataset generated by DIgSILENT PowerFactory time-domain simulations and normalized with Min-Max scaling.	Results shown on the IEEE 39-bus / 10-machine New England system using 342 simulated stable and unstable disturbance scenarios.	• Captures temporal dependencies in post-fault dynamics using a lightweight recurrent architecture. • Achieves strong real-time performance, with 97.1% accuracy, 100% precision, and 1.6 ms average inference time for the best feature set. • Shows that augmenting rotor-angle inputs with speed and acceleration improves prediction reliability.	• Validated only on one benchmark system and a relatively small simulated dataset. • Focused on binary stable/unstable classification rather than trajectory prediction or stability margins. • Generalization to larger systems, broader disturbance sets, and noisy field measurements is not yet demonstrated.

Reference	Topic / Focus	Techniques Used	Test System / Case Study	Main Benefits	Limitations
[45]	Risk-aware AI control architecture for resilient distribution-grid operation under extreme events, linking prediction, decision-making, and execution.	• Structured integrative review spanning probabilistic resilience modeling, AI-based hazard/outage prediction, adaptive control, and distributed intelligence. • Proposes the layered RACER framework connecting sensing, risk inference, AI-enabled control, safety verification, distributed execution, and human oversight. • Describes how probabilistic risk signals modify objectives, constraints, reserves, switching actions, and restoration priorities.	Conceptual and literature-synthesis study for DER-rich distribution grids and microgrid-interactive networks; includes an illustrative severe-storm operating scenario rather than a benchmark simulation case.	• Integrates previously fragmented streams of risk assessment, forecasting, control, and resilience management into one operational architecture. • Introduces explicit safety-verification and human-supervision layers between AI decisions and physical control actions. • Provides an implementation blueprint and research agenda for risk-conditioned distribution-grid control.	• The framework is conceptual only and is not validated through numerical studies, hardware-in-the-loop experiments, digital twins, or field deployment. • Computational burden, scalability, communication requirements, and interoperability with existing utility platforms remain unverified. • Assumes sufficiently reliable measurements, topology data, and uncertainty estimates, which may be unavailable during severe events.

2.2. Model reduction methods

Measurement-based model reduction provides an established approach for simplifying large-scale power-system models while preserving essential dynamic behavior. The system is typically partitioned into a detailed **study area** and a simplified **external system**, whose influence is captured through dynamic equivalents derived from boundary measurements, as illustrated in Figure 2.1. This formulation enables focused simulations with significantly reduced computational cost.

Representative techniques include ARX-based system identification [7, 22], transfer-function estimation using PMU or FDR data [63, 21], and data-driven black-box models such as ANFIS and recurrent neural networks [54, 52]. The representative studies, model structures, and data requirements for these approaches are summarised in Table 2.2. Collectively, these works demonstrate that measurement-driven equivalents can effectively reproduce the dynamic impact of external areas under a range of operating conditions and establish key principles—such as boundary-focused modeling and scenario-dependent adaptation—that underpin computationally efficient and flexible simulation frameworks.

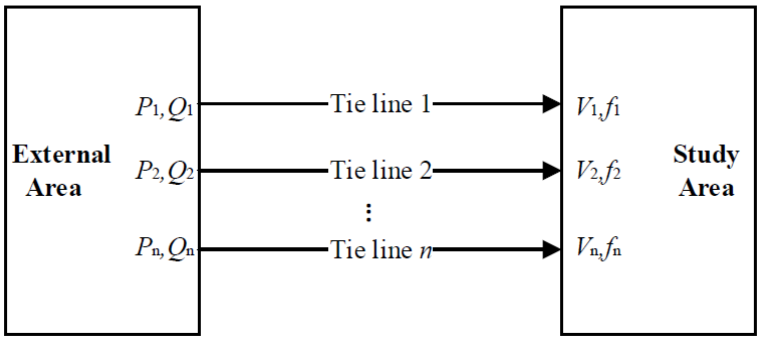


Figure 2.1: Division of the power system into a study area and an external system for measurement-based model reduction [63].

Table 2.2: Summary of related works on measurement-based approaches

Reference	Topic / Focus	Techniques Used	Test System / Case Study	Main Benefits	Limitations
[7]	Develop a measurement-based method to create a dynamic equivalent of an external power system area using autoregressive modeling.	• Use ARX model to capture the relationship between boundaries. • System identification using measurements or PMU data. • Validate using the full system model.	Results shown on 3 systems: • 23-bus Model • NPCC Model • 25,000-bus EI Model	Does not require detailed network model information.	• Valid only near the operating point used for training. • Accuracy depends on measurement quality and event data. • black-box: lacks physical interpretability.
[63]	Develop a measurement-based method to identify dynamic equivalent of the external area using measurements at the boundary nodes.	• System identification using PMU and FDR measurements . • Using the measurements and a Transfer function identification procedure, obtain the TF of the external area. • Replace external area with the Transfer Function. • Evaluate using measurement data and the full system model.	Results shown on Reduced NPCC (140 buses & 48 machines) systems with External Area the NEPOOL system.	• Measurement-based: does not require having full detailed models of the external area. • Higher order Transfer function demonstrated to capture the main system behaviours with a descent accuracy.	• Because the reduced model is built from specific boundary measurements, it may only be valid for the operating conditions/topology under which identification was done. • Being a reduced equivalent, internal details of the external area are lost. • The accuracy depends on the quality and representativeness of the measurement data and the events used for identification.

Reference	Topic / Focus	Techniques Used	Test System / Case Study	Main Benefits	Limitations
[22]	Development of a measurement-based model reduction method using ARX equivalents to represent the external area.	• System identification using PMU and FDR measurements . • Assume linearity of the system, validated by testing the superposition principle on two different events. • Using the measurements and a Transfer function identification procedure, obtain the TF of the external area. • Use an ARX model to improve the parameterization in the transfer-function identification of the external system dynamics. • Compared simulations are performed between the reduced and the original model.	Results shown on NPCC and EI systems.	• By assuming the system is linear, the model can exploit the superposition principle, reducing the need to train on multiple events. • Using ARX models allows a more accurate, parametric representation of the external system compared to a simple transfer-function fit. • No need for detailed modeling of the external system using full first-principles; uses only input-output measurements. • ARX model identification provides a structured way to extract transfer-function parameters from measurements.	• Assumes the external system is linear; may not accurately capture nonlinear effects during large disturbances. • Accurate ARX parameter identification relies on high-quality and sufficiently rich input-output data. • Valid only near the operating point used for training. • black-box: lacks physical interpretability.
[54]	Develop a dynamic equivalent using measurement-based methods and ANFIS.	• Wide-Area measurements obtained by the PMUs and FDRs at the boundaries between the external and the study area. • Use the Takagi-Sugeno based ANFIS model as a black box for the external area. • Trained only on generator trip events • Compared simulations are performed between the reduced and the original model.	Results shown on NPCC system.	• Measurement-based: does not require having full detailed models of the external area. • The ANFIS model predicts in real-time with high accuracy. • Although the model was trained only on generator trip events, it was still able to accurately capture bus fault responses.	• Not evaluated under different operating points. • Experiments conducted on a small-scale system. • Trained on a single type of event. • Acts as a black-box model, lacking physical interpretability.
[52]	Use of RNN models to capture the dynamic behaviour of the external area, enabling its replacement with a data-driven equivalent.	• Use of RNN and Transformer architectures. • Supervised learning approach. • Measurement-based dataset constructed using PMU and FDR data. • Training and evaluation using the full system model.	Results demonstrated on the NPCC system (140 buses & 48 machines)	• Works with a very small dataset (22 training, 12 validation). • Requires relatively low computational resources for training.	The experiments were conducted only on generator and load trip events, limiting the model's exposure to diverse disturbances and reducing its ability to predict unexpected system behaviours.

Reference	Topic / Focus	Techniques Used	Test System / Case Study	Main Benefits	Limitations
[23]	Online tuning of dynamic equivalents for large power systems using wide-area measurements.	• Measurement-based identification. • Online parameter tuning. • Uses wide-area PMU/FDR measurements. • Updates equivalent as conditions change.	Results shown on NPCC system.	• No need for full external-area models. • Adapts to changing system conditions.	• Depends on PMU quality and event coverage. • Accuracy limited by data richness. • Reduced model loses internal system details.
[21]	Use multiple disturbance events to improve dynamic equivalents obtained through measurement-based transfer-function identification.	• System identification via transfer functions. • Uses multiple event types for training. • Measurement data (PMU) from boundary points.	Results shown on NPCC system.	• Better accuracy across different operating conditions. • Reduces dependence on single-event training. • Maintains dynamic fidelity of the study area.	• Requires a variety of event data (multiple events). • Depends on measurement coverage / quality. • Still abstracts away internal structure of the external area.
[10]	Online TSA using WAMS data. Focus on critical contingency identification.	• TSI based on Critical Clearing Time (CCT) prediction. • Training: SVM classifier trained with features (angles, speeds, voltages) from simulated PMU/WAMS data.	WECC 179-bus system with 29 generators.	• Enables fast, online TSA for real-time control decisions. • High classification accuracy for pre-defined critical contingencies.	• High dependency on the comprehensiveness of the training dataset. • Potential poor performance with novel/unforeseen contingencies.
[27]	Online TSA using an approach that is model-free (data-driven) and adaptive, focusing on preventing false alarms.	• An adaptive DT based classifier that uses features from power system measurements and adapts its structure based on new data to maintain accuracy. • Offline training with simulated data, followed by online updating/adaptation of the decision boundaries.	New England 39-bus system.	• Can adapt to system changes (e.g., topology, operating conditions) without full retraining. • Demonstrated high accuracy and low misclassification rates (especially false alarms) under various operating conditions.	• Initial offline training requires extensive simulation data covering various scenarios. • The performance of the adaptive mechanism is critical and needs careful tuning to avoid instability or over-adaptation.
[48]	TSP using a machine learning approach, specifically focusing on Fast, Small-scale Data-driven classification based on PMU measurements.	• NB classifier used for stability prediction. • Uses a small set of features extracted from PMU data (pre-fault/post-fault rotor angles, speeds, voltages) to reduce complexity.	IEEE 50-generator, 145-bus system (North American power system model).	• NB classifier is computationally simple and very fast, making it ideal for real-time applications. • Demonstrates the feasibility of using limited PMU data for online TSA in large systems.	• NB assumes feature independence, which may not hold true for correlated power system measurements, potentially affecting accuracy. • The simplicity may lead to lower accuracy compared to more complex classifiers (like SVM or DT) in highly complex or boundary cases.

Reference	Topic / Focus	Techniques Used	Test System / Case Study	Main Benefits	Limitations
[55]	Online TSA using an optimized machine learning model for high-speed prediction in large interconnected power systems.	• Optimized Extreme Gradient Boosting (XGBoost) algorithm. • Uses features derived from PMU data, including inertia center-based measurements (relative rotor angles/speeds).	• New England 39-bus system • NPCC 140-bus system	• Excellent performance in terms of accuracy, robustness, and speed for online prediction, outperforming traditional ML models like SVM and DT. • Suitable for large-scale systems due to the efficiency of the XGBoost algorithm.	• Hyperparameter tuning of XGBoost is complex and critical for optimal performance. • Requires a large volume of high-quality simulation data for robust training.

2.3. Key Performance Indicators in Dynamic Simulation

Key Performance Indicators (KPIs) are widely used in power-system studies to assess component- and system-level behavior during disturbances. They provide objective, quantitative measures and serve as the basis for decisions such as component classification, contingency screening, and stability assessment. Their role can be summarized as follows:

- **Quantify deviations from steady-state:** KPIs measure the extent to which each component departs from its nominal operating point during a disturbance.
- **Capture stability- and control-related information:** Indicators highlight components critical for overall system stability or proximity to operational limits.
- **Support operational decisions and modeling:** By providing component-level and system-level metrics, KPIs guide model simplification, contingency prioritization, and real-time assessment.
- **Enable multi-indicator approaches:** Using several complementary KPIs ensures that both intrinsic dynamic behavior and operational relevance are captured, enhancing robustness and generalization.

Dynamic activity indicators specifically quantify the intrinsic dynamic response of individual components following a disturbance. These indicators focus on how strongly and persistently a component deviates from its steady-state operating point, independent of broader system-wide stability outcomes. Typical examples include:

- Maximum deviation of key state variables relative to steady-state values.
- Rate of change of states or outputs during the transient response.
- Energy- or variance-based measures computed over predefined time windows.

Dynamic activity concepts have also been exploited to improve computational efficiency in time-domain simulation. Localization-based approaches identify dynamically active components or regions and selectively focus numerical integration effort on those elements. In particular, Aristidou and Van Cutsem [2] show that following a disturbance, only a subset of system components exhibits significant dynamic response, while the remainder stays close to equilibrium. By detecting activity and freezing or simplifying inactive components, substantial simulation speedups can be achieved without globally reducing model order. Although primarily developed for accelerating numerical integration and enabling parallel computation, these methods highlight the practical relevance of component-level dynamic activity measures.

In parallel, a wide class of KPIs has been developed to capture stability- and control-oriented aspects of system behaviour. These indicators identify components or areas that are critical for overall system stability or close to operational limits. Typical examples include:

- Frequency nadir and rate-of-change metrics for frequency stability studies.
- Voltage deviation and recovery time indicators, such as FVSI, LQP, L_{mn}, and LVSI.
- Power-angle deviation or synchronism-related measures for rotor-angle stability studies (TSI, CF, SF, CF/SF).
- Component contributions to system-wide dynamics via participation-factor indicators (Time-variant PF, Wavelet PF, EMT PF boundary).

Because different dynamic phenomena dominate under different operating conditions, adaptive KPI selection is often required:

- Frequency stability studies emphasize frequency- and inertia-related KPIs.
- Voltage stability studies prioritize voltage magnitude and reactive power indicators.
- Multi-domain analyses combine KPIs from multiple categories to capture interacting dynamic effects.

Norm-based performance measures constitute another important class of dynamic metrics and are widely used in control theory and model-reduction literature to quantify discrepancies between full-order and reduced-order systems. Classical approaches employ integral square error and H_2, H_∞, L_1, and

L_∞ norms to preserve dominant dynamic characteristics [11, 57, 14, 3]. Later developments introduce estimation- and optimization-based strategies, including least-squares methods and mixed reduction techniques for linear time-invariant systems [28, 42], as well as heuristic optimization approaches such as genetic algorithms and particle swarm optimization [37, 38, 50]. More recent work combines integral square error minimization with H_∞ constraints to improve robustness [17].

These norm-based measures quantify the magnitude and persistence of dynamic responses and are mathematically related to energy- and variance-based activity metrics. However, within the context of model reduction, they are primarily used as global optimization objectives rather than as explicit indicators for identifying dynamically active components.

A wide range of KPIs spanning dynamic activity, stability, participation, vulnerability, and model-reduction objectives has been proposed in the literature. Representative examples are summarized in Tables 2.3–2.6.

Table 2.3: Representative Voltage Stability Indicators from the Literature

Indicator	Reference	Purpose / Remarks
FVSI, LQP, L_{mn}, LVSI	[29, 41, 24, 8, 9, 20, 5]	Identify weak buses and stressed lines; steady-state; not suited for dynamic activity.
PMU-based indices	[44, 12]	Real-time voltage collapse monitoring; system- or area-level.

Table 2.4: Transient Stability and Coherency Indicators

Indicator	Reference	Purpose / Remarks
TSI	[51, 60]	Post-fault transient stability; system-level.
CF, SF, CF/SF	[66]	Generator coherency; area-level indicators.

Table 2.5: Participation-Factor-Based Indicators

Indicator	Reference	Purpose / Remarks
Time-variant PF	[58]	Time-domain contribution of states; offline analysis.
Wavelet PF	[36]	Frequency-localized dynamic dominance.
EMT PF boundary	[35]	EMT–phasor hybrid modeling relevance.

Table 2.6: Norm-Based Performance Measures in Model Reduction

Indicator / Method	Reference	Purpose / Remarks
Integral square error minimization	[11, 57]	Global time-domain error minimization.
L_1, L_∞, H_2, H_∞ norms	[3, 37, 38]	System-level robustness and energy metrics.
Mixed-method and estimation-based reduction	[28, 42, 17]	Offline reduction with fixed structure.

2.4. Gap Analysis

Although the existing literature on measurement-based model reduction and data-driven dynamic prediction provides important insights into managing power-system complexity, these approaches do not align with the specific objectives of developing a high-fidelity operator-training platform. Model-reduction techniques, in particular, are fundamentally oriented toward simplifying large power systems by replacing extensive network areas with dynamic equivalents derived from boundary measurements. This strategy is well suited to accelerating simulations or reducing computational load in offline dynamic-stability studies, but it runs counter to the requirements of operator training.

For training purposes, the system model must preserve the internal structure and detail of the network in order to maintain consistency with the SCADA view and to expose operators to realistic operational scenarios, alarms, and contingency behaviours. Removing large parts of the system, even if done in a dynamically accurate manner, undermines the realism required for effective operator learning. Thus, while model-reduction research contributes valuable methodologies for simplifying complex systems, it is inherently misaligned with applications in which fidelity and completeness are essential.

A similar misalignment emerges in the literature on data-driven prediction of system dynamics. These studies focus on forecasting transient trajectories or classifying stability outcomes using ML and statistical models, with the aim of approximating system behaviour without executing full-order simulations. This enables faster assessment following disturbances or rapid screening of potential operating points. However, this predictive focus does not address the challenge that is central to operator training: determining the *level of modelling detail* required for a specific training session to operate within real-time constraints while still producing realistic system responses.

Existing predictive methods typically assume a fixed model structure and do not treat model fidelity as an adjustable parameter. As a result, they offer no guidance on how to tailor the system representation so that it remains both computationally feasible and sufficiently detailed for training purposes.

Taken together, these observations reveal a clear gap in current research. While the literature excels at reducing system complexity and predicting dynamic responses, it does not provide a methodology for forecasting the *appropriate granularity* of the system model itself—an essential capability for real-time operator-training environments. What is missing is an approach that dynamically determines how much detail must be retained in the model to preserve operational realism while ensuring that simulations meet real-time execution requirements. Addressing this gap requires a shift in perspective: rather than predicting system dynamics alone, the focus must move toward predicting the modelling fidelity necessary for accurate and responsive operator-training simulations. This gap directly motivates the research direction pursued in this work.

3

Proposed Methodology

Building on the identified research gap, this chapter proposes two complementary methodologies to determine the appropriate modelling fidelity for operator-training simulations. Both approaches start by identifying representative operating points and simulating system behaviour under various contingencies. The first methodology predicts the error and computational time of reduced-order models relative to a full-detail benchmark, guiding the selection of a model that balances accuracy and efficiency. The second methodology predicts which components of the system will be dynamically active for each operating point and contingency, enabling hybrid models that retain full detail only where needed. The key distinction between the two approaches lies in the predicted outputs: the first forecasts model performance, while the second identifies dynamically active system components.

3.1. Methodology Overview

3.1.1. Methodology 1: Model Performance Prediction

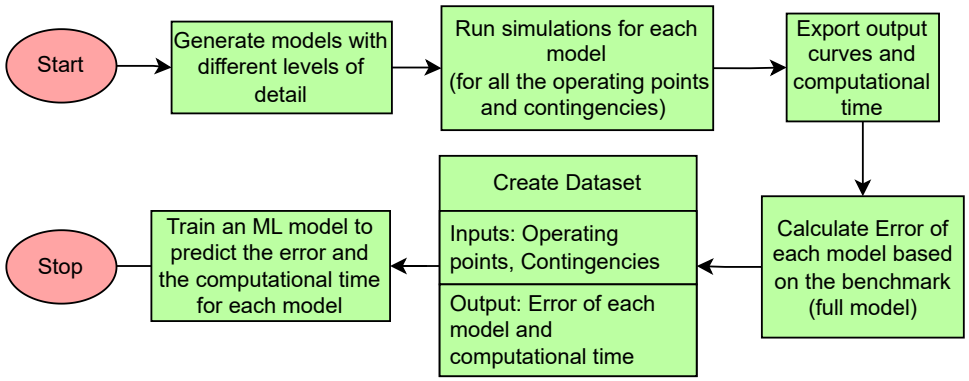


Figure 3.1: Offline process of proposed methodology 1

Figure 3.1 outlines the offline workflow of the first proposed methodology, which can be described through the following stages:

1. Create dynamic models with different levels of detail, using the full-detail model as the benchmark, and save them in a database.
2. Run simulations for all model levels across all operating points and contingencies.
3. Export the output curves and computational times for each model from the simulations.
4. Compute the error of each reduced model relative to the benchmark.

5. Construct a dataset linking operating points and contingencies to model error and computational time.
6. Train a machine learning model to predict for all the reduced-order models the error and computational time.

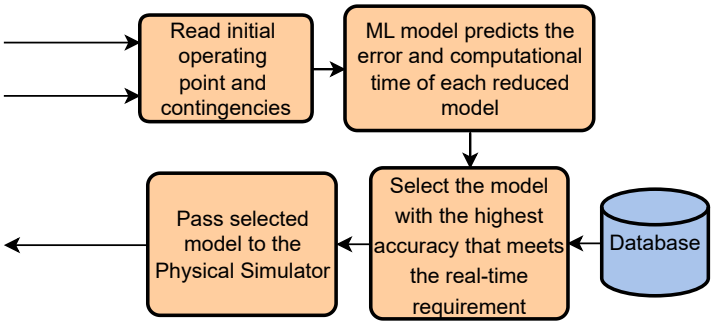


Figure 3.2: Online process of proposed methodology 1

Figure 3.2 outlines the Online workflow of the first proposed methodology, which can be described through the following stages:

1. Read from the Game Master the initial operating point and the contingencies of this training session.
2. The ML model will predict the error and the computational time of each model compared to the full model.
3. From the models inside the database, select the one that meets the real-time requirement and has the highest accuracy.
4. Pass the selected model to the Physical Simulator.

3.1.2. Methodology 2: Dynamic Activity Prediction

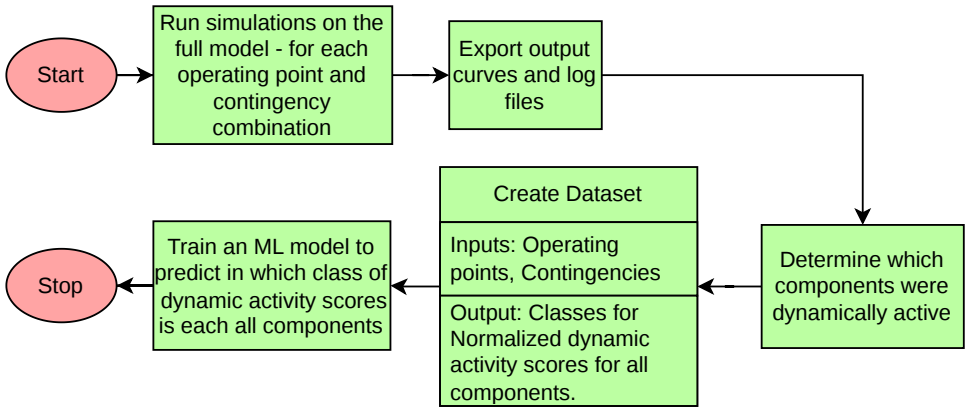


Figure 3.3: Offline process of proposed methodology 2

Figure 3.3 outlines the offline workflow of the proposed methodology, which can be described through the following stages:

1. Run simulations on the full model under all operating points and contingency scenarios.
2. Extract the output curves and log files from the simulations.
3. Compute a normalized activity score for each component in every simulation.

4. Construct a dataset linking operating points and contingencies to the normalized activity scores of each component.
5. Train a machine learning model to predict the normalized activity score for each system component.

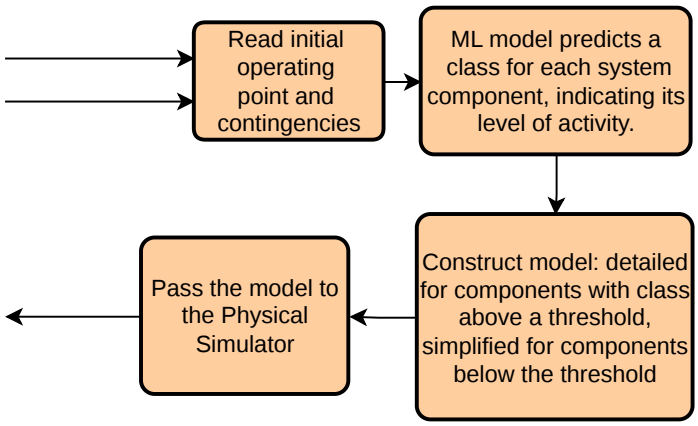


Figure 3.4: Online process of proposed methodology 2

Figure 3.4 outlines the online workflow of the proposed methodology, which can be described through the following stages:

1. Read from the Game Master the initial operating point and the contingencies of this session.
2. The ML model predicts a normalized activity score for all system components.
3. Construct a system model where components with scores above a threshold are modeled in full detail, and the others are simplified.
4. Pass the constructed model to the Physical Simulator.

3.1.3. High Level Comparison of Proposed Methodologies

The first methodology benefits from using pre-tested system models, ensuring that simulations will run successfully and reliably. Its main drawback is the need to generate multiple models with varying levels of detail to achieve good coverage, which can be time-consuming. The second methodology constructs models on the fly by identifying dynamically active system components, avoiding the creation of numerous pre-built models. However, these models are not pre-validated, introducing a risk of simulation failure.

3.1.4. Selected Methodology and Component-Level Dynamic Activity Assessment

Among the two proposed approaches, this work adopts the component-level dynamic activity assessment as the primary methodology for adaptive model construction in real-time operator-training simulations. This choice addresses the research gap identified in Chapter 2: the lack of methods capable of adjusting modeling fidelity at the level of individual components while preserving the full system structure required for operational realism.

The first methodology, based on model performance prediction, remains a viable complementary approach. By selecting from a predefined set of reduced-order models, it offers predictable numerical behavior and guaranteed simulation feasibility. However, its dependence on extensive offline model preparation and fixed model structures limits flexibility when operating conditions, contingencies, or system configurations change. For large-scale systems and diverse training scenarios, maintaining a comprehensive library of reduced models becomes impractical.

In contrast, the selected methodology preserves the full network topology and dynamically adjusts modeling detail by quantifying the expected dynamic contribution of individual components under a

given operating point and contingency. Instead of simplifying entire areas or switching between pre-built models, this approach provides fine-grained control over model fidelity while maintaining consistency with the SCADA view and the internal system structure—both essential for effective operator training.

KPI Calculation

The selected methodology starts by quantifying component-level dynamic activity from post-contingency trajectories. Following the approach of Aristidou and Van Cutsem [2], two KPI groups are monitored in this work: bus voltage magnitude and generator apparent power. For each component i, the KPI is defined as the maximum variance of the relevant trajectory over a moving time window:

$$\begin{aligned} S_i^{(k)} &= \mathrm{Var}\big(y_i(t)\big), \qquad t \in [t_k, t_k + \Delta T_\mathrm{w}], \\ t_{k+1} &= t_k + \Delta T_\mathrm{s}, \qquad S_i = \max_k S_i^{(k)}, \end{aligned} \tag{3.1}$$

where $y_i(t)$ denotes the post-disturbance trajectory of the monitored variable, ΔT_w is the window length, and ΔT_s is the window shift. This definition captures the strongest dynamic excursion of each component during the simulated time horizon. Components that are disconnected, de-energized, or subject to control actions such as protection trips are treated separately as non-simplifiable cases and are not processed through the KPI calculation.

3.1.5. First Approach: GAT-Based Prediction

The first approach represents the initial formulation of the methodology. Starting from the KPI values defined above, it applies group-wise normalization, converts the resulting scores into ordinal activity classes, represents each operating-point–contingency pair as a graph, and trains two graph-attention predictors: one for bus-voltage activity and one for generator apparent-power activity.

Normalization and activity-class construction

To avoid scale bias across heterogeneous physical quantities, normalization is applied separately to each KPI group. A KPI group G collects variables sharing the same physical meaning and units, such as all bus voltage magnitudes or all generator apparent powers. Each group is normalized independently using global min–max scaling over the complete dataset $\mathcal{D}$:

$$\tilde{S}_i^{(G)} = \frac{S_i^{(G)} - \min_{j \in G,\, \mathcal{D}} S_j^{(G)}}{\max_{j \in G,\, \mathcal{D}} S_j^{(G)} - \min_{j \in G,\, \mathcal{D}} S_j^{(G)}}. \tag{3.2}$$

Global normalization preserves absolute severity information across operating points and contingencies. In this work, voltage magnitudes form one KPI group and apparent power forms another. The normalized scores $u \in [0, 1]$ are then discretized into K ordinal classes, each representing an increasing level of dynamic severity, while the highest class is reserved for disconnected or controlled components.

Power-system graph representation

In the first approach, each scenario is represented as a homogeneous graph, illustrated in Fig. 3.5. Buses, generators, and loads are treated as nodes, while transmission lines, transformers, and connection links are treated as edges. This homogeneous formulation is adopted for simplicity and training stability, with component and connection types encoded as features rather than through separate message-passing rules.

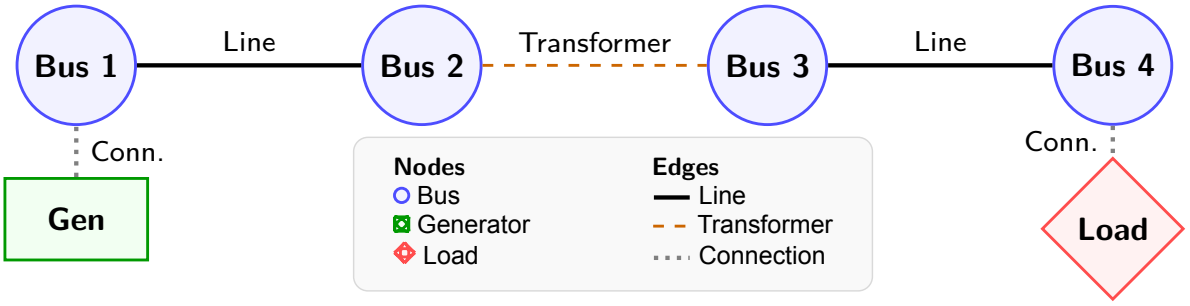


Figure 3.5: Graph construction principle used to map the power system into a homogeneous graph representation.

Each node is described by a 7-dimensional feature vector

$$\mathbf{x}_i = [t_i,\ v_i,\ \theta_i,\ p_i,\ q_i,\ f_i,\ d_i], \tag{3.3}$$

where t_i is the node type, (v_i, θ_i) are the voltage magnitude and angle, (p_i, q_i) are the active and reactive injections, f_i is a binary event indicator, and d_i is the graph-distance feature. The graph-distance term captures the relative position of the component with respect to the disturbance. It is computed as the shortest impedance-weighted distance from the node to the disturbance location using Dijkstra's algorithm and is then log-transformed to reduce skewness and improve numerical stability [56]. Each edge is described by a 4-dimensional attribute vector

$$\mathbf{e}_{ij} = [c_{ij},\ f_{ij},\ r_{ij},\ x_{ij}], \tag{3.4}$$

where c_{ij} is the edge type, f_{ij} is an event flag, and (r_{ij}, x_{ij}) are the branch resistance and reactance.

Training architecture and ordinal regression formulation

The first approach uses two GAT-based predictors trained in parallel: a bus-level voltage model and a generator-level apparent-power model. Both models share the same edge-aware message-passing backbone but differ in their output heads and training masks. The voltage model is trained only on bus nodes, whereas the apparent-power model is trained only on generator nodes. In this way, each predictor is restricted to the component category for which its activity labels are defined.

The graph-attention forward architecture used in this first approach is shown in Fig. 3.6.

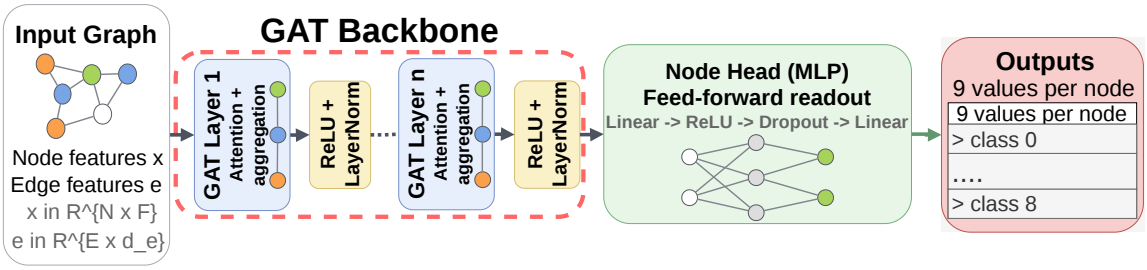


Figure 3.6: Graph attention forward architecture for node-level ordinal classification.

Activity prediction is formulated as an ordinal regression problem rather than as a standard multi-class classification task. The CORAL formulation [6] decomposes the K-class problem into $K-1$ ordered binary tasks and enforces rank-consistent predictions. Let $\mathbf{z}_i \in \mathbb{R}^{K-1}$ denote the logits for sample i, and let $\mathbf{t}_i \in \{0,1\}^{K-1}$ denote the corresponding CORAL encoding. The resulting training objective is

$$\mathcal{L} = \frac{1}{B(K-1)} \sum_{i=1}^{B} \sum_{k=0}^{K-2} \mathrm{BCE}(z_{i,k}, t_{i,k}; w_k) + \lambda \mathcal{L}_{\mathrm{pen}}, \tag{3.5}$$

where

$$w_k = \frac{N_k^-}{\max(N_k^+, 1)} \tag{3.6}$$

corrects ordinal class imbalance. The additional underestimation penalty is computed from the predicted severity level $\hat{y}_i$ and the true label y_i:

$$\delta_i = \max(0, y_i - \hat{y}_i), \tag{3.7}$$

$$\mathcal{L}_{\text{pen}} = \frac{1}{\max(|H|, 1)} \sum_{i \in H} \delta_i^2, \qquad H = \{i : y_i \geq \tau\}. \tag{3.8}$$

This additional term biases the model toward conservative predictions in safety-critical conditions, where underestimating activity is more problematic than overestimating it.

Hyperparameter selection

For the first approach, Optuna (TPE) explores the GAT–CORAL architecture and a subset of decoding/loss knobs, while the remaining training choices are set a priori. Model selection among trials uses a severity-aware validation score

$$\text{Score} = \text{high_recall} + w_{F_1}\, \text{high_}F_1 - w_{\text{loss}}\, \mathcal{L}_{\text{val}}, \tag{3.9}$$

where high_recall and $\text{high_}F_1$ are evaluated on classes at or above a fixed high-class threshold, and $\mathcal{L}_{\text{val}}$ is the validation loss. The coefficients w_{F_1} and w_{loss} belong to this selection rule and are not sampled.

Training uses a fixed batch size, epoch budget, and early-stopping patience, together with the fixed high-class threshold used in the score above. Optuna then searches:

- **Number of GAT layers:** receptive field of message passing.
- **Hidden channels:** width of the GAT embeddings.
- **Number of attention heads:** multi-head interaction patterns.
- **MLP depth and width:** capacity of the CORAL prediction head.
- **Dropout rate:** regularization of the encoder and head.
- **Learning rate** and **weight decay:** Adam step size and ℓ_2 regularization.
- **Underestimation-penalty weight λ:** strength of $\mathcal{L}_{\text{pen}}$.
- **CORAL prediction threshold:** decoding of the ordered binary logits into an ordinal class.

The best checkpoint is retained from the trial and epoch that maximize the severity-aware score. Training-set feature statistics are kept for consistent scaling at deployment.

3.1.6. Second Approach: Pair-Aware GINE-Based Prediction

The first approach established a complete graph-based workflow, but it also highlighted two limitations. First, the normalization stage introduced an additional transformation between the KPI and the final activity class, even though the KPI itself already reflects dynamic severity on a physically meaningful scale. Second, the prediction model treated all components within one graph-processing pipeline without explicitly modeling the interaction between a specific target component and the contingency being evaluated. The second approach was therefore introduced as a revised formulation that addresses these issues while preserving the same overall objective of predicting component-level dynamic activity for adaptive model construction.

As in the first approach, KPI calculation is performed beforehand and is therefore not repeated here. The main changes concern how the activity classes are constructed and how the prediction model conditions each estimate on the electrical context of the disturbance and on the target component itself.

Power-system graph representation

In the revised formulation, each operating-point–contingency scenario is represented as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where buses, generators, and loads form the node set and lines, transformers, and connection links form the edge set. The node description is made explicitly contingency-dependent so that the

same component can be interpreted differently under different disturbances. Under contingency c, the node feature vector is

$$\mathbf{x}_i = \big[t_i,\, v_i,\, \theta_i,\, p_i,\, q_i,\, f_i,\, d_{i,c}\big], \tag{3.10}$$

where t_i denotes the node type, v_i the steady-state voltage, θ_i the angle, (p_i, q_i) the active and reactive injections, f_i a binary fault indicator, and $d_{i,c}$ the electrical distance to the contingency.

An important element of this representation is the electrical-distance feature, which encodes the relative position of each component with respect to the disturbance. It is derived from the bus-admittance matrix $\mathbf{Y}$. First, the corresponding bus-impedance matrix is obtained as

$$\mathbf{Z} = \mathbf{Y}^{-1}, \tag{3.11}$$

and the electrical distance between buses i and j is defined as

$$d_{ij} = |Z_{ij}|. \tag{3.12}$$

For a node contingency at bus a, the distance of node i to the contingency is

$$\bar{d}_{i,c} = d_{ia}, \tag{3.13}$$

whereas for an edge contingency between buses u and v, it is defined as

$$\bar{d}_{i,c} = \min(d_{iu}, d_{iv}). \tag{3.14}$$

Finally, a logarithmic transform is applied to compress the spread of large distances and improve numerical stability:

$$d_{i,c} = \log(1 + \bar{d}_{i,c}). \tag{3.15}$$

The same contingency-aware representation is used for edges. For edge (i, j), the feature vector is

$$\mathbf{e}_{ij} = \big[c_{ij},\, f_{ij},\, r_{ij},\, x_{ij},\, b_{1,ij},\, g_{1,ij},\, b_{2,ij},\, g_{2,ij}\big], \tag{3.16}$$

where c_{ij} denotes the edge type, f_{ij} the fault flag, and the remaining entries are series and shunt parameters.

Physically aware cut selection and label construction

Scenarios are split at the operating-point level into train, validation, and test sets so that all contingencies of one operating point remain together. The revised formulation then assigns activity classes directly from the KPI scale, avoiding the additional normalization stage used in the first approach. Let $x \geq 0$ be a KPI and let $\boldsymbol{\tau} = (\tau_0, \ldots, \tau_{B-1})$ be an ordered cut set with $0 < \tau_0 < \cdots < \tau_{B-1}$. The corresponding ordinal class is defined as

$$c(x; \boldsymbol{\tau}) = \sum_{m=0}^{B-1} \mathbb{I}(x > \tau_m). \tag{3.17}$$

With solver precision k, a trajectory within numerical tolerance yields variance at most k^2; the inactive cut is therefore fixed as

$$\tau_0 = \alpha\, k^2, \qquad \alpha \geq 1. \tag{3.18}$$

The inactive threshold alone is not sufficient to define the full severity structure. The remaining cuts are therefore selected on the training scenarios through a multi-objective procedure that balances three requirements: usable class proportions, physical consistency with the observed trajectories, and coherence at scenario level.

Let $\mathbf{p} = (p_1, \ldots, p_K)$ denote the shares of the K KPI classes. Usability is evaluated first through the class distribution. The normalized entropy term is

$$H(\mathbf{p}) = -\frac{1}{\log K} \sum_{j=1}^{K} p_j \log p_j, \tag{3.19}$$

the class-floor term is

$$U_{\text{floor}}(\mathbf{p}) = \min_{j\in\{2,\dots,K\}} \min\left(1, \frac{p_j}{p_j^{\min}}\right), \tag{3.20}$$

and the weakest-class penalty term is

$$U_1(\mathbf{p}) = \begin{cases} 1, & p_1 \le p_1^{\max}, \\ \min\left(1, \dfrac{1-p_1}{1-p_1^{\max}}\right), & \text{otherwise.} \end{cases} \tag{3.21}$$

These are combined into the usability score

$$U = 0.45\, H(\mathbf{p}) + 0.35\, U_{\text{floor}}(\mathbf{p}) + 0.20\, U_1(\mathbf{p}). \tag{3.22}$$

Beyond class balance, the cuts must also preserve the physical ordering of the responses. For this reason, physical consistency is evaluated through the following morphological severity score:

$$\varphi = \log_{10}\left(\eta + E + \sqrt{\max(V,0)} + \lambda_D D\right), \tag{3.23}$$

where E is the peak excursion from the pre-event mean, V the post-event variance, D the settling residual, $\eta = 10^{-12}$, and $\lambda_D = 0.5$. Given ordinal KPI activity classes c, the physical-consistency score is

$$P = 0.65\, \max\big(\rho(\varphi, c), 0\big) + 0.35\, M(\varphi, c), \tag{3.24}$$

where ρ is the Spearman correlation and M is the fraction of consecutive class-mean pairs with non-decreasing mean φ. A third criterion is introduced at scenario level through the Spearman correlation R between each scenario's 95th-percentile KPI and its mean ordinal activity class. The final combined score is

$$J = 0.40\, U + 0.40\, P + 0.20\, \max(R, 0). \tag{3.25}$$

The final cut set is selected from the Pareto front of nondominated solutions with respect to (U, P), after which action and disconnection cases are assigned the flag class $C-1$.

Pair-aware GINE architecture

The prediction model is also revised. Instead of relying on a single graph-processing path followed by an ordinal head, the second approach uses two independent graph isomorphism networks with edge features: one for bus-voltage activity and one for generator apparent-power activity. Their shared architecture is shown in Fig. 3.7. Continuous node and edge features are first mapped into a common hidden space, after which residual GINE layers, jumping-knowledge aggregation, event encoding, graph-level pooling, and pair-aware fusion are used to produce contingency-specific predictions for each target component.

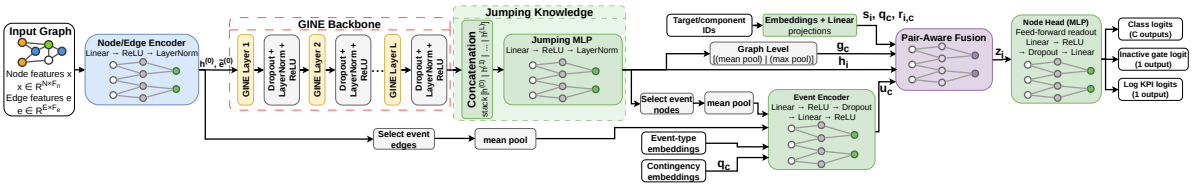


Figure 3.7: Architecture of the pair-aware GINE model for component-level dynamic activity prediction.

Let $\mathcal{N}(i)$ denote the set of neighbors of node i, let $\mathbf{h}_i^{(l)}$ denote the node embedding at layer l, and let $\tilde{\mathbf{e}}_{ij}$ denote the encoded edge attributes. Message passing is then performed through the GINE update

$$\mathbf{h}_i^{(l+1)} = \mathrm{MLP}^{(l)}\Big((1+\epsilon^{(l)})\mathbf{h}_i^{(l)} + \sum_{j\in\mathcal{N}(i)} \phi\big(\mathbf{h}_j^{(l)} + \tilde{\mathbf{e}}_{ij}\big)\Big), \tag{3.26}$$

with a two-layer MLP of the form

$$\mathrm{MLP}^{(l)}(\mathbf{z}) = \mathbf{W}_2^{(l)}\, \xi\Big(\mathbf{W}_1^{(l)}\mathbf{z} + \mathbf{b}_1^{(l)}\Big) + \mathbf{b}_2^{(l)}. \tag{3.27}$$

After L layers, jumping-knowledge concatenation and a linear projection produce the target embedding $\mathbf{h}_i$, which is then combined with disturbance-specific context.

For each contingency c, the graph-level context is summarized as

$$\mathbf{g}_c = \big[\mathrm{mean}(\{\mathbf{h}_j\}),\ \max(\{\mathbf{h}_j\})\big], \tag{3.28}$$

and the interaction between target and contingency identity embeddings is

$$\mathbf{r}_{i,c} = \tanh\big(\mathbf{W}_s\mathbf{s}_i \odot \mathbf{W}_q\mathbf{q}_c\big). \tag{3.29}$$

The interaction between the target component and the disturbed region is then collected in the pair-aware fusion vector

$$\boldsymbol{\delta}_{i,c} = \big[\mathbf{h}_i,\ \mathbf{u}_c,\ \mathbf{g}_c,\ |\mathbf{h}_i - \tilde{\mathbf{u}}_c|,\ \mathbf{h}_i \odot \tilde{\mathbf{u}}_c,\ d_{i,c}\big], \tag{3.30}$$

and the decoder input is

$$\mathbf{z}_{i,c} = \big[\boldsymbol{\delta}_{i,c},\ \mathbf{s}_i,\ \mathbf{q}_c,\ \mathbf{r}_{i,c}\big]. \tag{3.31}$$

A shared MLP maps this fused representation to three outputs:

$$\boldsymbol{\ell}_{i,c} = g_{\mathrm{cls}}(\mathbf{z}_{i,c}),\ a_{i,c} = g_{\mathrm{gate}}(\mathbf{z}_{i,c}),\ \hat{z}_{i,c} = g_{\mathrm{reg}}(\mathbf{z}_{i,c}), \tag{3.32}$$

namely class logits, an inactivity-gate logit, and a standardized log-KPI estimate.

Training objective, model selection, and inference

Training combines class supervision, inactivity detection, and auxiliary regression. Let $y_{i,c} \in \{0, \ldots, C-1\}$ denote the target class, with KPI classes $0, \ldots, C-2$ and flag class $C-1$. The gate target is $\mathbb{I}(y_{i,c} = 0)$. Where a finite KPI exists, the regression target is the train-set standardized value

$$z^{\star}_{i,c} = \big(\log_{10}(S_i + \varepsilon) - \mu\big)/\sigma, \tag{3.33}$$

with training mean μ and standard deviation σ. The total training loss is

$$\mathcal{L} = w_{\mathrm{cls}}\mathcal{L}_{\mathrm{CE}} + w_{\mathrm{reg}}\mathcal{L}_{\mathrm{Huber}} + w_{\mathrm{gate}}\mathcal{L}_{\mathrm{BCE}} + w_{\mathrm{ord}}\mathcal{L}_{\mathrm{CDF}}. \tag{3.34}$$

After each validation epoch, the three decoding paths are evaluated jointly through the validation score

$$\mathrm{Score} = 0.4\,\mathrm{BA} + 0.3\,F_1^{\mathrm{m}} + 0.2\,\mathrm{Acc} + 0.1\,\mathrm{Acc}_1, \tag{3.35}$$

using balanced accuracy, macro-F_1, exact accuracy, and within-one-class accuracy. The best epoch and decode path are retained for each trial, and the winning hyperparameter configuration is finally retrained on the union of the training and validation sets.

At inference, one operating-point–contingency graph is prepared using the same construction as in training, and the forward pass yields $\boldsymbol{\ell}_{i,c}$, $a_{i,c}$, and $\hat{z}_{i,c}$. The checkpointed decode mode then selects one of the following outputs:

- **Class head:** $\hat{y}_{i,c} = \arg\max_k\ \ell^{(k)}_{i,c}$.
- **Inactivity gate:** if $\mathrm{sigmoid}(a_{i,c}) \geq \gamma$, assign class 0; otherwise use $\hat{y}_{i,c} = \arg\max_{k\geq 1}\ \ell^{(k)}_{i,c}$.
- **Log-KPI path:** denormalize $\hat{z}_{i,c}$ to a KPI value, map it through $\boldsymbol{\tau}$ to $\{0, \ldots, C-2\}$, and keep the flag whenever $\arg\max_k \ell^{(k)}_{i,c} = C-1$.

These decoded component-level predictions are then passed to the shared simplification stage.

Hyperparameter selection

For the second approach, training settings and Optuna search knobs are separated in the DYNAGNN configuration. Fixed a priori (under `training` / `training.pair_aware`) are the batch size, epoch budget, early-stopping patience, multi-task loss weights $w_{\mathrm{cls}}, w_{\mathrm{reg}}, w_{\mathrm{gate}}, w_{\mathrm{ord}}$, class- and gate-weighting modes, inactivity-gate threshold γ, KPI floor ε, decode-path policy, and the validation-score mix defined above. Optuna (TPE) then samples only the entries under `optuna.hparams`:

- **Hidden dimension H:** capacity of node and edge embeddings.
- **Number of GINE layers L:** neighborhood depth.
- **Type-embedding dimension:** component and edge type capacity.
- **Node-identity embedding dimension:** target-component identity capacity.
- **Contingency-identity embedding dimension:** fault-identity capacity.
- **Pair-embedding dimension:** target–contingency interaction capacity.
- **Decoder width:** shared node-head width.
- **Dropout rate:** encoder/decoder regularization.
- **Learning rate** and **weight decay:** AdamW step size and ℓ_2 regularization.

Voltage and apparent-power models are tuned independently on the same protocol. For the Nordic 32 architecture comparison in Chapter 4, hyperparameters are selected by 5-fold cross-validation on the train and validation operating points: each Optuna trial is scored by the mean fold validation selection score, and the best trial is then retrained once on those operating points. The French transfer uses the same fixed training settings with an adjusted Optuna budget and search ranges.

3.1.7. Shared Substation-Level Simplification

The online application of either approach is the same and is summarised in Fig. 3.8. Given the current operating point and contingency description, the system state is converted into the graph representation of the selected approach and features are scaled with the corresponding training-set statistics. The voltage model and the apparent-power model then run in parallel: the former assigns activity classes to buses and the latter to generators. Those component-level predictions are aggregated at substation scope and used for Adaptive Model Simplification, without an additional time-domain simulation.

Let $\hat{y}_i^{(\mathrm{V})}$ and $\hat{y}_j^{(\mathrm{S})}$ denote the decoded classes of bus i and generator j, and let $\mathcal{B}_s$ and $\mathcal{G}_s$ denote the buses and dynamically modelled generators of substation s. Given a threshold ϵ, substation s retains full node-breaker detail if any associated component satisfies $\hat{y}_i^{(\mathrm{V})} \geq \epsilon$ or $\hat{y}_j^{(\mathrm{S})} \geq \epsilon$; otherwise it is converted to bus-breaker form. In this way, both approaches share the same downstream simplification rule even though they differ in how the component-level activity classes are predicted.

A simplified run is compared with the Full model on solving time and on voltage error. The interval $t < 9\,\mathrm{s}$ is initialization, so solving time is counted only for $t \geq 9\,\mathrm{s}$. $\|T_{\mathrm{sol}}\|_\infty$ is the largest step solve time in that window, in seconds. RT Viol. counts the steps in that window whose solve time exceeds $1\,\mathrm{s}$, the simulated step. The run is real time when RT Viol. $= 0$, that is, when $\|T_{\mathrm{sol}}\|_\infty \leq 1\,\mathrm{s}$. Voltage error is the DTW RMSE [15] of bus voltage in pu. FastDTW does not normalise the curves: it returns only the alignment, with absolute sample difference as the local cost, and the RMSE is then computed on the paired samples. The reported value is the maximum over buses. Because the voltage is in pu, this RMSE is in pu.

3.2. Data requirements

- Full dynamic model – needed as a benchmark. Include seasonal topology changes because topology affects predictions.
- Representative operating points – or data to generate them, needed to cover typical system conditions for realistic predictions.
- Variety of contingencies – e.g., all N-1 scenarios, needed to capture system behaviour under different disturbances.

3.3. Implementation

The pipeline is implemented in Python 3 and driven by Dynaωo [13] for all time-domain simulations. The main scientific stack used across both approaches is:

- **PyTorch** and **PyTorch Geometric** for GAT–CORAL and pair-aware GINE training and inference;
- **Optuna** for Tree-structured Parzen Estimator (TPE) hyperparameter search;

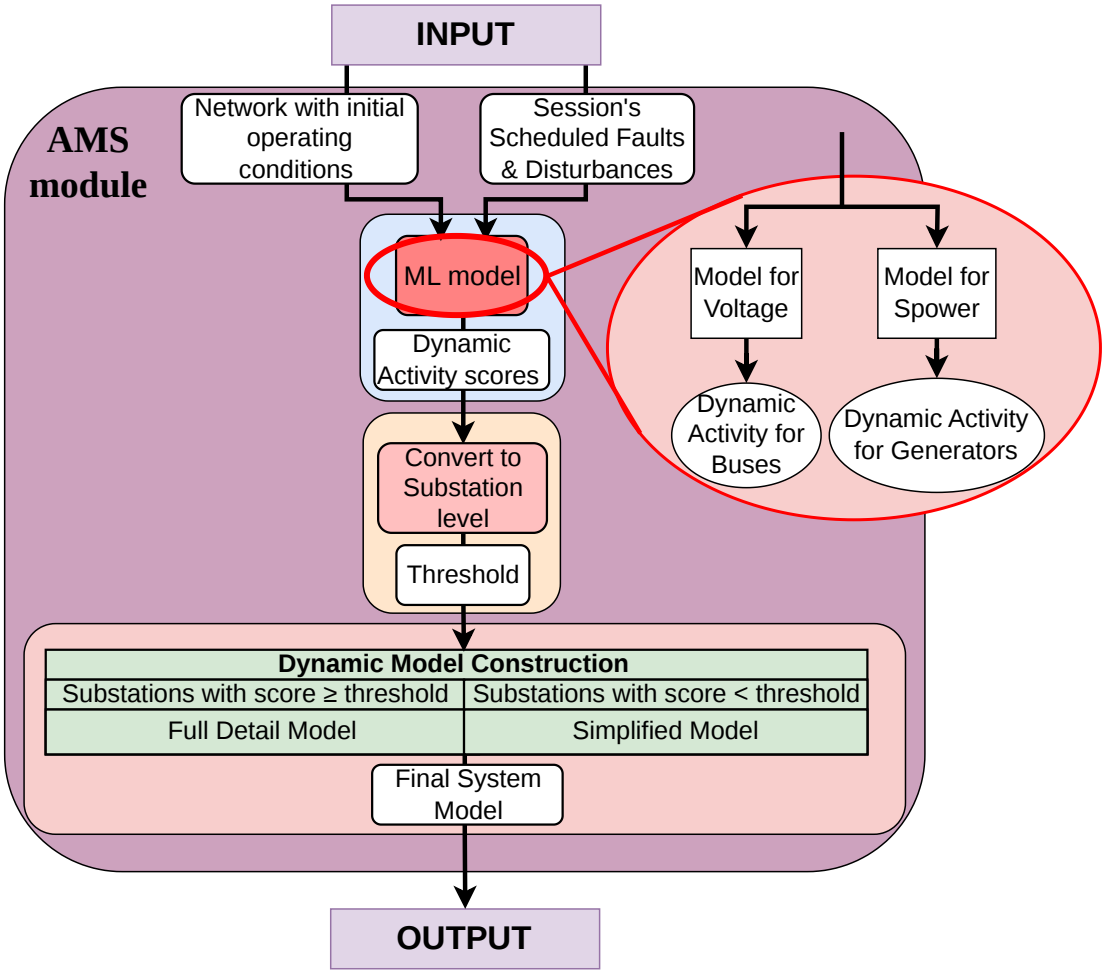


Figure 3.8: AMS methodology diagram with parallel voltage and apparent-power models and substation-level conversion.

- **NumPy** and **pandas** for KPI tables, splits, and I/O (CSV/Parquet/JSON);
- **SciPy** (sparse linear algebra) for bus-admittance / impedance distances in the second approach;
- **pandapower** to load IIDM networks when building operating-point graphs;
- **fastdtw** for Dynamic Time Warping alignment of simplified versus full trajectories.

Representative code fragments for the steps below are collected in Appendix A. This section keeps only the workflow description and the numerical choices needed to reproduce the experiments.

3.3.1. Operating points and contingencies

French transmission system.

The French benchmark uses 61 operating points spanning peak load (∼80 GW), a stressed France–Spain border snapshot, minimum load (∼30 GW), regional load-scaled variants of those bases, a winter worst-case close to OP 1, one summer day of hourly snapshots (OPs 12–35), two high renewable cases (OPs 36–37), and one winter day of hourly snapshots (OPs 38–61); Table 3.1 lists the set. Regional scaling marks area loads as controllable and couples them to Dynawo's load-variation macros (Appendix A). Contingencies cover buses, lines, and generators with OP-dependent filtering, yielding 73 924 valid operating-point–contingency pairs (Table 3.2). The full-detail French model contains approximately 319 880 continuous variables, 334 848 discrete variables, and 276 151 root functions.

OP	Snapshot / scenario (summary)
1	2025-01-14 09:00 — peak load in 2025 (∼80 GW).
2	2025-09-08 18:00 — stressed context for France–Spain border studies.
3	2025-09-21 05:00 — minimum load in 2025 (∼30 GW).
4	Scaled from OP 1: North -20%, Normandy–Paris -30%.
5	Scaled from OP 1: North +10%, South-East +20%.
6	Scaled from OP 2: +10%.
7	Scaled from OP 2: +20%.
8	Scaled from OP 2: +30%.
9	Scaled from OP 2: +20% and West + South-West +40%.
10	Scaled from OP 2: +20% and West + South-West +50%.
11	2025-01-14 08:45 — winter worst-case snapshot close to OP 1.
12–35	Hourly snapshots on 2025-07-18, from 00:00 to 23:00.
36	2025-07-21 14:10 — high wind and solar conditions (around 40%).
37	2025-09-03 10:30 — high wind and solar conditions (around 40%).
38–61	Hourly snapshots on 2026-02-19, from 00:00 to 23:00.

Table 3.1: French-system operating points used in the implementation.

Table 3.2: Grouped French contingency counts by operating-point set.

OP group	# Bus	# Generator	# Line	Total
1, 4–5	283	96	856	1235
2, 6–10	283	59	859	1201
3	283	53	859	1195
11	283	95	856	1234
12, 20	283	64	858	1205
13, 22–23	283	60	858	1201
14, 24–25	283	58	858	1199
15	283	56	858	1197
16, 18	283	54	858	1195
17	283	53	858	1194
19, 21	283	61	858	1202
26–27	283	57	858	1198
28	283	59	858	1200
29	283	60	858	1201
30	283	62	858	1203
31	283	63	858	1204
32, 34	283	68	858	1209
33, 35	283	66	858	1207
36	283	60	857	1200
37	283	58	859	1200
38, 45	283	78	859	1220
39, 41–42	283	76	859	1218
40, 43	283	75	859	1217
44	283	77	859	1219
46, 48, 50, 53–54, 60	283	82	859	1224
47, 55	283	84	859	1226
49	283	83	859	1225
51, 61	283	80	859	1222
52	283	81	859	1223
56, 59	283	86	859	1228
57–58	283	89	859	1231

Nordic 32 test system.

The Nordic workflow generates 35 operating points from a single reference case by scripted load scaling. The train/validation/test assignment is fixed at the operating-point level before sampling (25/5/5), with each split covering low-, medium-, and high-loading bands. With global factor g, area multiplier $a_{A(i)}$,

and per-load factor f_i,

$$s_i = g \cdot a_{A(i)} \cdot f_i, \qquad P_i = P_i^0 s_i, \qquad Q_i = Q_i^0 s_i, \qquad P_{\text{target}} = \sum_{i=1}^{22} P_i^0 s_i. \tag{3.36}$$

After ramping the load deltas, a short equilibrium run replaces the initial IIDM. Table 3.3 lists the resulting points; Table 3.4 summarises the contingency inventory (7 419 valid pairs over lines, busbar sections, generators, loads, and transformers).

Table 3.3: Nordic operating points used in the implementation.

OP	Split	Band	Pattern	Target P (MW)
1	test	low	`uniform`	9016.88
2	train	medium	`central_up`	9780.31
3	train	low	`north_up`	8964.4
4	train	high	`south_up`	10634.69
5	train	high	`north_central_stress`	10841.2
6	test	medium	`mixed`	10037.99
7	validation	low	`uniform`	8986.15
8	validation	medium	`central_up`	9655.28
9	train	high	`north_up`	11140.25
10	train	high	`south_up`	10675.03
11	train	high	`north_central_stress`	10750.3
12	train	low	`mixed`	8962.74
13	train	medium	`uniform`	9851.1
14	train	high	`central_up`	10392.03
15	train	low	`north_up`	9091.6
16	validation	high	`south_up`	10239.01
17	test	high	`north_central_stress`	10719.6
18	train	low	`mixed`	8510.63
19	test	low	`uniform`	9249.76
20	train	high	`central_up`	10701.03
21	train	low	`north_up`	8716.37
22	train	low	`south_up`	9380.54
23	test	medium	`north_central_stress`	10034.11
24	train	medium	`mixed`	9766.86
25	train	low	`uniform`	9250.99
26	train	high	`central_up`	10616.87
27	validation	high	`north_up`	10376.14
28	train	medium	`south_up`	9827.71
29	train	low	`north_central_stress`	9004.13
30	train	high	`mixed`	10829
31	train	low	`uniform`	9161.9
32	validation	medium	`central_up`	10316.34
33	train	medium	`north_up`	10294.06
34	train	medium	`south_up`	10595.83
35	train	low	`north_central_stress`	9888.03

Table 3.4: Grouped Nordic contingency counts by operating-point set.

OP group	# Bus	# Generator	# Load	# Line	# Transformer	Total
1, 3, 7, 12, 15, 21–22, 25, 31	72	20	22	52	50	216
2, 6, 16	69	20	22	52	50	213

OP group	# Bus	# Generator	# Load	# Line	# Transformer	Total
4	63	18	22	51	48	202
5, 14, 17	67	19	22	52	49	209
8, 13, 24, 27–29, 35	71	20	22	52	50	215
9	55	13	21	51	42	182
10, 20, 33	68	20	22	52	50	212
11	65	17	22	52	47	203
18	74	20	22	52	50	218
19, 23, 26, 32, 34	70	20	22	52	50	214
30	60	16	21	51	45	193

3.3.2. Full-detail simulation and KPI labelling

For each valid pair, the case is copied, the contingency is injected, and Dynawo is launched with event time $t_{\text{event}} = 10\,\text{s}$. Voltage and generator P, Q curves are exported to CSV; apparent power uses $S = \sqrt{P^2 + Q^2}$ and, where available, the generator S_{nom} for per-unit normalization. The sliding-window KPI of Chapter 3 is evaluated on the post-event window from 9 s to 100 s with $\Delta T_{\text{w}} = 5\,\text{s}$ and $\Delta T_{\text{s}} = 1\,\text{s}$. Disconnection flags are inferred from sustained near-zero voltage; action flags from protection/control timelines with $t \geq 10\,\text{s}$. Flagged components receive the reserved highest class independently of the KPI ladder. Supporting routines appear in Appendix A.

3.3.3. First approach (GAT--CORAL)

On the French system, the first-approach experiments reported in Chapter 4 use operating points 1–10 with an 80%/10%/10% scenario split after labelling. KPIs are group-wise min–max normalized to $[0, 1]$ and cut into nine equal ordinal bins (cuts at $k/9$ for $k = 1, \ldots, 8$); the flag class is 9. Homogeneous OP graphs carry the seven node features and four edge attributes of the methodology, with an impedance-weighted Dijkstra distance $\log(1+d)$ to the disturbance. Two masked GAT–CORAL models (voltage on buses, apparent power on generators) are trained with Optuna as specified in Chapter 3: fixed batch size, epoch budget, patience, high-class threshold, and selection weights; searched GAT width/depth, heads, MLP size, dropout, learning rate, weight decay, underestimation penalty λ, and CORAL decode threshold. Analytic listings are in Appendix A.

3.3.4. Second approach (pair-aware GINE)

Operating-point graphs are built from IIDM/`.dyd` with pandapower; electrical distances use $\mathbf{Z} = \mathbf{Y}^{-1}$ and $d_{i,c} = \log(1+\bar{d}_{i,c})$. Physically aware cuts are selected on training operating points only; the selected ladders used for labelling and training are those of Table 4.6 in Chapter 4 (inactive cut $\tau_0 = 10^{-7}$). Nordic 32 uses the 25/5/5 operating-point split above; the French transfer uses the same contingency philosophy with an approximately 80%/10%/10% operating-point split (49/6/6). Two independent pair-aware GINE models follow the fixed training settings and Optuna search of Chapter 3 (batch size, epochs, patience, loss weights, γ, and validation-score mix fixed; architecture and optimizer knobs searched). Code for cut scoring, graph construction, the PairAwareGINE module, and the Optuna loop is in Appendix A.

3.3.5. Adaptive Model Simplification and comparison

Decoded bus and generator classes are aggregated per substation. With retention threshold ϵ, a substation keeps node-breaker detail if any associated prediction is at least ϵ, or if it hosts the fault/action; otherwise it is converted to bus-breaker form by writing IIDM switch retention flags. Solving time and voltage DTW RMSE follow the definitions in Shared Substation-Level Simplification above (implemented with `fastdtw` in `modules/max_dtw_rmse.py`). Model size is read from Dynawo logs (continuous, discrete, and root-function counts). The Regional baseline retains the event location and its third-order graph neighbourhood. Dynawo's ideal switch implementation uses $n_g^{(\text{sw})} = 0$ root functions per switch, so root-count changes under AMS follow from the set of active components rather than from per-switch roots (Appendix A).

4

Results

This chapter reports the empirical results in the same order as the methodology and implementation. The KPI calculation is illustrated first. The remainder of the chapter covers the first approach—class construction, test-set prediction, and simplification testing—and then the second approach: physically aware cut selection, Nordic 32 prediction and architecture comparison, scalability on the French transmission system, and adaptive simplification.

4.1. KPI calculation

Dynamic activity is quantified as the maximum sliding-window variance of each post-contingency trajectory, using window length $\Delta T_{\mathrm{w}} = 5\,\mathrm{s}$ and step $\Delta T_{\mathrm{s}} = 1\,\mathrm{s}$ (Chapter 3). The construction is illustrated in Fig. 4.1 on an apparent-power signal after a line disconnection.

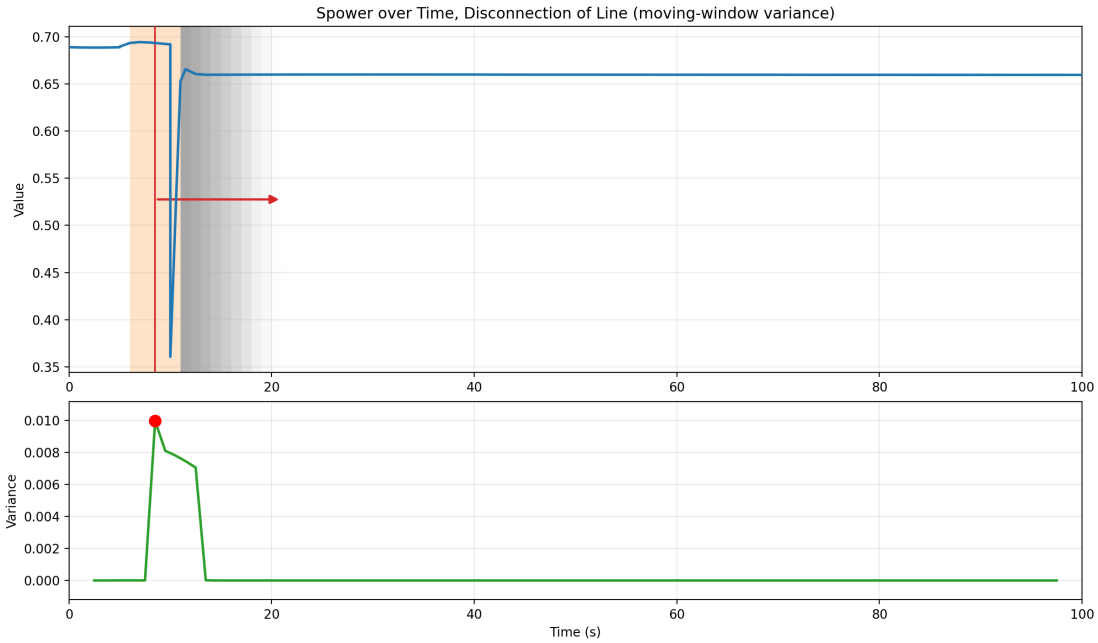


Figure 4.1: Sliding-window variance KPI for a line-disconnection case.

Before the disturbance the monitored signal is nearly flat, so every $5\,\mathrm{s}$ window has a negligible variance. Once the disconnection occurs, the trajectory undergoes a sharp transient and any window that overlaps that interval records a large intra-window variance. In Fig. 4.1, the orange band identifies the window of maximum variance and the red vertical line its centre; the fading grey bands mark later

placements as the analysis advances in $1\,\mathrm{s}$ steps. The lower panel reports the variance against window centre time: it stays near zero in quiescent intervals, rises when the window enters the transient, peaks at the red marker, and returns to zero after settling. The KPI retained for labelling is this peak value. Larger KPIs therefore indicate stronger short-term fluctuations inside the analysis window, not merely a static offset from the pre-event level.

4.2. Results of the First Approach

The first approach implements the GAT–CORAL pipeline of Chapter 3: group-wise min–max normalisation of the KPIs, fixed ordinal cuts on the unit interval, graph construction with an impedance-weighted distance feature, and two masked predictors for voltage and generator apparent power. The following subsections report the resulting labels, the prediction quality on the held-out test set, and the effect of the ensuing adaptive simplification.

4.2.1. Class construction and label distributions

After group-wise normalisation to $[0, 1]$, continuous KPI scores are discretised into nine ordered bins that produce classes 0–8. Class 9 is reserved for disconnected or controlled components and is assigned from the flag tables independently of the KPI cuts. The same cut set is used for voltage and apparent power; the resulting intervals are listed in Table 4.1.

Table 4.1: First-approach ordinal classes on the normalised KPI scale.

Class	Normalised KPI interval
0	$[0,\ 1/9)$
1	$[1/9,\ 2/9)$
2	$[2/9,\ 3/9)$
3	$[3/9,\ 4/9)$
4	$[4/9,\ 5/9)$
5	$[5/9,\ 6/9)$
6	$[6/9,\ 7/9)$
7	$[7/9,\ 8/9)$
8	$[8/9,\ 1]$
9	disconnection / action flag

On the resulting ordinal scale, lower classes correspond to near-quiescent responses, while higher classes capture stronger excursions and sustained fluctuations (Fig. 4.2). Class 9 covers components that leave the continuous KPI pathway because of switching or protection. This mapping is the reason the first approach trains ordinal predictors rather than regressors on the raw KPI: the learning target is a ranked activity level that can be thresholded for simplification.

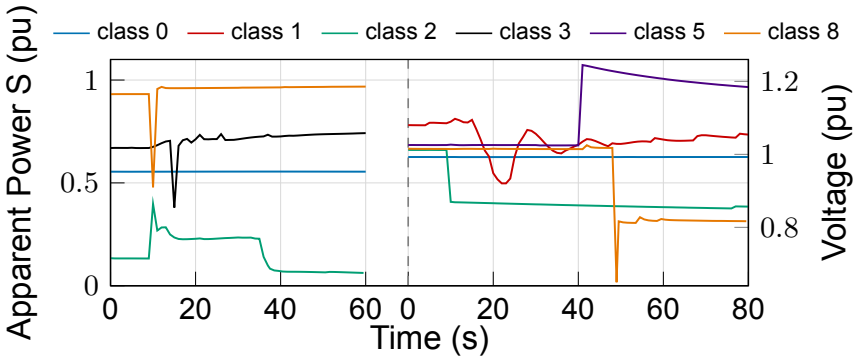


Figure 4.2: Example trajectories for different activity classes.

The empirical frequencies on the French voltage and apparent-power datasets are strongly skewed toward the inactive and low-activity bins, with comparatively few mid-to-high severity samples and a

separate mass in Class 9 from flagged events (Fig. 4.3). That imbalance motivates the severity-aware training objective and the asymmetric underestimation penalty: rare high-activity cases must not be systematically missed, even if that increases conservative overestimation on quiescent components.

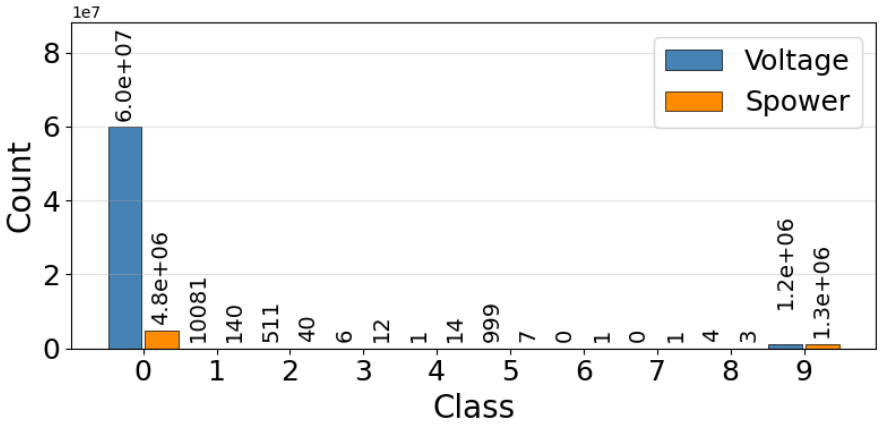


Figure 4.3: Class distribution of the first-approach datasets.

4.2.2. Prediction outcomes on the test set

Both models are evaluated on the held-out scenarios of the first-approach split. Correctness, underestimation, overestimation, and mean absolute ordinal error (MAE) are reported in Table 4.2 at node level and after max-based aggregation to substations.

Table 4.2: Prediction outcomes at node and substation level

Level / Model	Correct	Under	Over	MAE
Spower	79.47% (445432/560475)	0.01% (75/560475)	20.51% (114968/560475)	1.8467
Voltage	93.26% (5392523/5782301)	0.01% (419/5782301)	6.73% (389359/5782301)	0.6017
Substation	94.26% (4431815/4701914)	0.01% (339/4701914)	5.74% (269760/4701914)	0.5361

At node level, voltage accuracy reaches 93.26% and apparent-power accuracy 79.47%. Underestimation stays at 0.01% in both cases, as intended by the asymmetric loss: missing an active component would risk simplifying a substation that still participates in the transient, whereas overestimation only retains unnecessary detail. Residual errors are therefore dominated by overestimation (6.73% for voltage, 20.51% for apparent power). The lower apparent-power score is consistent with the smaller labelled generator set and with more heterogeneous machine responses. After aggregation to substations, accuracy rises to 94.26%, underestimation remains 0.01%, and MAE falls to 0.5361, so local node-level noise is attenuated when the simplification decision is taken at substation scope.

A qualitative check on a generator-disconnection scenario confirms the same pattern (Fig. 4.4). Network colouring follows the ordinal activity scale, with warmer colours marking substations retained in full node-breaker detail. Around the disturbance neighbourhood, most affected substations are recovered; the remaining mismatches are predominantly overestimations, in agreement with Table 4.2.

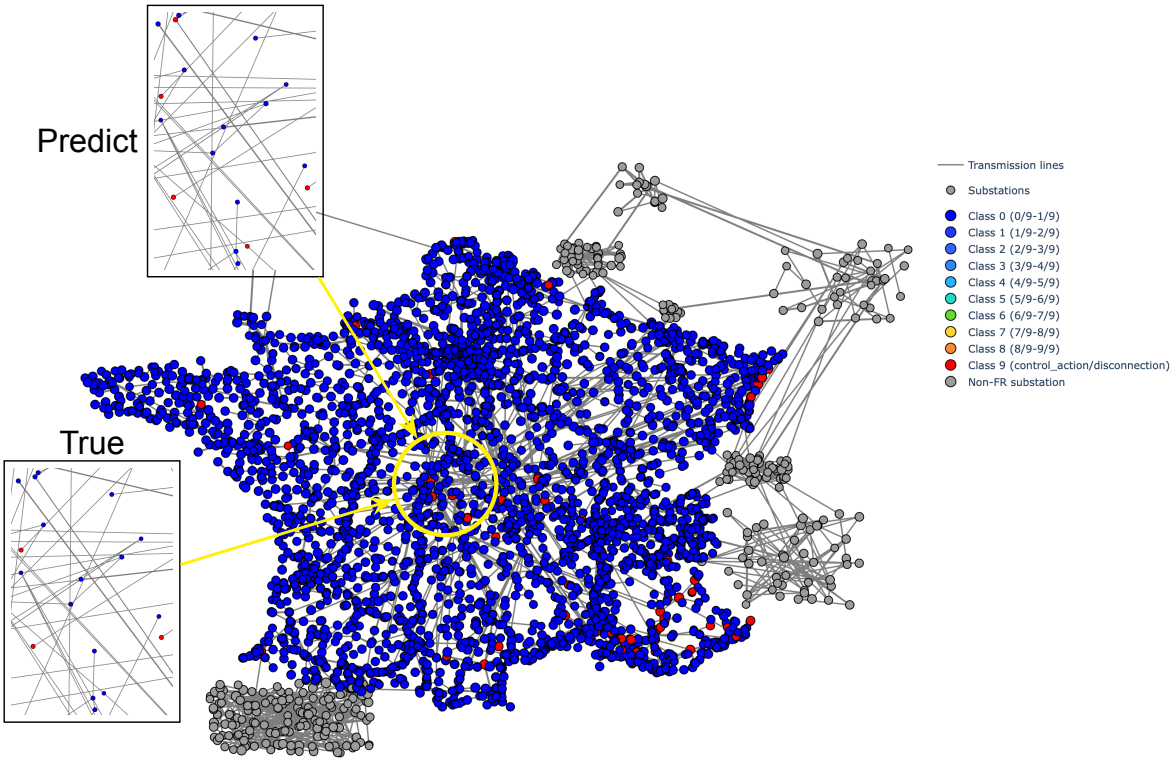


Figure 4.4: Predicted versus true substation activity classes.

Per-model diagnostics further show that incorrect predictions typically land only a few steps away from the true class on the ordinal ladder (Figs. 4.5–4.8), which keeps the MAE moderate even when exact accuracy is imperfect. Cases in which a flagged Class-9 component is predicted below the reserved label remain rare, confirming that the asymmetric objective largely prevents the most critical misses.

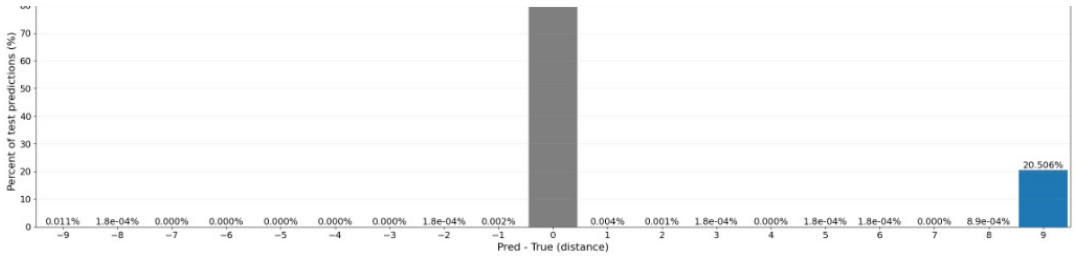


Figure 4.5: Apparent-power test distance histogram.

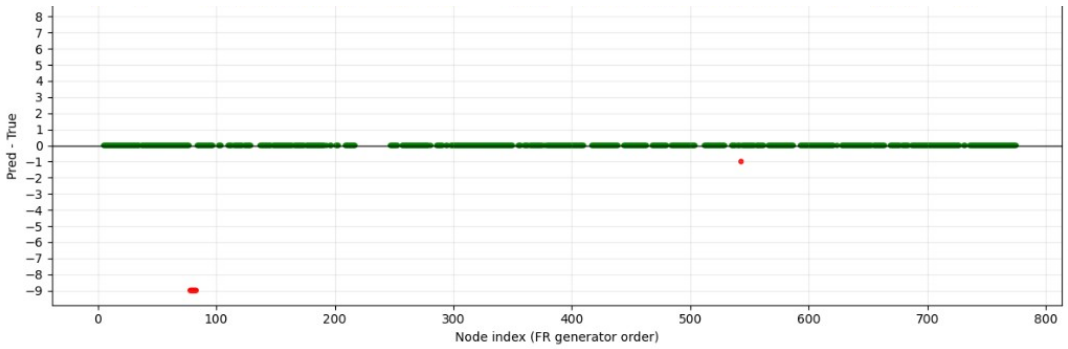


Figure 4.6: Approach 1: Apparent-power Pred–True per node example.

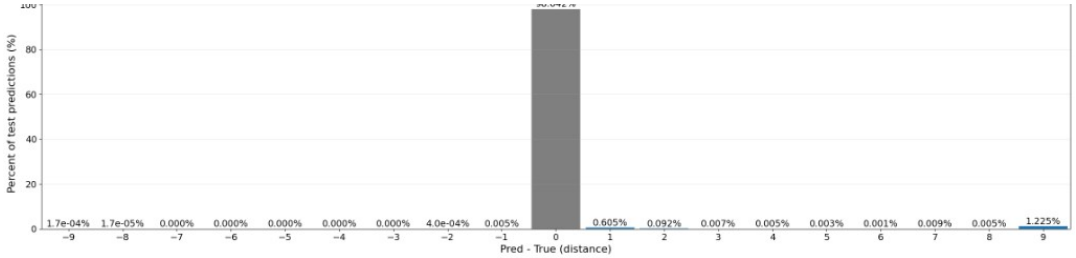


Figure 4.7: Voltage test distance histogram.

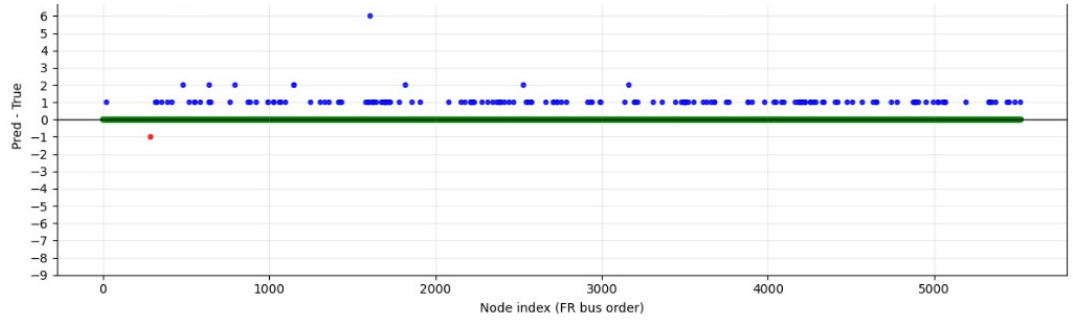


Figure 4.8: Approach 1: Voltage Pred–True per node example.

4.2.3. Simplification testing

Predicted activity classes are converted into a simplified network with the shared substation rule of Chapter 3. All experiments below use $\epsilon = 1$: only substations with aggregated predicted class $\hat{y}_{\text{sub}} < \epsilon$ (Class 0) are converted from node-breaker to bus-breaker form, while every other substation—and every action or fault location—retains full detail. This is the most conservative operating point of the framework.

A topological baseline, denoted Regional, is included for reference. It keeps the directly affected substation and all substations within a third-order graph neighbourhood in full detail and simplifies the remainder, without a learned activity map. Solving time and voltage DTW RMSE are those defined in Chapter 3 (Shared Substation-Level Simplification).

Model-size reduction

The present analysis is restricted to node-breaker to bus-breaker conversion at substation level; simplification of generator, load, and control models is left for future work. Relative to the full model, AMS retains on average 43.13% of the continuous variables, 50.61% of the discrete variables, and 87.21% of the root functions (Table 4.3). Regional reduces size further, but at a much larger trajectory error, as shown below. The comparatively modest reduction in root functions under AMS reflects their indirect dependence on network topology rather than a one-to-one link to switching devices.

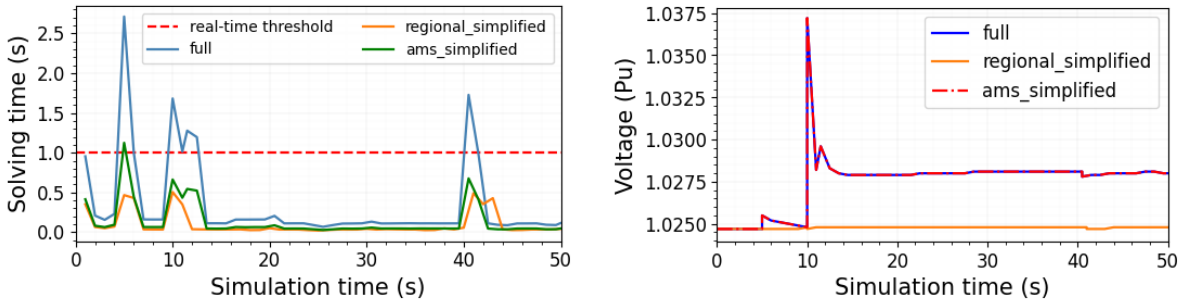
Table 4.3: System-size reduction after topology simplification.

System	System Size	System Size (%)
Full	$(319880,\ 334848,\ 276151)$	$(100.00\%,\ 100.00\%,\ 100.00\%)$
Regional	$(102328,\ 136081,\ 225216)$	$(31.99\%,\ 40.64\%,\ 81.56\%)$
AMS	$(137962,\ 169460,\ 240838)$	$(43.13\%,\ 50.61\%,\ 87.21\%)$

Real-time performance and voltage fidelity

Three representative test contingencies—a line disconnection, a busbar-section disconnection, and a generator disconnection—are used to compare the full model, Regional, and AMS with $\epsilon = 1$ (Table 4.4 and Fig. 4.9). The full node-breaker model incurs between 7 and 23 real-time violations per contingency

and cannot meet the wall-clock budget. Regional removes all violations and yields speedups of about 5.3–$5.9\times$, but DTW errors rise to the order of 10^{-2}, three orders of magnitude above the solver tolerance of 10^{-4}, so the trajectory deviation is no longer negligible. AMS occupies the intermediate trade-off: only one timing violation remains per contingency, the maximum solver step time is roughly halved relative to the full model (speedups 2.4–$2.6\times$), and DTW errors stay around 10^{-5}, well below the solver tolerance. The per-step solve-time reduction under AMS and the close alignment of the voltage response with the full-model reference are visible in Fig. 4.9.



(a) Solver time comparison.

(b) Dynamic response across model configurations.

Figure 4.9: Solver time and voltage response for Full, Regional, and AMS.

Table 4.4: Real-time performance and voltage fidelity ($\epsilon = 1$).

Scenario	Model	RT Viol.	$\|\|T_{sol}\|\|_\infty$	DTW	Speedup
Line Disc.	Full	8	2.7245	ref	ref
	Regional	0	0.5056	0.035	5.39x
	AMS	1	1.1268	1.55×10^{-5}	2.42x
Busbar Sect.	Full	7	2.7321	ref	ref
	Regional	0	0.5132	0.036	5.32x
	AMS	1	1.1036	1.53×10^{-5}	2.48x
Generator	Full	23	2.6736	ref	ref
	Regional	0	0.4547	0.056	5.88x
	AMS	1	1.0390	6.2932×10^{-5}	2.57x

With $\epsilon = 1$, the first-approach AMS therefore reduces model size and timing pressure while keeping voltage fidelity within solver tolerance. It does not yet eliminate every real-time violation under the most severe transients, but its advantage over Regional is selectivity: computational effort is concentrated on substations predicted to be dynamically active, rather than on a fixed topological neighbourhood.

4.3. Results of the Second Approach

The second approach revises both the labelling stage and the predictor. Activity classes are assigned directly on the raw KPI scale through physically aware cuts, and two task-specific pair-aware GINE models replace the GAT–CORAL formulation. Nordic 32 is used for the full prediction pipeline and for architecture comparison; the French transmission system tests industrial-scale scalability and Adaptive Model Simplification (AMS). The subsections below follow that order.

4.3.1. Physically aware cut selection and class construction

With $k = 10^{-4}$ and $\alpha = 10$, the inactive threshold is fixed at $\tau_0 = 10^{-7}$. Higher cuts are selected on the training operating points by the multi-objective procedure of Chapter 3, which balances usability of the class shares against physical consistency with the morphological severity φ.

On Nordic 32 voltage, 1891 candidate ladders are evaluated. Table 4.5 reports three of them. Candidate A collapses most of the active mass into class 4 (share 0.716) and is therefore difficult to learn. Candidate B lies on the Pareto front. Candidate C attains usability 0.999, physical score 0.970, and combined score 0.983, and is selected for labelling. Fig. 4.10 shows the full candidate cloud for both

KPIs on Nordic 32, with the Pareto front of nondominated (U, P) solutions. The same selection is applied independently to the French system (Fig. 4.11). Selected cuts for both systems and both tasks are listed in Table 4.6.

Table 4.5: Examples of Nordic 32 voltage cut candidates.

Quantity	A	B	C (selected)
τ_1	3.75×10^{-6}	2.5×10^{-6}	5×10^{-6}
τ_2	5×10^{-6}	3.75×10^{-5}	3.75×10^{-5}
τ_3	7.5×10^{-6}	3.75×10^{-4}	1.5×10^{-4}
Class-1 share	0.227	0.191	0.251
Class-2 share	0.024	0.330	0.270
Class-3 share	0.033	0.369	0.249
Class-4 share	0.716	0.109	0.229
Usability score	0.803	0.969	0.999
Physical score	0.892	0.976	0.970
Combined score	0.838	0.974	0.983

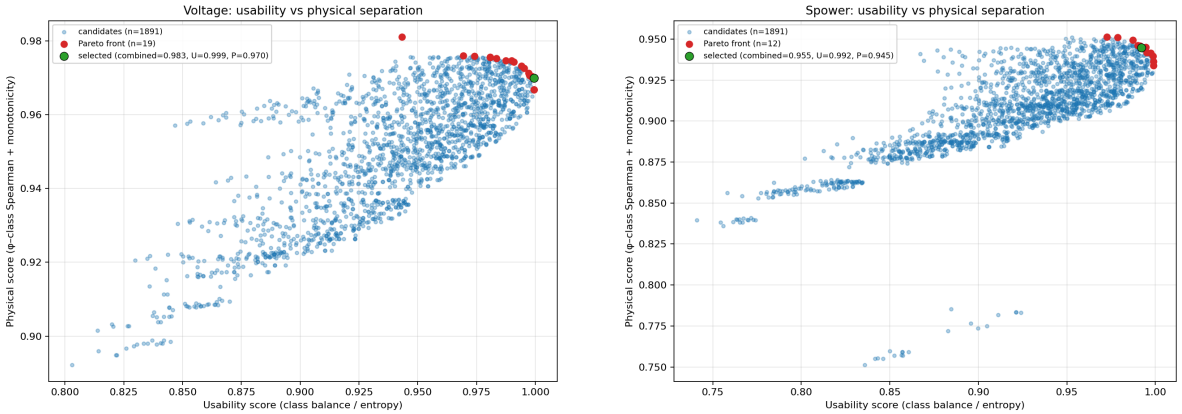


Figure 4.10: Nordic 32 cut selection: usability versus physical score for voltage (left) and apparent power (right).

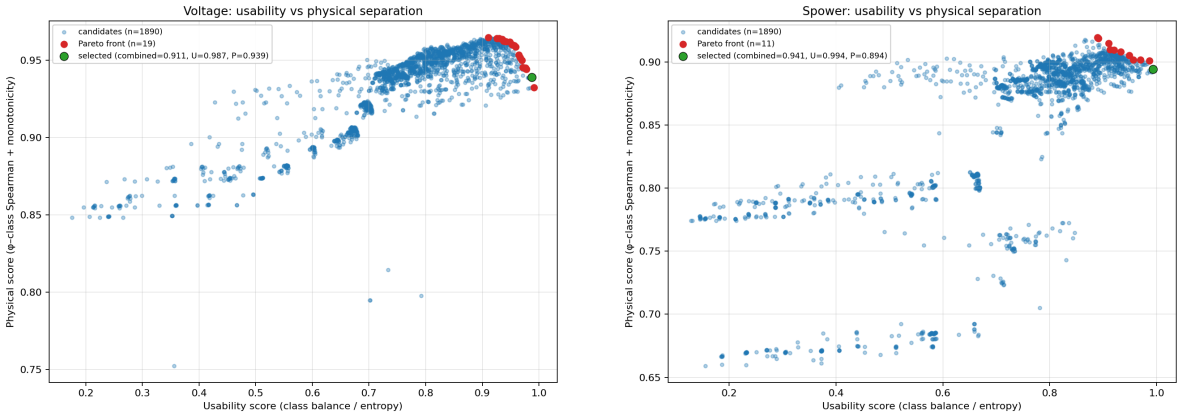


Figure 4.11: French-system cut selection: usability versus physical score for voltage (left) and apparent power (right).

Table 4.6: Selected KPI cuts for Nordic 32 and the French system.

System	Task	τ_0	τ_1	τ_2	τ_3
Nordic 32	Voltage	10^{-7}	5×10^{-6}	3.75×10^{-5}	1.5×10^{-4}
Nordic 32	S_{power}	10^{-7}	2×10^{-5}	10^{-4}	6×10^{-4}
French	Voltage	10^{-7}	5×10^{-7}	2.5×10^{-6}	10^{-5}
French	S_{power}	10^{-7}	5×10^{-7}	2.5×10^{-6}	7.5×10^{-6}

The effect of the selected Nordic cuts on the raw KPI distribution is shown in Fig. 4.12. Vertical lines mark τ_0–τ_3; mass below τ_0 forms class 0, while successive intervals define classes 1–4. Class 5 is reserved for disconnection and action flags and is assigned independently of the KPI ladder. Boxplots of the same raw KPIs stratified by the resulting class (Fig. 4.13) confirm that median severity increases monotonically along the ordinal scale. The morphological score φ follows the same ordering (Fig. 4.14): higher classes correspond to larger post-event excursion, variance, and settling residual, so the discrete labels remain physically interpretable rather than purely statistical bins.

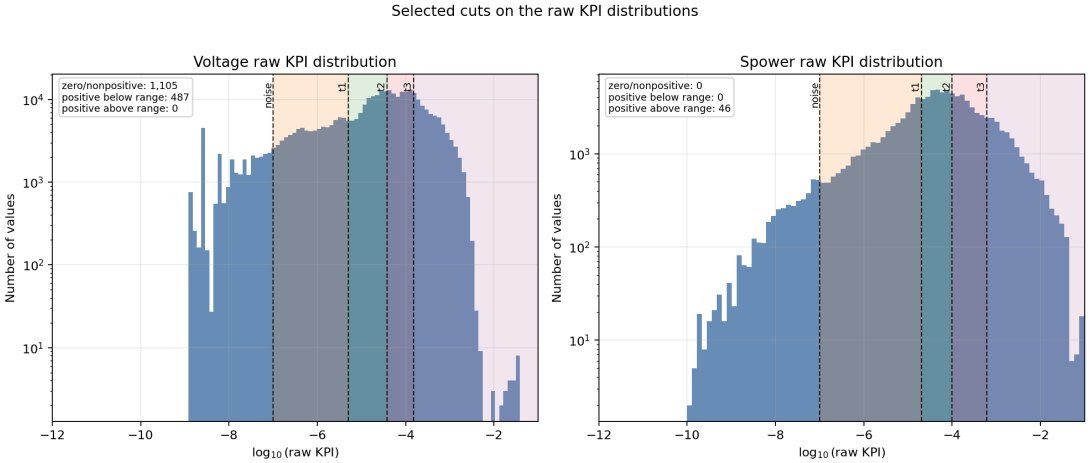


Figure 4.12: Nordic 32 raw KPI histograms with the selected class cuts.

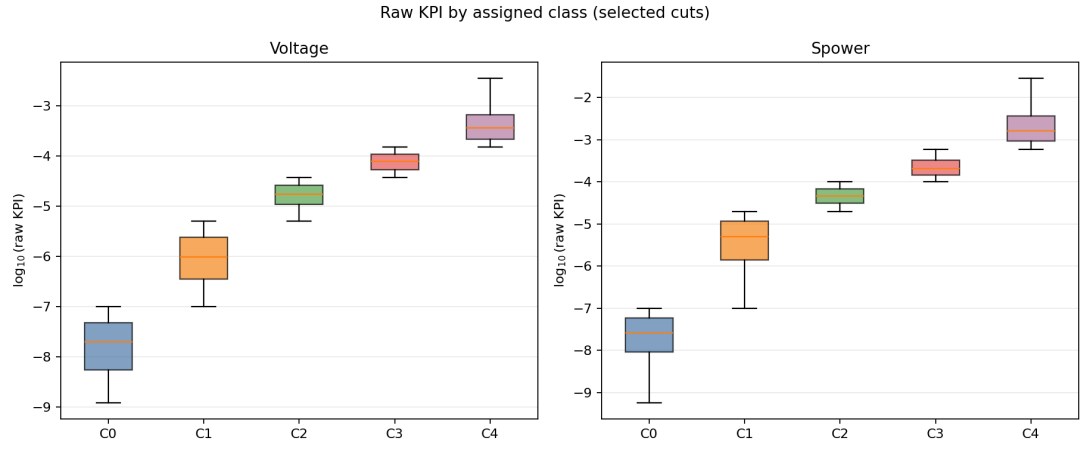


Figure 4.13: Nordic 32 raw KPI distributions by assigned class.

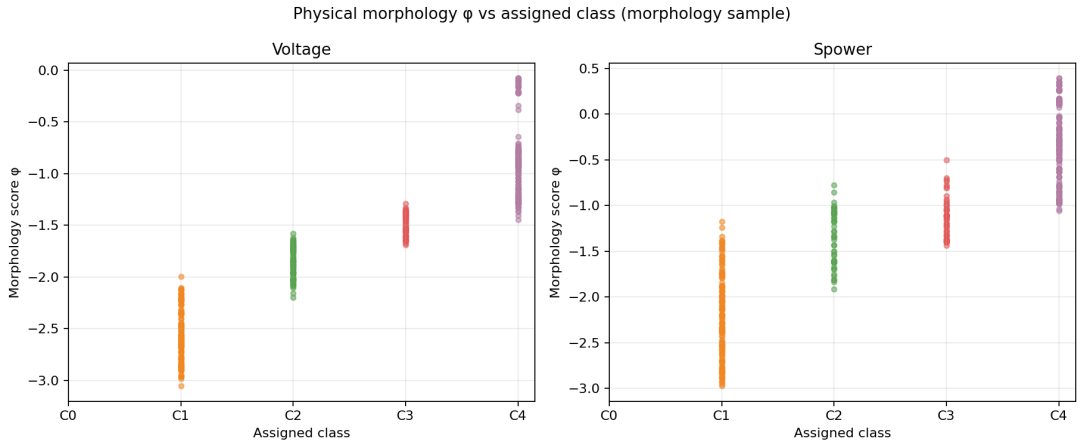


Figure 4.14: Morphological severity φ versus assigned class on Nordic 32.

Representative centred trajectories $\Delta(t) = y(t) - \bar{y}_{\mathrm{pre}}$ for classes 0–4 are shown in Fig. 4.15. Quiescent responses remain near zero; higher classes exhibit larger excursions and longer settling. The empirical class counts after cut selection appear in Fig. 4.16: Nordic 32 is comparatively balanced among the active bins, whereas the French datasets are dominated by the inactive class, consistent with localised post-event dynamics on a large network rather than with a failure of the cut procedure.

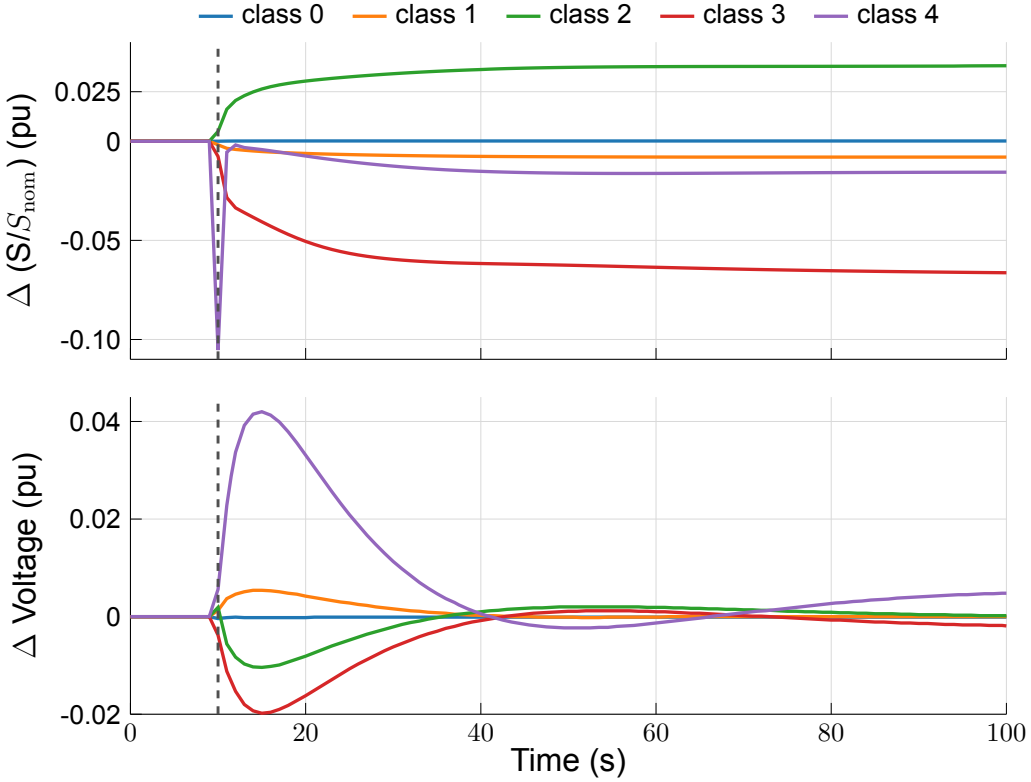


Figure 4.15: Representative Δ trajectories for classes 0–4.

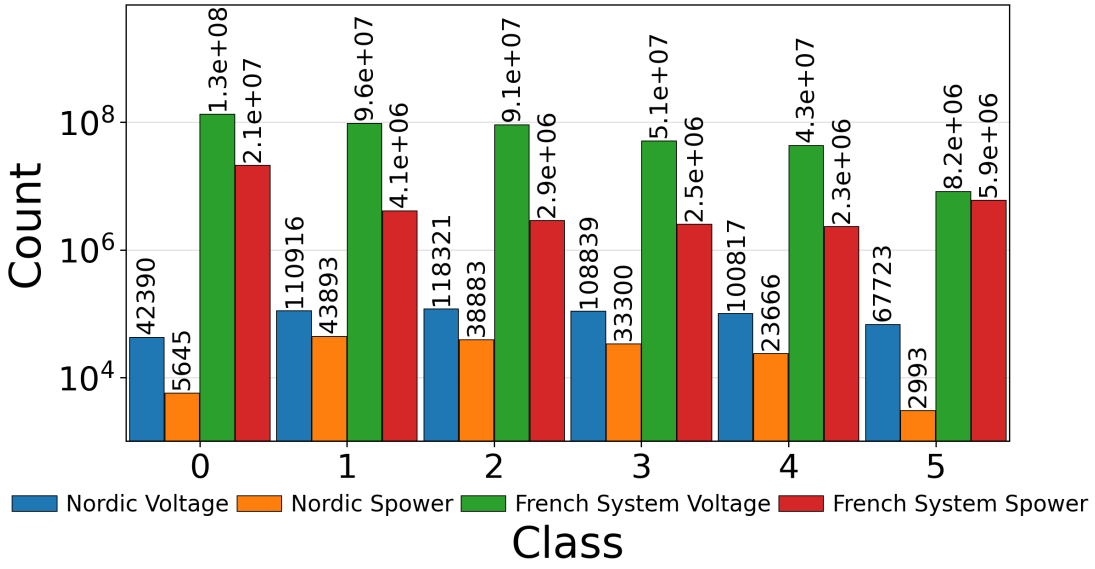


Figure 4.16: Class distributions after physically aware cut selection.

4.3.2. Prediction outcomes on Nordic 32

Voltage and apparent-power models are tuned independently on Nordic 32 under a shared Optuna protocol and then evaluated on the held-out operating points. Before presenting the figures, the two complementary diagnostics used throughout this section are defined explicitly, because they answer different questions.

Confusion matrix (row-normalised).

For each true class c, the corresponding row of the confusion matrix reports the empirical distribution of predicted classes $\hat{c}$. Because the matrix is row-normalised, each row sums to one and the diagonal entry is the recall of class c. Off-diagonal entries therefore answer: *given that the true label is c, how often is the model predicted as each alternative?* This view is essential under class imbalance: a high overall accuracy can coexist with weak recall on rare severe classes, and the row-normalised matrix exposes that behaviour directly. It does *not* by itself quantify how far an error is on the ordinal ladder—a swap between neighbouring classes and a jump from class 0 to class 5 both appear as single off-diagonal cells.

Distance histogram ($\mathrm{Pred} - \mathrm{True}$).

The signed ordinal offset $d = \hat{c} - c$ is computed for every supervised prediction and summarised as a percentage histogram over $d \in \{-5, \ldots, 5\}$. Exact matches occupy $d = 0$; positive d are over-predictions and negative d under-predictions. This diagnostic therefore answers: *when the model is wrong, by how many classes, and in which direction?* It is the natural companion to the confusion matrix for an ordinal target: adjacent-class confusion concentrates mass on $d = \pm 1$, whereas unstructured errors spread mass to large $|d|$. Scenario-level node plots of the same quantity d are used below to show where those offsets occur inside a single contingency.

Voltage model

On the Nordic voltage test set, exact agreement reaches 90.31% (Fig. 4.17). Under-prediction totals about 5.9% and is dominated by a one-class miss (4.14% at $d = -1$); over-prediction totals about 3.8%, again concentrated at $d = +1$. Offsets beyond ± 2 are rare. The row-normalised confusion matrix (Fig. 4.18) shows the same pattern class by class: recalls range from 0.82 (class 5) to 0.96 (class 1), and almost all leakage is into neighbouring bins. Class 5 (flag) remains the weakest diagonal entry, with limited leakage into classes 2–4, consistent with the fact that flagged components are labelled outside the continuous KPI pathway.

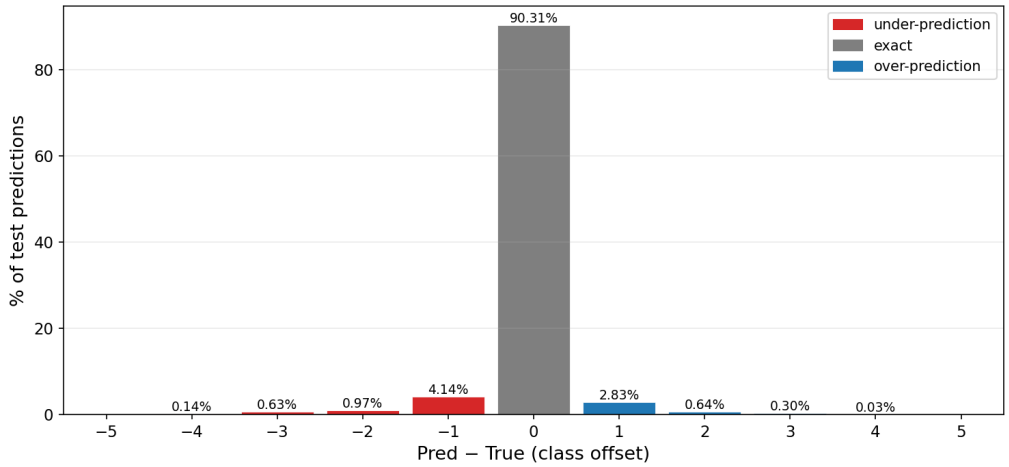


Figure 4.17: Nordic 32 voltage: test distance histogram.

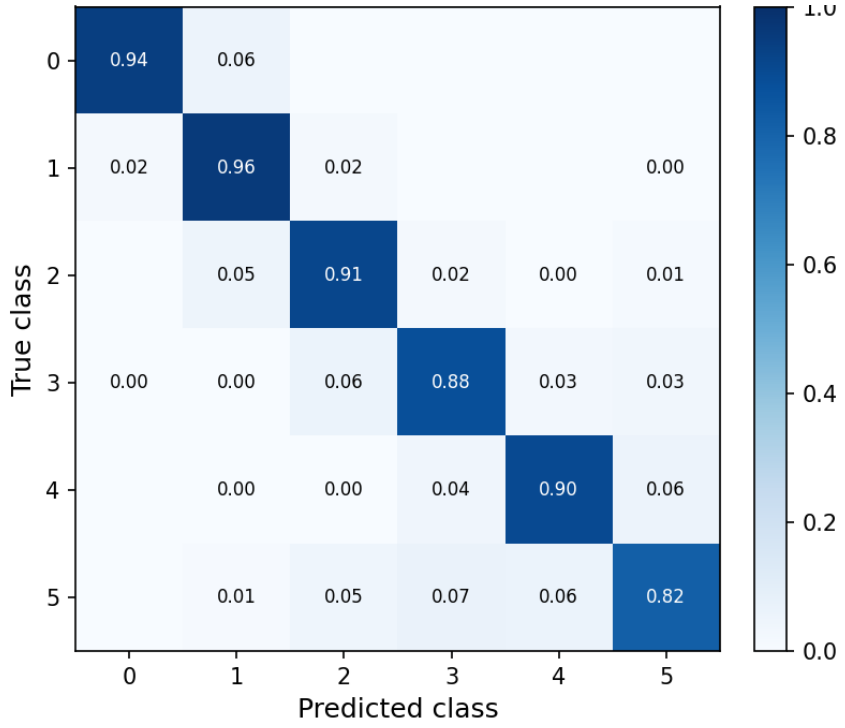


Figure 4.18: Nordic 32 voltage: row-normalised confusion matrix.

A representative over-prediction scenario (Fig. 4.19) illustrates the spatial reading of the same diagnostics. The plot shows $d = \mathrm{Pred} - \mathrm{True}$ with green markers at exact matches, blue at over-prediction, and red at under-prediction. In this contingency every visible error is confined to ± 1.

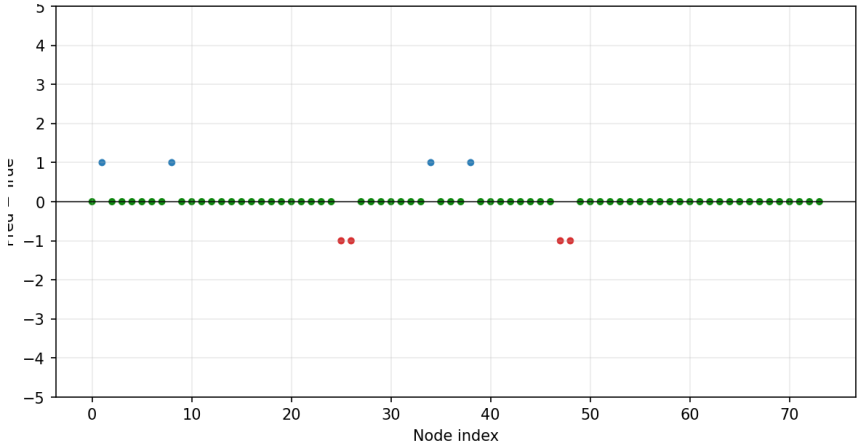


Figure 4.19: Approach 2: Nordic 32 voltage Pred–True per node example.

Apparent-power model

The apparent-power predictor is even tighter on the same Nordic split (Fig. 4.20): 92.74% exact, 4.13% at $d = -1$, 3.07% at $d = +1$, and essentially no mass beyond ± 2. The confusion matrix (Fig. 4.21) confirms high recall on every class, including a perfect 1.00 on class 5, with residual errors almost exclusively adjacent to the diagonal. The node-level OVER example in Fig. 4.22 again shows that the few mismatches stay within one ordinal step of the truth.

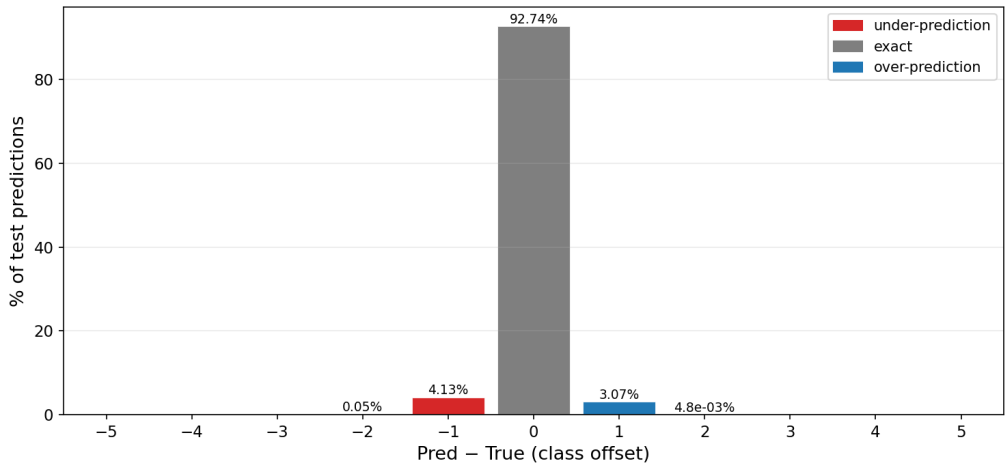


Figure 4.20: Nordic 32 apparent power: test distance histogram.

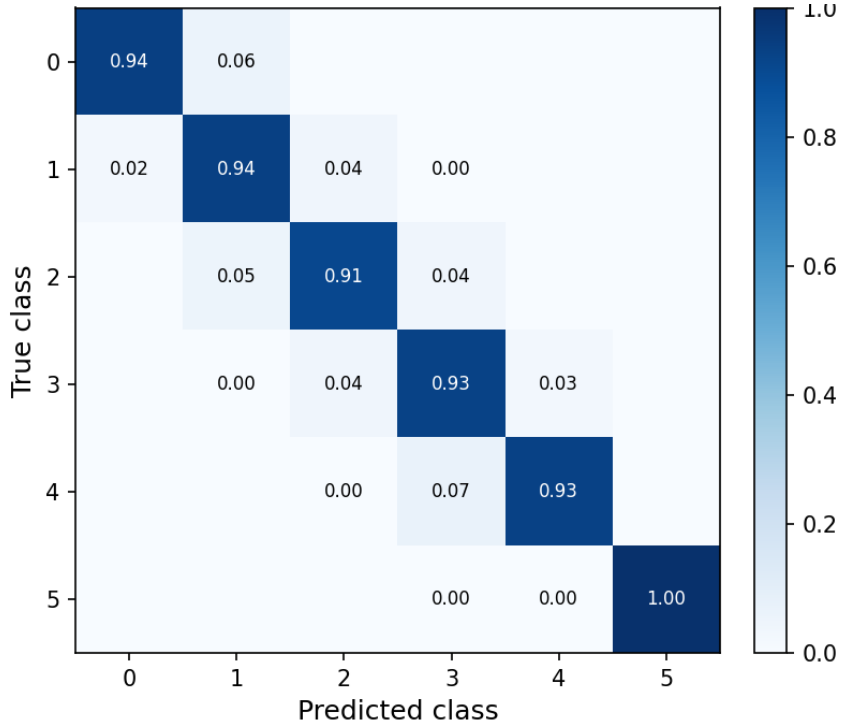


Figure 4.21: Nordic 32 apparent power: row-normalised confusion matrix.

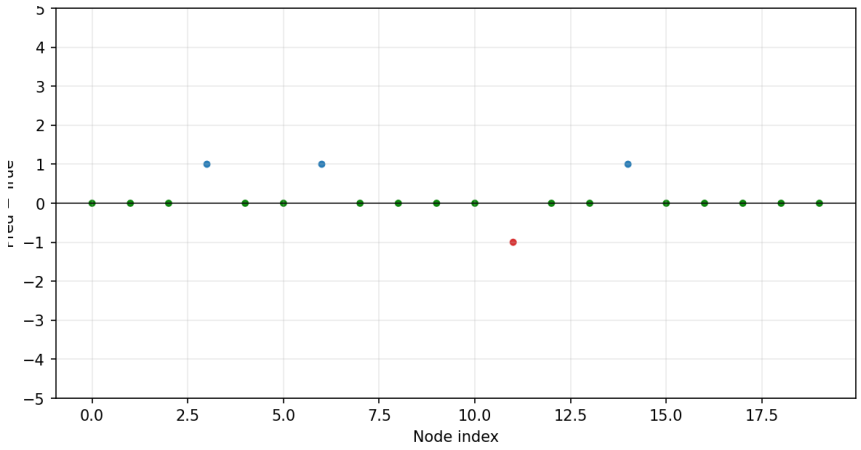


Figure 4.22: Approach 2: Nordic 32 apparent power Pred–True per node example.

Taken together, the Nordic diagnostics show that the pair-aware GINE recovers the ordinal activity map with high exact accuracy and with residual errors that remain near-misses on the severity ladder. The next subsection places that performance against simpler baselines trained on the same graphs and cuts.

4.3.3. Architecture comparison on Nordic 32

Six predictors of increasing structural content are evaluated on the Nordic 32 voltage and apparent-power tasks. XGBoost uses only local pre-event features of each supervised target. The remaining models share the pair-aware decoder. The MLP omits message passing and the graph-level context $\mathbf{g}_c$, fusing only the encoder state $\mathbf{h}_i^{(0)}$ with the contingency representation. GATv2, NNConv, TransformerConv, and GINE restore the full pair-aware fusion with edge attributes in the convolution and $\mathbf{h}_i$ in place of $\mathbf{h}_i^{(0)}$, so only the encoder differs.

Hyperparameters are selected by 5-fold cross-validation on the train and validation operating points: each Optuna trial is scored by the mean fold validation selection score, and the best trial is then retrained once. Fig. 4.23 reports those validation scores (5-fold mean±std). Table 4.7 reports held-out test metrics together with parameter count, training wall time, and inference as the held-out median (mean±std). Pair-aware fusion already improves on XGBoost; graph embeddings improve further; the four edge-aware encoders then stay close, while GINE has the lowest inference cost among them.

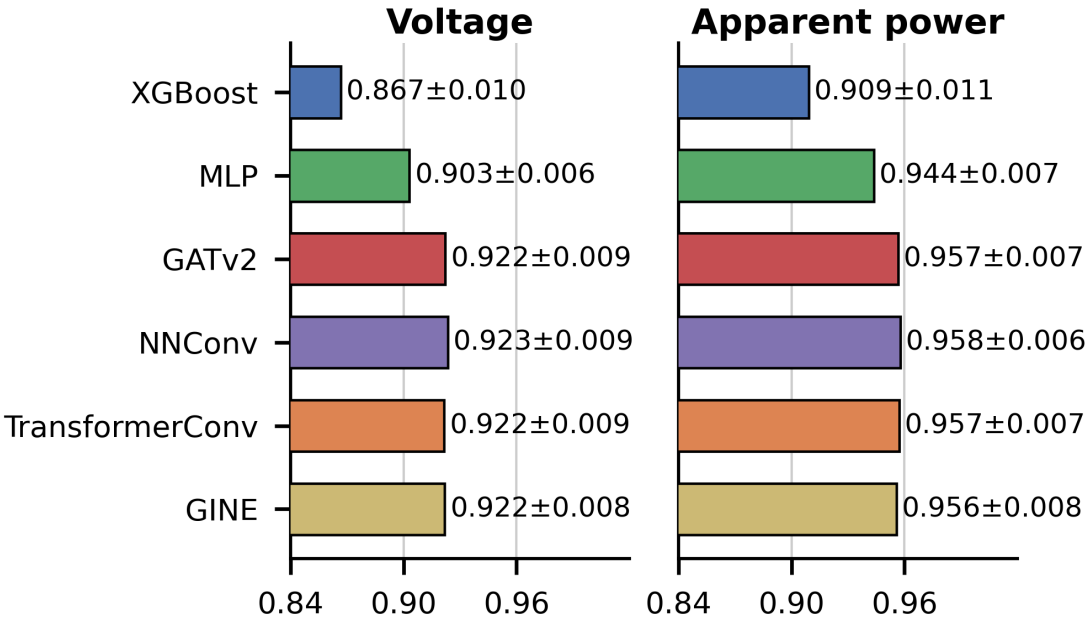


Figure 4.23: Validation selection score on Nordic 32 (5-fold mean±std).

Table 4.7: Nordic 32 architecture comparison on the held-out test set. Inference is the median (mean±std), in ms per scenario. Train wall is Optuna plus the final retrain.

Task	Model	BA	Acc.	Acc_1	Macro-F_1	Scen. $\overline{F_1}$	P10 F_1	Cl. 5 F_1	#Params	Train wall	Infer.(ms/scen.)
Voltage	XGBoost	0.859	0.852	0.975	0.858	0.803	0.580	0.859	—	15m 19s	$0.552\,(0.552{\pm}0.004)$
	MLP	0.893	0.888	0.977	0.892	0.855	0.678	0.871	6.48×10^5	13h 39m	$1.286\,(1.482{\pm}0.273)$
	GATv2	0.919	0.917	0.980	0.917	0.887	0.728	0.876	1.99×10^6	17h 15m	$1.457\,(1.642{\pm}0.285)$
	NNConv	0.918	0.916	0.979	0.917	0.884	0.721	0.870	5.15×10^7	17h 02m	$2.058\,(2.248{\pm}0.294)$
	TransformerConv	0.918	0.916	0.978	0.917	0.885	0.721	0.873	2.58×10^6	13h 40m	$1.424\,(1.617{\pm}0.300)$
	GINE	0.914	0.913	0.979	0.914	0.882	0.728	0.876	1.60×10^6	16h 38m	$1.402\,(1.597{\pm}0.303)$
S_{power}	XGBoost	0.883	0.869	0.989	0.888	0.830	0.600	0.997	—	6m 39s	$0.161\,(0.162{\pm}0.005)$
	MLP	0.921	0.908	0.997	0.920	0.886	0.685	0.990	4.79×10^5	5h 02m	$1.344\,(1.554{\pm}0.291)$
	GATv2	0.937	0.925	0.999	0.937	0.904	0.729	0.992	3.74×10^6	11h 26m	$1.497\,(1.703{\pm}0.290)$
	NNConv	0.940	0.932	1.000	0.942	0.913	0.740	0.991	5.15×10^7	22h 59m	$2.043\,(2.236{\pm}0.299)$
	TransformerConv	0.937	0.929	0.999	0.937	0.910	0.733	0.993	4.12×10^6	12h 36m	$1.457\,(1.684{\pm}0.314)$
	GINE	0.934	0.922	0.998	0.934	0.898	0.700	0.984	2.26×10^6	9h 58m	$1.399\,(1.595{\pm}0.306)$

4.3.4. Scalability on the French transmission system

The same pipeline is transferred to the French transmission model without architectural redesign. Relative to the Nordic protocol, only the Optuna budget, a subset of the search ranges, and the loss-weight mix are adjusted (Chapter 3). Held-out metrics are reported in Table 4.8. Within-one accuracy remains high ($\mathrm{Acc}_1 = 0.930$ for voltage and 0.917 for apparent power), while exact pooled accuracy is lower than on Nordic 32, as expected under stronger inactive dominance and a much larger graph. Scenario-level F_1 is more variable, and class 5 F_1 is the weakest entry on both tasks.

Table 4.8: Held-out test metrics of the pair-aware GINE on the French system.

Task	BA	Acc.	Acc_1	Macro-F_1	Scen. $\overline{F_1}$	P10 F_1	Cl. 5 F_1
Voltage	0.700	0.679	0.930	0.675	0.608	0.398	0.715
S_{power}	0.717	0.792	0.917	0.714	0.593	0.451	0.704

Voltage model

The French voltage distance histogram (Fig. 4.24) places 67.95% of predictions at exact match. Over-prediction ($\approx 21.7\%$) exceeds under-prediction ($\approx 10.4\%$), with most of the mass still at $d = \pm 1$. The row-normalised confusion matrix (Fig. 4.25) shows recalls between 0.63 and 0.81, with the strongest diagonals on classes 4 and 5 and with leakage predominantly into neighbouring classes—again the ordinal near-miss pattern, now on a $\sim$ 6000-bus network. Fig. 4.26 shows the signed offset d for a representative UNDER contingency: the dense green band at zero confirms that the bulk of the network is recovered exactly, while scattered blue and red markers mark the residual over- and under-predictions.

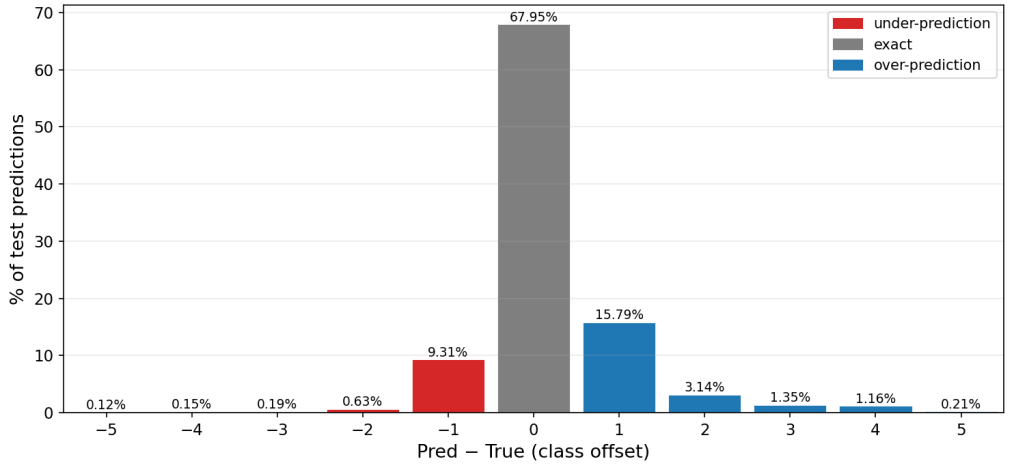


Figure 4.24: French voltage: test distance histogram.

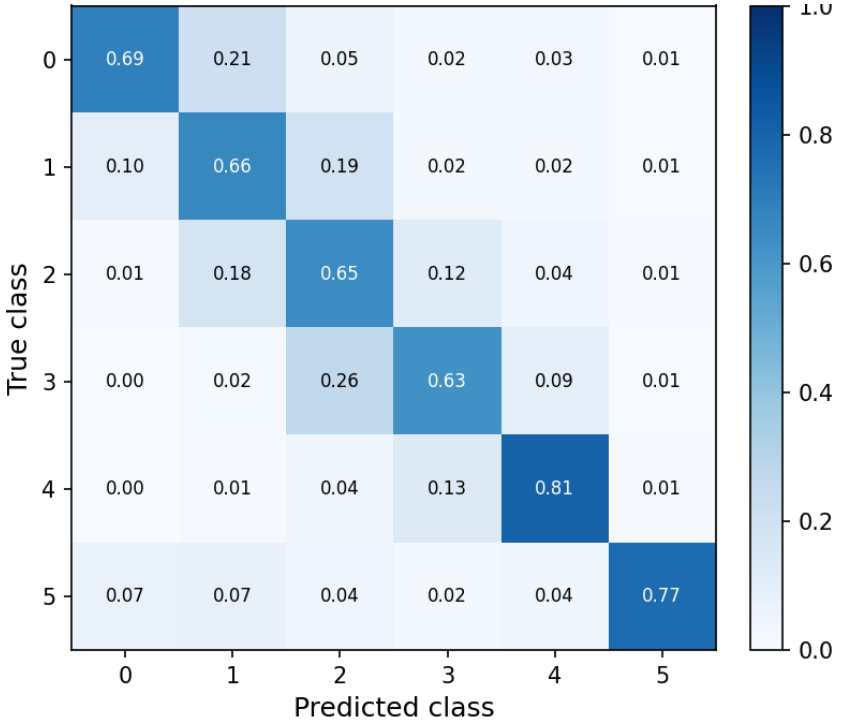


Figure 4.25: French voltage: row-normalised confusion matrix.

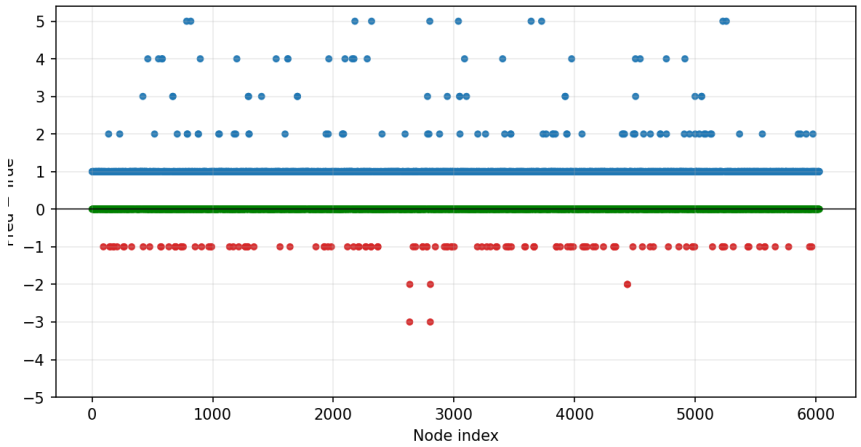


Figure 4.26: Approach 2: French voltage Pred–True per node example.

Apparent-power model

On French apparent power, exact agreement rises to 79.24% (Fig. 4.27), with a nearly symmetric ± 1 tail and only about 1% of predictions in each larger-offset bin. The confusion matrix (Fig. 4.28) is strongest on classes 0 and 4 (recalls 0.94 and 0.86) and weaker on the intermediate bins, where adjacent-class swaps dominate. The UNDER node-level offset plot (Fig. 4.29) again shows a dominant correct band with sparse under- and over-predictions across the generator set.

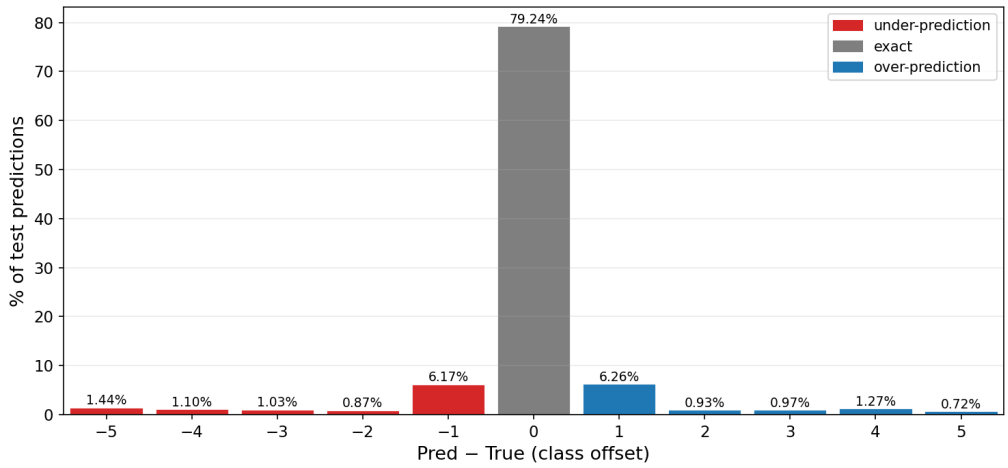


Figure 4.27: French apparent power: test distance histogram.

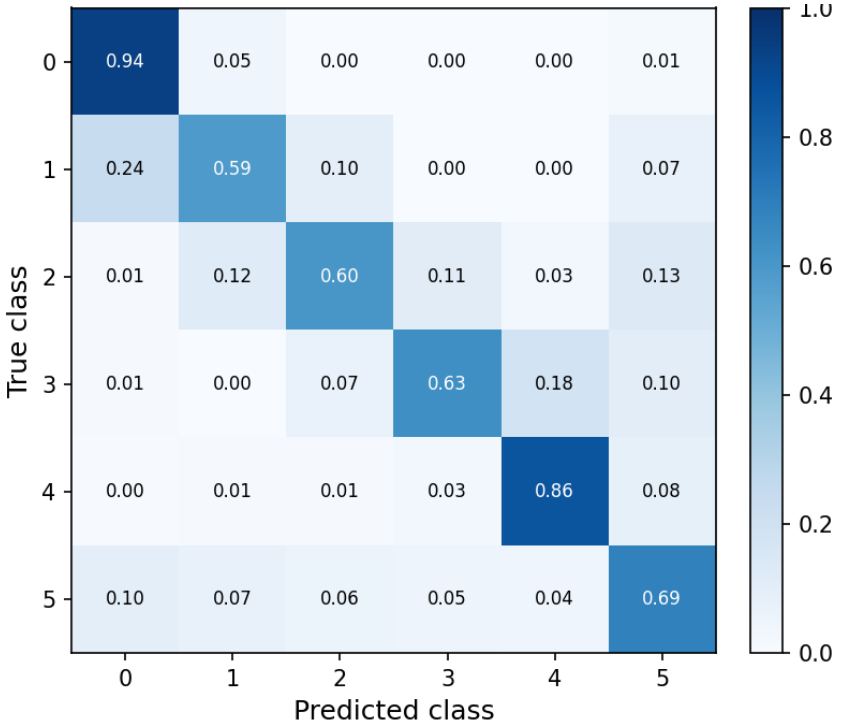


Figure 4.28: French apparent power: row-normalised confusion matrix.

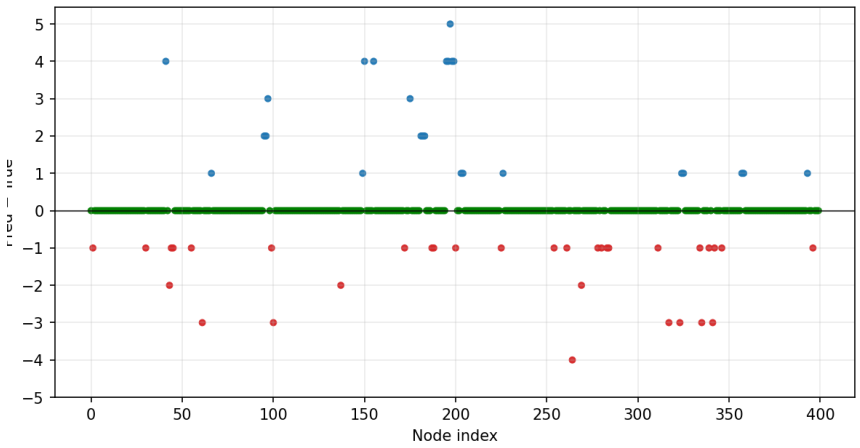


Figure 4.29: Approach 2: French apparent power Pred–True per node example.

Overall, the French results demonstrate that the second-approach pipeline scales without redesign: exact scores drop relative to Nordic 32, but within-one accuracy remains above 0.91 and the residual errors stay predominantly neighbouring on the ordinal ladder—acceptable for screening and for retain/collapse thresholds in AMS.

4.3.5. Adaptive simplification and real-time performance

Predicted activity classes are converted into a simplified French network with the shared substation rule of Chapter 3. On the French test set (6768 scenarios), four system models are compared:

- **Full:** fully detailed node-breaker model.
- **Regional:** the French TSO heuristic that retains only substations within three hops of the event.
- **AMS oracle:** simplification driven by the true activity classes.

- **AMS Prediction:** simplification driven by the predicted activity classes.

Raising the retention threshold ϵ shrinks the retained model and improves timing, but also increases the risk—for AMS Prediction—of simplifying substations that still matter. AMS oracle is included as a reference for the intrinsic limitation of substation-level simplification when the keep-set is perfect.

Table 4.9 reports average system sizes for $\epsilon \in \{1, \ldots, 5\}$. At $\epsilon = 5$, AMS Prediction retains on average 27.74% of the continuous variables, 34.23% of the discrete variables, and 64.36% of the root functions, close to the oracle sizes and comparable to Regional in footprint while remaining contingency-specific.

Table 4.9: Average system size after substation-level simplification on the French test set.

System	System size	System size (%)
Full	$(374\,989,\ 397\,301,\ 353\,160)$	$(100.00\%,\ 100.00\%,\ 100.00\%)$
Regional	$(97\,760,\ 130\,232,\ 224\,645)$	$(26.07\%,\ 32.78\%,\ 63.61\%)$
AMS oracle (ϵ=1)	$(249\,963,\ 277\,215,\ 295\,643)$	$(66.66\%,\ 69.77\%,\ 83.71\%)$
AMS oracle (ϵ=2)	$(195\,427,\ 224\,557,\ 270\,209)$	$(52.12\%,\ 56.52\%,\ 76.51\%)$
AMS oracle (ϵ=3)	$(147\,435,\ 178\,201,\ 247\,819)$	$(39.32\%,\ 44.85\%,\ 70.17\%)$
AMS oracle (ϵ=4)	$(121\,578,\ 153\,212,\ 235\,745)$	$(32.42\%,\ 38.56\%,\ 66.75\%)$
AMS oracle (ϵ=5)	$(101\,440,\ 133\,574,\ 226\,164)$	$(27.05\%,\ 33.62\%,\ 64.04\%)$
AMS Prediction (ϵ=1)	$(273\,296,\ 299\,774,\ 306\,547)$	$(72.88\%,\ 75.45\%,\ 86.80\%)$
AMS Prediction (ϵ=2)	$(208\,895,\ 237\,556,\ 276\,480)$	$(55.71\%,\ 59.79\%,\ 78.29\%)$
AMS Prediction (ϵ=3)	$(155\,127,\ 185\,647,\ 251\,422)$	$(41.37\%,\ 46.73\%,\ 71.19\%)$
AMS Prediction (ϵ=4)	$(128\,190,\ 159\,546,\ 238\,779)$	$(34.19\%,\ 40.16\%,\ 67.61\%)$
AMS Prediction (ϵ=5)	$(104\,033,\ 135\,996,\ 227\,294)$	$(27.74\%,\ 34.23\%,\ 64.36\%)$

Table 4.10 and Fig. 4.30 illustrate the timing–fidelity trade-off on four representative contingencies: Well (line), Over (bus), Under (generator), and Under (w/o) (line). Solving time and voltage error follow Chapter 3. Well matches the oracle retain set; Over retains extra substations; Under removes a topology-dependent control action and incurs a non-negligible voltage error; Under (w/o) is also under-predicted but without that action, so the voltage error stays within tolerance.

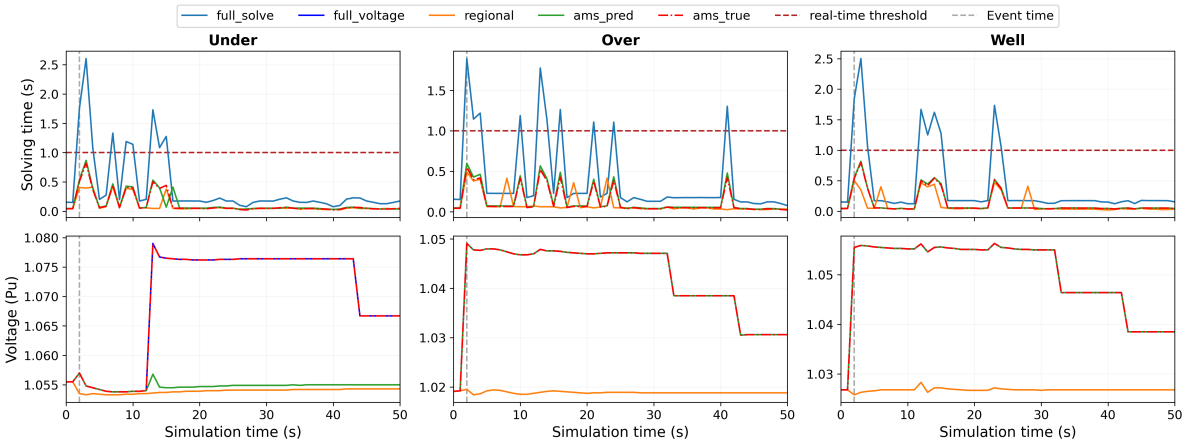


Figure 4.30: Solving time and voltage for Under, Over, and Well scenarios ($\epsilon = 5$).

Table 4.10: Real-time performance and voltage fidelity across scenarios and systems.

Scenario	System	RT Viol.	$\|T_{\mathrm{sol}}\|_\infty$	DTW	Speedup
Well (Line)	Full	9	2.4999	ref	ref
	Regional	0	0.5032	0.018	4.97x
	AMS oracle (ϵ=1)	5	1.7470	2.89×10^{-5}	1.43x
	AMS oracle (ϵ=2)	1	1.1905	2.89×10^{-5}	2.10x
	AMS oracle (ϵ=3)	0	0.9867	2.89×10^{-5}	2.53x
	AMS oracle (ϵ=4)	0	0.8871	2.89×10^{-5}	2.82x
	AMS oracle (ϵ=5)	0	0.7983	2.89×10^{-5}	3.13x
	AMS Prediction (ϵ=1)	5	1.7215	2.89×10^{-5}	1.45x
	AMS Prediction (ϵ=2)	1	1.1490	2.89×10^{-5}	2.18x
	AMS Prediction (ϵ=3)	1	1.0074	2.89×10^{-5}	2.48x
	AMS Prediction (ϵ=4)	0	0.9207	2.89×10^{-5}	2.72x
	AMS Prediction (ϵ=5)	0	0.8179	2.89×10^{-5}	3.06x
Over (Bus)	Full	10	1.8891	ref	ref
	Regional	0	0.4907	0.018	3.85x
	AMS oracle (ϵ=1)	2	1.3188	1.07×10^{-5}	1.43x
	AMS oracle (ϵ=2)	0	0.9473	1.07×10^{-5}	1.99x
	AMS oracle (ϵ=3)	0	0.6961	1.07×10^{-5}	2.71x
	AMS oracle (ϵ=4)	0	0.5851	1.07×10^{-5}	3.23x
	AMS oracle (ϵ=5)	0	0.5473	1.07×10^{-5}	3.45x
	AMS Prediction (ϵ=1)	6	1.5622	1.07×10^{-5}	1.21x
	AMS Prediction (ϵ=2)	2	1.2446	1.07×10^{-5}	1.52x
	AMS Prediction (ϵ=3)	1	1.0397	1.07×10^{-5}	1.82x
	AMS Prediction (ϵ=4)	0	0.9051	1.07×10^{-5}	2.09x
	AMS Prediction (ϵ=5)	0	0.5978	1.07×10^{-5}	3.16x
Under (Generator)	Full	9	2.6017	ref	ref
	Regional	0	0.4257	0.019	6.11x
	AMS oracle (ϵ=1)	7	2.3077	1.45×10^{-5}	1.13x
	AMS oracle (ϵ=2)	3	1.7373	1.45×10^{-5}	1.50x
	AMS oracle (ϵ=3)	1	1.0800	1.45×10^{-5}	2.41x
	AMS oracle (ϵ=4)	0	0.8831	1.45×10^{-5}	2.95x
	AMS oracle (ϵ=5)	0	0.8199	1.45×10^{-5}	3.17x
	AMS Prediction (ϵ=1)	8	2.3235	1.45×10^{-5}	1.12x
	AMS Prediction (ϵ=2)	2	1.5278	1.45×10^{-5}	1.70x
	AMS Prediction (ϵ=3)	1	1.1806	0.014	2.20x
	AMS Prediction (ϵ=4)	0	0.9478	0.014	2.75x
	AMS Prediction (ϵ=5)	0	0.8672	0.014	3.00x
Under (w/o) (Line)	Full	9	2.4992	ref	ref
	Regional	0	0.5026	0.018	4.97x
	AMS oracle (ϵ=1)	5	1.7376	2.89×10^{-5}	1.44x
	AMS oracle (ϵ=2)	1	1.1889	2.89×10^{-5}	2.10x
	AMS oracle (ϵ=3)	0	0.9947	2.89×10^{-5}	2.51x
	AMS oracle (ϵ=4)	0	0.8936	2.89×10^{-5}	2.80x
	AMS oracle (ϵ=5)	0	0.7988	2.89×10^{-5}	3.13x
	AMS Prediction (ϵ=1)	5	1.7051	2.89×10^{-5}	1.47x
	AMS Prediction (ϵ=2)	1	1.1430	2.89×10^{-5}	2.19x
	AMS Prediction (ϵ=3)	0	0.9981	2.89×10^{-5}	2.50x
	AMS Prediction (ϵ=4)	0	0.9216	2.89×10^{-5}	2.71x
	AMS Prediction (ϵ=5)	0	0.8212	2.89×10^{-5}	3.04x

Regional is faster, yet that speed comes with an accuracy penalty (voltage DTW of order 10^{-2}) and with a static retain rule that does not adapt to which components will move. AMS pursues the opposite trade-off. AMS oracle shows that contingency-specific retain sets can meet—and beat—the real-time budget while keeping voltage DTW at $\sim 10^{-5}$, within solver noise of the Full trajectories. AMS Prediction approaches that oracle behaviour whenever the critical keep-set is recovered, as in the Well, Over, and Under (w/o) cases, with speedups up to $3.16\times$ at $\epsilon = 5$. The residual risk is underprediction of substations that host topology-dependent control actions: simplifying such a substation changes the simulated control response and produces a visible voltage error, as in the Under generator example. Overprediction has the opposite effect—extra substations are retained, fidelity is preserved, and only speedup is reduced. The design aim is deliberately conservative: a modest excess of overprediction is acceptable if it suppresses underprediction of control-critical substations, because the cost of overprediction is computational, whereas the cost of underprediction is trajectory error.

5

Conclusion

This report set out an Adaptive Model Selection framework for real-time operator training. The motivating constraint is practical: a fixed full-detail representation of a large transmission system cannot reliably meet a SCADA-compatible refresh, while a fixed coarse reduction restores timing only by discarding dynamics that trainees are meant to see. The proposed answer is to predict component-level dynamic activity from the operating point and contingency description, then retain node-breaker detail only where the predicted severity crosses a retention threshold ϵ.

Two formulations were developed along that line. The first approach normalised KPIs to the unit interval, assigned nine ordinal bins plus a flag class, and trained GAT–CORAL predictors on the French transmission system. Exact agreement on the test set was high for voltage and still useful for apparent power, with underestimation suppressed by design. Aggregating predictions to substations further improved the decision that AMS actually uses. In the simplification tests at $\epsilon = 1$, AMS occupied the intended intermediate trade-off against Full and Regional models: fewer timing violations than the full model, and voltage DTW errors within solver tolerance, whereas Regional removed timing pressure at the cost of DTW errors of order 10^{-2}.

The second approach revised both the labels and the decoder. Physically aware cuts on the raw KPI scale, anchored at $\tau_0 = 10^{-7}$ to solver precision, produced ordinal classes that remain readable against morphological severity. Pair-aware models then conditioned each target on its contingency pair. On Nordic 32, pair-aware fusion improves on a local XGBoost baseline, graph embeddings improve further, and the edge-aware encoders stay close, with GINE the cheapest among them (held-out BA 0.914 / 0.934 at about 1.4 ms per scenario). On the French system, exact accuracy falls under inactive dominance, yet within-one accuracy stays at 0.930 and 0.917, which is the relevant regime for substation retention. Driven by those maps, AMS Prediction at $\epsilon = 5$ cuts continuous and discrete model size by about 72% and 66% across 6768 scenarios, with speedups above $3\times$ whenever the critical keep-set is recovered.

The residual limitation is clear from both approaches. Underprediction of a substation that hosts a topology-dependent control action can change the simulated control response and produce a visible voltage error; overprediction only keeps extra detail and reduces speedup. The design therefore accepts a modest excess of overprediction. A further intrinsic bound is the present simplification scope itself: even a perfect keep-set only switches substations between node-breaker and bus-breaker topology.

Overall, the results confirm that graph-based activity prediction can support hybrid-detail training simulations that are both interactive and faithful to the full reference when the keep-set is adequate. Future work includes alternative electrical-distance definitions, additional targets such as system- or component-level frequency activity for EMT studies, and illustrating the methodology on other applications not evaluated here, in particular online Dynamic Security Assessment (DSA).

References

[1] Jun An et al. “Power system transient stability margin assessment using steady-state information”. In: *2019 IEEE Innovative Smart Grid Technologies - Asia (ISGT Asia)* (2019). DOI: 10.1109/isgt-asia.2019.8881332.

[2] Petros Aristidou and Thierry Van Cutsem. “Parallel Computing and Localization Techniques for Faster Power System Dynamic Simulations”. In: (2014).

[3] R. El-attar and M. Vidyasagar. “Order reduction by ℓ_1- and ℓ_∞-norm minimization”. In: (1978). DOI: 10.1109/tac.1978.1101830.

[4] Syafiq Kamarul Azman et al. “A Unified Online Deep Learning Prediction Model for Small Signal and Transient Stability”. In: *IEEE Transactions on Power Systems* (2020). DOI: 10.1109/tpwrs.2020.2999102.

[5] Liu Baozhu and Liu Bolong. “A novel static voltage stability index based on equilibrium solution region of branch power flow”. In: *2008 Third International Conference on Electric Utility Deregulation and Restructuring and Power Technologies* (2008). DOI: 10.1109/drpt.2008.4523517.

[6] Wenzhi Cao, Vahid Mirjalili, and Sebastian Raschka. “Rank consistent ordinal regression for neural networks with application to age estimation”. In: *Pattern Recognition Letters* (2020). DOI: 10.1016/j.patrec.2020.11.008.

[7] Jidong Chai et al. “Measurement-based system reduction using autoregressive model”. In: *2016 IEEE/PES Transmission and Distribution Conference and Exposition (T&D)* (2016). DOI: 10.1109/tdc.2016.7519981.

[8] K. Chakraborty, Gagari Deb, and S. Deb. “Voltage stability assessment in radial distribution system by line stability indicator (LSI) and its improvement using SVC”. In: *2016 IEEE 1st International Conference on Power Electronics, Intelligent Control and Energy Systems (ICPEICES)* (2016). DOI: 10.1109/icpeices.2016.7853132.

[9] S. Chanda and B. Das. “Identification of weak buses in a power network using novel voltage stability indicator in radial distribution system”. In: *India International Conference on Power Electronics 2010 (IICPE2010)* (2011). DOI: 10.1109/iicpe.2011.5728121.

[10] D. Chaniotis and M.A. Pai. “Model reduction in power systems using Krylov subspace methods”. In: *IEEE Power Engineering Society General Meeting, 2005* (2005). DOI: 10.1109/pes.2005.1489192.

[11] M. Chidambara. “Two simple techniques for the simplification of large dynamic systems”. In: *IEEE Transactions on Automatic Control* (1969). DOI: 10.1109/jacc.1969.4169310.

[12] Kalana Dulanjith Dharmapala et al. “Machine Learning Based Real-Time Monitoring of Long-Term Voltage Stability Using Voltage Stability Indices”. In: *IEEE Access* (2020). DOI: 10.1109/access.2020.3043935.

[13] *Dynawo: A Comprehensive Power System Dynamic Simulator*. Supported by the Linux Foundation Energy initiative. 2025. URL: https://lfenergy.org/projects/dyna%CF%89o.

[14] Eduard Eitelberg. “Model reduction by minimizing the weighted equation error”. In: *International Journal of Control* (1981). DOI: 10.1080/00207178108922585.

[15] D. Fabozzi and T. Cutsem. “Assessing the Proximity of Time Evolutions through Dynamic Time Warping”. In: (2011). DOI: 10.1049/iet-gtd.2011.0415.

[16] F. Gomez et al. “Support Vector Machine-Based Algorithm for Post-Fault Transient Stability Status Prediction Using Synchronized Measurements”. In: *IEEE Transactions on Power Systems* (2011). DOI: 10.1109/tpwrs.2010.2082575.

[17] Nasiri Soloklo Hasan and Maghfoori Farsangi Malihe. “ORDER REDUCTION BY MINIMIZING INTEGRAL SQUARE ERROR AND H_∞ NORM OF ERROR”. In: *Journal of Advances in Computer Research* (2014).

[18] Jiyu Huang et al. “Recurrent Graph Convolutional Network-Based Multi-Task Transient Stability Assessment Framework in Power System”. In: *IEEE Access* (2020). DOI: 10.1109/access.2020.2991263.

[19] Rakibul Islam et al. “Machine Learning for Power System Stability and Control”. In: *Results in Engineering* (2025). DOI: 10.1016/j.rineng.2025.105355.

[20] Bazilah Ismail et al. “New Line Voltage Stability Index (BVSI) for Voltage Stability Assessment in Power System: The Comparative Studies”. In: *IEEE Access* (2022). DOI: 10.1109/access.2022.3204792.

[21] Zhihao Jiang et al. “Enhanced dynamic equivalent identification method of large-scale power systems using multiple events”. In: *Electric Power Systems Research* (2020). DOI: 10.1016/j.epsr.2020.106569.

[22] Zhihao Jiang et al. “Measurement-based Power System Dynamic Model Reduction Using ARX Equivalents”. In: (2019). DOI: 10.1109/pesgm40551.2019.8974099.

[23] Zhihao Jiang et al. “Online Tuning of Dynamic Equivalents for Large-Scale Power Systems Using Wide-area Measurements”. In: *2021 IEEE Power & Energy Society General Meeting (PESGM)* (2021). DOI: 10.1109/pesgm46819.2021.9637846.

[24] Lalit Kumar et al. “Voltage Stability Enhancement Using Shunt Devices and Identification of Weak Bus through Voltage Stability Indices”. In: *IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology* (2019). DOI: 10.1109/rteict46194.2019.9016980.

[25] Aoyu Lei et al. “Two-stage transient stability assessment using ensemble learning and cost sensitivity”. In: *Frontiers in Energy Research* (2024). DOI: 10.3389/fenrg.2024.1491846.

[26] Zijian Lv, X. Chen, and Zijian Feng. “Graph Embedding Dynamic Feature-based Supervised Contrastive Learning of Transient Stability for Changing Power Grid Topologies”. In: *arXiv.org* (2023). DOI: 10.48550/arxiv.2308.00537.

[27] Feng Ma and V. Vittal. “A Hybrid Dynamic Equivalent Using ANN-Based Boundary Matching Technique”. In: *IEEE Transactions on Power Systems* (2012). DOI: 10.1109/tpwrs.2012.2182783.

[28] Ehsan Malekshahi and Seyed-Mohammad-Ali Mohammadi. “The model order reduction using LS, RLS and MV estimation methods”. In: *International Journal of Control Automation and Systems* (2014). DOI: 10.1007/s12555-013-0250-2.

[29] Ghanshyam Meena et al. “Voltage Security Assessment Using Different Stability Indices”. In: *2025 IEEE 5th International Conference on Sustainable Energy and Future Electric Transportation (SEFET)* (2025). DOI: 10.1109/sefet65155.2025.11255265.

[30] Tayeb Meridji et al. “A power system stability assessment framework using machine-learning”. In: *Electric power systems research* (2023). DOI: 10.1016/j.epsr.2022.108981.

[31] Aliaa A. Okasha et al. “A GRU-Based Model for Real-Time Transient Stability Prediction in Electrical Power Systems”. In: *Lecture notes in electrical engineering* (2026). DOI: 10.1007/978-981-95-5941-1_14.

[32] Savvas Panagi et al. “Towards an open-source real-time operator-training platform: Analysis of computational efficiency”. In: *Electric Power Systems Research* (2026). DOI: 10.1016/j.epsr.2026.113626.

[33] Miguel Ramírez, Félix Rafael Segundo Sevilla, and Petr Korba. “Power System Transient Stability Prediction with a CNN Trained with Steady-State Input Variables”. In: *2024 8th International Conference on Power and Energy Engineering (ICPEE)* (2024). DOI: 10.1109/icpee64457.2024.10949969.

[34] Uma Rani and Karthika RN. “A Stacked Machine Learning Framework for Predicting Electrical Power Grid Stability”. In: *Research Square (Research Square)* (2024). DOI: 10.21203/rs.3.rs-5734810/v1.

[35] Mahsa Sajjadi et al. “A Participation Factor-Based Approach for Defining the EMT Model Boundary for Power System Simulations with Inverter-Based Resources”. In: *IECON 2020 The 46th Annual Conference of the IEEE Industrial Electronics Society* (2024). DOI: 10.1109/iecon55916.2024.10905745.

[36] Mahsa Sajjadi et al. “Estimation of Participation Factors Using the Synchrosqueezed Wavelet Transform”. In: *IEEE Power & Energy Society General Meeting* (2023). DOI: 10.1109/pesgm52003.2023.10252202.

[37] Reem Salim and M. Bettayeb. “H2 optimal model reduction using Genetic Algorithms and Particle Swarm Optimization”. In: *2009 6th International Symposium on Mechatronics and its Applications* (2009). DOI: 10.1109/isma.2009.5164812.

[38] Reem Salim and Maamar Bettayeb. “H_2 and H_∞ optimal model reduction using genetic algorithms”. In: *Journal of The Franklin Institute-engineering and Applied Mathematics* (2011). DOI: 10.1016/j.jfranklin.2009.10.016.

[39] P. Sarajcev et al. “Power System Transient Stability Assessment Using Stacked Autoencoder and Voting Ensemble”. In: *Energies* (2021). DOI: 10.3390/en14113148.

[40] Zonghe Shao et al. “A Novel Data-Driven LSTM-SAF Model for Power Systems Transient Stability Assessment”. In: *IEEE Transactions on Industrial Informatics* (2024). DOI: 10.1109/tii.2024.3379629.

[41] Noopur Shekhawat, Abhishek Gupta, and A. Kumar Sharma. “Voltage Stability Assessment Using Line Stability Indices”. In: *International Conference on Recent Advances and Innovations in Engineering* (2018). DOI: 10.1109/icraie.2018.8710435.

[42] Afzal Sikander and Rajendra Prasad. “Linear time-invariant system reduction using a mixed methods approach”. In: *Applied Mathematical Modelling* (2015). DOI: 10.1016/j.apm.2015.04.014.

[43] Sungyoon Song, Sang-Won Min, and Seungmin Jung. “Steady-State Data-Driven Dynamic Stability Assessment in the Energy Management System”. In: *Research Square (Research Square)* (2024). DOI: 10.21203/rs.3.rs-3848411/v1.

[44] Valéria Monteiro de Souza, Hugo Rodrigues de Brito, and Kjetil Uhlen. “Comparative analysis of online voltage stability indices based on synchronized PMU measurements”. In: *Sustainable Energy, Grids and Networks* (2024). DOI: 10.1016/j.segan.2024.101544.

[45] Mohamad Fani Sulaima et al. “Risk-aware AI control framework for resilient distribution grids under extreme events”. In: *Energy Conversion and Management X* (2026). DOI: 10.1016/j.ecmx.2026.102115.

[46] Pei-Hao Sun et al. “Rotor Angle Stability Prediction Using Temporal and Topological Embedding Deep Neural Network Based on Grid-informed Adjacency Matrix”. In: *Journal of Modern Power Systems and Clean Energy* (2024). DOI: 10.35833/mpce.2023.000364.

[47] Peiyuan Sun et al. “Fast Transient Stability Prediction Using Grid-informed Temporal and Topological Embedding Deep Neural Network”. In: *ArXiv* (2022).

[48] Yingyun Sun et al. “Reduction of power system dynamic model using Krylov subspace method”. In: *2014 International Conference on Power System Technology* (2014). DOI: 10.1109/powercon.2014.6993633.

[49] Bendong Tan and Junbo Zhao. “Bayesian Post-Fault Power System Dynamic Trajectory Prediction”. In: *IEEE Transactions on Power Systems* (2025). DOI: 10.1109/tpwrs.2025.3534232.

[50] K. Tan and Yun Li. “L_∞ identification and model reduction using a learning genetic algorithm”. In: (2002). DOI: 10.1049/cp:19960711.

[51] C. K. Tang et al. “Transient stability index from conventional time domain simulation”. In: (1994). DOI: 10.1109/59.336108.

[52] Naga Lakshmi Thotakura, Zhihao Jiang, and Yilu Liu. “Time Series Prediction of Power System Events Using LSTM, GRU, and Transformer Networks to Increase Computation Efficiency”. In: *IEEE Power & Energy Society General Meeting* (2024). DOI: 10.1109/pesgm51994.2024.10688391.

[53] Maurizio Titz et al. “Predicting dynamic stability from static features in power grid models using machine learning”. In: *Chaos: An Interdisciplinary Journal of Nonlinear Science* (2024). DOI: 10.1063/5.0175372.

[54] Ning Tong et al. “Dynamic Equivalence of Large-Scale Power Systems Based on Boundary Measurements”. In: *American Control Conference* (2020). DOI: 10.23919/acc45564.2020.9147425.

[55] Ning Tong et al. “Dynamic Model Reduction for Large-Scale Power Systems Using Wide-Area Measurements”. In: *IEEE Access* (2020). DOI: 10.1109/access.2020.2992624.

[56] R. West. “Best practice in statistics: The use of log transformation”. In: *Annals of Clinical Biochemistry* (2021). DOI: 10.1177/00045632211050531.

[57] D. Wilson. “Optimum solution of model-reduction problem”. In: (1970). DOI: 10.1049/piee.1970.0227.

[58] Tianwei Xia et al. “Time-variant Nonlinear Participation Factors Considering Resonances in Power Systems”. In: *2022 IEEE Power & Energy Society General Meeting (PESGM)* (2022). DOI: 10.1109/pesgm48719.2022.9916932.

[59] Rong Yan et al. “Fast Transient Stability Batch Assessment Using Cascaded Convolutional Neural Networks”. In: *IEEE Transactions on Power Systems* (2020). DOI: 10.1109/pesgm41954.2020.9282095.

[60] Xinlin Ye, J. Morales, and J. Milanović. “Truncated Transient Stability Index for On-line Power System Transient Stability Assessment”. In: *IEEE PES Innovative Smart Grid Technologies Conference Europe* (2021). DOI: 10.1109/isgteurope52324.2021.9640058.

[61] James J.Q. Yu et al. “Intelligent Time-Adaptive Transient Stability Assessment System”. In: *IEEE Transactions on Power Systems* (2018). DOI: 10.1109/tpwrs.2017.2707501.

[62] Nan Yu et al. “Transient Stability Assessment of Power Systems Built upon a Deep Spatio-Temporal Feature Extraction Network”. In: *Energies* (2025). DOI: 10.3390/en18174547.

[63] Xuemeng Zhang et al. “Measurement-based power system dynamic model reductions”. In: *2017 North American Power Symposium (NAPS)* (2017). DOI: 10.1109/naps.2017.8107178.

[64] Lipeng Zhu, D. Hill, and Chao Lu. “Hierarchical Deep Learning Machine for Power System Online Transient Stability Prediction”. In: *IEEE Transactions on Power Systems* (2020). DOI: 10.1109/tpwrs.2019.2957377.

[65] Lipeng Zhu et al. “Integrated Data-Driven Power System Transient Stability Monitoring and Enhancement”. In: *IEEE Transactions on Power Systems* (2024). DOI: 10.1109/tpwrs.2023.3266387.

[66] Faycal Znidi, Hamzeh Davarikia, and Heena Rathore. “Power Systems Transient Stability Indices: Hierarchical Clustering Based Detection of Coherent Groups Of Generators”. In: *2024 IEEE Texas Power and Energy Conference (TPEC)* (2021). DOI: 10.1109/tpec60005.2024.10472280.

Appendices

A

Implementation Code Listings

This appendix collects the analytic code fragments that support the Implementation section of Chapter 3. The main text keeps the numerical choices needed to reproduce the experiments; the listings below document the corresponding computational steps.

A.1. Operating point definition and extension

French transmission system.

```
load_names = read_loads_from_csv(csv_path, column_name)
add_to_par_file(par_file_path, load_names)
add_to_dyd_file(dyd_file_path, load_names)

...

def read_loads_from_csv(csv_path, column_name):
    df = pd.read_csv(csv_path)
    return df[column_name].dropna().unique().tolist()

def add_to_par_file(par_file_path, load_names):
    ...
    for load_name in load_names:
        param_line = (
            f'<par type="BOOL" name="{load_name}_isControllable" '
            'value="true"/>'
        )
        new_params.append(param_line)
    ...

def add_to_dyd_file(dyd_file_path, load_names):
    ...
    for load_name in load_names:
        macro_line = (
            '<macroConnect connector="VariationAreaToNetworkLoad" '
            'id1="MODEL_LOAD_VARIATION_AREA" id2="NETWORK" '
            'index1="0" name2="{load_name}"/>'
        )
        new_entries.append(macro_line)
    ...
```

Nordic test system.

```
def sample_area_factors(pattern, rng):
    if pattern == "uniform":
        return {"North": 1.0, "Central": 1.0, "South": 1.0}
    ...

def run_dynawo(execution_path, op_dir, timeout_sec=1800):
    cmd = ["bash", str(execution_path), "jobs", str(op_dir / "Nordic.jobs")]
```

```
    proc = subprocess.run(
        cmd, cwd=str(op_dir), capture_output=True,
        text=True, timeout=timeout_sec, check=False
    )
    output = (proc.stdout or "") + (proc.stderr or "")
    return proc.returncode == 0, output

loads = [
    ("01_1", "South", p_mw_01),
    ...,
    ("72_1", "South", p_mw_72),
]
split_bands = [("train", "low"), ..., ("test", "high")]

for op_number in range(1, num_ops + 1):
    split, band = split_bands[op_number - 1]
    g_lo, g_hi = GLOBAL_BAND_RANGES[band]
    global_factor = rng.uniform(g_lo, g_hi)
    area_pattern = AREA_STRESS_PATTERNS[(op_number - 1) % len(AREA_STRESS_PATTERNS)]
    area_factors = sample_area_factors(area_pattern, rng)

    deltas = {}
    total_target = 0.0
    for load_id, area, p_mw in loads:
        f_load = rng.uniform(*PER_LOAD_RANGE)
        scale = global_factor * area_factors[area] * f_load
        deltas[load_id] = (scale - 1.0, scale - 1.0)
        total_target += p_mw * scale

    # Copy the base Dynawo case into operating_point_<N>.
    # Write Event.dyd and Event.par from the sampled deltas.
    # Patch Network.par so the loads become controllable.
    op_dir = output_root / f"operating_point_{op_number}"

ok, message = run_dynawo(args.execution_path, op_dir)
if ok:
    shutil.copy2(op_dir / "outputs/finalState/outputIIDM.xml", op_dir / "Nordic.xiidm")
```

A.2. Full-detail simulation driver

Full-detail simulation driver.

```
def read_contingencies(csv_path):
    rows = []
    for contingency_id, fault_name, fault_type, op_raw in csv_rows:
        op_keys_set = set(op_raw.split(",")) if op_raw else None
        rows.append((contingency_id, fault_name, fault_type, op_keys_set))
    return rows

def should_contingency_exist(op_key, op_keys_set):
    return op_keys_set is None or op_key in op_keys_set

def create_scenario_folder(op_path, output_dir, contingency_id, fault_name, fault_type, op_name):
    scen_dir = output_dir / op_name / f"contingency_{contingency_id}"
    copy_contents(op_path, scen_dir)
    write_event_files(scen_dir, contingency_id, fault_name, fault_type)
    return scen_dir

def run_job(execution_path, jobs_file, operating_point, contingency):
    success = run_dynawo_job(
        execution_path=execution_path,
        jobs_file=jobs_file,
        operating_point=operating_point,
        contingency=contingency,
    )
    return success

contingencies = read_contingencies(args.contingencies_csv)
operating_points = discover_operating_points(args.snapshots_dir)

```

```
for op_path in operating_points:
    op_key = extract_op_key(op_path.name)
    for contingency_id, fault_name, fault_type, op_keys_set in contingencies:
        if should_contingency_exist(op_key, op_keys_set):
            scen_dir = create_scenario_folder(
                op_path, args.output_dir, contingency_id,
                fault_name, fault_type, op_path.name
            )
            jobs_file = find_jobs_file(str(scen_dir))
            run_success = run_job(
                args.execution_path, jobs_file, op_path.name,
                f"contingency_{contingency_id}"
            )
            status = "Success" if run_success else "Failed"
            write_csv_result(op_path.name, f"contingency_{contingency_id}", status)
```

A.3. KPI Extraction, Disconnection, and Action Flags

KPI Extraction, Disconnection, and Action Flags.

```
records = build_records_for_operating_point(op_dir)
generator_ids_with_snom = {rec.static_id: rec.snom_mva for rec in records}
write_apparent_power_csv(
    time_values, p_by_label, q_by_label, output_path,
    normalize_with_per_label_denominator=generator_ids_with_snom,
    id_to_staticid=id_to_staticid,
)

...

def build_records_for_operating_point(op_dir):
    ...
    if exact_snom is not None:
        chosen = exact_snom
    elif limit_proxy is not None:
        chosen = limit_proxy
    ...

def write_apparent_power_csv(..., normalize_with_per_label_denominator=None, ...):
    s_vals = [math.hypot(p, q) for p, q in zip(p_vals, q_vals)]
    static_id = resolve_generator_static_id(label, id_to_staticid or {})
    denom = normalize_with_per_label_denominator.get(static_id, 0.0)
    if denom:
        s_vals = [s / denom for s in s_vals]
```

KPI Extraction, Disconnection, and Action Flags.

```
scores_bb = compute_max_variance(volt_path, window_sec, step_sec)
scores = aggregate_busbar_metrics_to_voltage_levels(scores_bb, busbar_to_vl)
kpi_voltage_path, kpi_spower_path, kpi_frequency_path = build_kpi_csvs_for_op(
    op_results_dir, op_name, window_sec=5.0, step_sec=1.0, ...
)

...

def compute_max_variance(csv_path, window_sec, step_sec, kpi_t_start=9.0, kpi_t_end=100.0):
    ...
    while t + window_sec <= t_loop_end:
        mask = in_kpi_interval & (time >= t) & (time < t + window_sec)
        if mask.any():
            max_var = max(max_var, float(np.var(series[mask])))
        t += step_sec
    return scores

def build_kpi_csvs_for_op(op_results_dir, op_name, window_sec=5.0, step_sec=1.0, ...):
    scores_bb = compute_max_variance(volt_path, window_sec, step_sec)
    scores = aggregate_busbar_metrics_to_voltage_levels(scores_bb, busbar_to_vl)
    ...
    scores = compute_max_variance(spower_path, window_sec, step_sec)
```

KPI Extraction, Disconnection, and Action Flags.

```
vl_t, vl_n, g_t, g_n = compute_disconnection_flags_for_case(
    local_cn_dir, ..., voltage_csv_path=voltage_csv_path
)

...

def compute_disconnection_flags_for_case(local_cn_dir, ..., voltage_csv_path=None):
    switch_open, gen_dm = parse_timeline_events(timeline)
    ...
    bb_trip, _ = deenergized_ids_from_voltage_csv(
        voltage_csv_path=voltage_csv,
        zero_threshold=voltage_zero_threshold,
        energized_threshold=voltage_energized_threshold,
        hold_seconds=voltage_hold_seconds,
    )
    vl_trip = expand_deenergized_columns_to_voltage_levels(bb_trip, busbar_to_vl, vl_ids)
    ...
    for dm in gen_dm:
        sid = dm_to_static.get(dm)
        if sid:
            static_trip.add(sid)
```

KPI Extraction, Disconnection, and Action Flags.

```
out = process_case_zip(local_zip)
voltage_path, spower_path = build_csvs_for_op(
    op_name, case_order, ..., output_dir
)

...

def process_case_zip(zip_path):
    timeline_bytes, iidm_bytes, dyd_bytes = payload
    action_models = parse_timeline_events(timeline_bytes)
    ...
    for model_name in action_models:
        impacted_components |= resolve_impacted_components_for_model(...)
    ...
    for vl in impacted_vls:
        impacted_gens |= {gid for gid in vl_to_gens.get(vl, set()) if gid in dynamic_gens}

def build_csvs_for_op(op_name, case_order, ..., output_dir):
    voltage_cols = ["OP", "Contingency"] + voltage_components
    spower_cols = ["OP", "Contingency"] + spower_components
    ...
    row[comp] = 1 / 0 / None
```

A.4. Implementation of the First Approach: GAT-Based Prediction

Normalization and activity-class construction.

```
def normalize_columns_minmax(df, value_cols, *, target_min=0.0, target_max=1.0):
    numeric_cols = {col: pd.to_numeric(df[col], errors="coerce") for col in value_cols}
    global_min = min(s.dropna().min() for s in numeric_cols.values() if not s.dropna().empty)
    global_max = max(s.dropna().max() for s in numeric_cols.values() if not s.dropna().empty)
    scale = global_max - global_min
    out_span = target_max - target_min
    for col, series in numeric_cols.items():
        df[col] = target_min + ((series - global_min) / scale) * out_span
    return df, float(global_min), float(global_max)

def kpi_value_to_class(series, *, class_cuts, norm_target_min=0.0, norm_target_max=1.0):
    numeric = pd.to_numeric(series, errors="coerce")
    span = float(norm_target_max) - float(norm_target_min)
    unit = (numeric - norm_target_min) / span
    clipped = unit.clip(lower=0.0, upper=1.0)
    cuts = np.asarray(class_cuts, dtype=float)
    class_idx = np.searchsorted(cuts, clipped.to_numpy(dtype=float), side="right")
```

```
    classes = pd.Series(class_idx, index=series.index, dtype="Int64")
    return classes.mask(numeric.isna())

def build_class_dataset(input_table, output_table, action_table=None,
                        disconnection_table=None, *, class_cuts,
                        norm_target_min=0.0, norm_target_max=1.0):
    df = read_output_table(input_table)
    cuts_arr = np.asarray(class_cuts, dtype=float)
    disconnect_class = int(cuts_arr.size) + 1
    value_cols = [c for c in df.columns if c not in ID_COLS]
    out = df.copy()
    for col in value_cols:
        out[col] = kpi_value_to_class(
            out[col], class_cuts=cuts_arr,
            norm_target_min=norm_target_min, norm_target_max=norm_target_max,
        )
    if action_table is not None and action_table.exists():
        out = _apply_flag_mask_to_matching_rows(
            out, read_output_table(action_table), replacement_value=disconnect_class
        )
    if disconnection_table is not None and disconnection_table.exists():
        out = _apply_flag_mask_to_matching_rows(
            out, read_output_table(disconnection_table),
            replacement_value=disconnect_class
        )
    write_output_table(out, output_table)
    return output_table

df_norm, global_min, global_max = normalize_columns_minmax(
    df, value_cols, target_min=0.0, target_max=1.0
)
build_class_dataset(
    kpi_table, class_table, action_table, disc_table, class_cuts=cuts_arr
)
```

Normalization and activity-class construction.

```
{
  "voltage": {
    "cuts": [
      0.1111111111111111, 0.2222222222222222,
      0.3333333333333333, 0.4444444444444444,
      0.5555555555555556, 0.6666666666666666,
      0.7777777777777778, 0.8888888888888888
    ]
  },
  "spower": {
    "cuts": [
      0.1111111111111111, 0.2222222222222222,
      0.3333333333333333, 0.4444444444444444,
      0.5555555555555556, 0.6666666666666666,
      0.7777777777777778, 0.8888888888888888
    ]
  }
}
```

Power-system graph representation.

```
EDGE_R_COL, EDGE_X_COL = 2, 3

def _edge_weight_from_rx(r, x, *, min_weight=1e-6):
    z_mag = float(np.hypot(float(r), float(x)))
    if not np.isfinite(z_mag):
        return min_weight
    return max(z_mag, min_weight)

def _weighted_adjacency_for_sample(data_i, *, build_csr=True):
    num_nodes = int(data_i.x.shape[0])
    ei = data_i.edge_index.detach().cpu()
```

```
    ea = data_i.edge_attr.detach().cpu()
    src, dst = ei[0].numpy(), ei[1].numpy()
    r_col, x_col = ea[:, EDGE_R_COL].numpy(), ea[:, EDGE_X_COL].numpy()
    best_pair_weight = {}
    for idx in range(int(src.shape[0])):
        s, d = int(src[idx]), int(dst[idx])
        if not (0 <= s < num_nodes and 0 <= d < num_nodes) or s == d:
            continue
        w = _edge_weight_from_rx(float(r_col[idx]), float(x_col[idx]))
        a, b = (s, d) if s < d else (d, s)
        prev = best_pair_weight.get((a, b))
        if prev is None or w < prev:
            best_pair_weight[(a, b)] = w
    adj = [[] for _ in range(num_nodes)]
    for (a, b), w in best_pair_weight.items():
        adj[a].append((b, w))
        adj[b].append((a, w))
    csr = None
    if build_csr and best_pair_weight:
        rows, cols, data = [], [], []
        for (a, b), w in best_pair_weight.items():
            rows.extend((a, b)); cols.extend((b, a)); data.extend((w, w))
        csr = csr_matrix((data, (rows, cols)), shape=(num_nodes, num_nodes))
    return adj, csr

def _dijkstra_distances_multi_source(adj, sources, csr_graph=None):
    n = len(adj)
    dist = np.full(n, np.inf, dtype=np.float64)
    heap, seen = [], set()
    for src in sources:
        s = int(src)
        if not (0 <= s < n) or s in seen:
            continue
        seen.add(s)
        dist[s] = 0.0
        heapq.heappush(heap, (0.0, s))
    while heap:
        d_cur, node = heapq.heappop(heap)
        if d_cur > dist[node]:
            continue
        for nb, w in adj[node]:
            d_next = d_cur + float(w)
            if d_next < dist[nb]:
                dist[nb] = d_next
                heapq.heappush(heap, (d_next, int(nb)))
    return dist

def _event_anchor_nodes(data_i, loc_type, loc_idx):
    num_nodes = int(data_i.x.shape[0])
    if loc_type == "node":
        idx = int(loc_idx)
        return [idx] if 0 <= idx < num_nodes else []
    if loc_type == "edge":
        eidx = int(loc_idx)
        s = int(data_i.edge_index[0, eidx].item())
        d = int(data_i.edge_index[1, eidx].item())
        return [n for n in (s, d) if 0 <= n < num_nodes]
    return []

def append_electrical_distance_feature(graph_dataset):
    for data_i in graph_dataset:
        num_nodes = int(data_i.x.shape[0])
        anchors = _event_anchor_nodes(
            data_i, str(data_i.event_location_type), int(data_i.event_location_index)
        )
        if not anchors:
            dz = np.zeros(num_nodes, dtype=np.float64)
        else:
            adj, csr_g = _weighted_adjacency_for_sample(data_i, build_csr=True)
            min_dist = _dijkstra_distances_multi_source(adj, anchors, csr_graph=csr_g)
            finite = np.isfinite(min_dist)
```

```
            if finite.any():
                max_finite = float(min_dist[finite].max())
                min_dist[~finite] = max_finite + 1.0
                dz = np.log1p(min_dist)
            else:
                dz = np.zeros(num_nodes, dtype=np.float64)
        dz_tensor = torch.tensor(dz, dtype=data_i.x.dtype, device=data_i.x.device)
        data_i.x = torch.cat([data_i.x, dz_tensor.unsqueeze(1)], dim=1)
        data_i.dz_fault = dz_tensor
    return graph_dataset

graph_dataset, skipped = build_graph_dataset_from_csv(dataset, graph_dir)
graph_dataset = append_electrical_distance_feature(graph_dataset)
```

Training architecture and ordinal regression.

```
class GAT_V(nn.Module):
    def __init__(self, *, in_channels, edge_dim, hidden_channels, hidden_dim,
                 num_layers, num_gnn_layers, num_classes, num_heads, dropout):
        super().__init__()
        self.convs, self.norms = nn.ModuleList(), nn.ModuleList()
        h = hidden_channels * num_heads
        self.convs.append(GATv2Conv(
            in_channels=in_channels, out_channels=hidden_channels,
            heads=num_heads, edge_dim=edge_dim, dropout=dropout, concat=True
        ))
        self.norms.append(nn.LayerNorm(h))
        for _ in range(int(num_gnn_layers) - 1):
            self.convs.append(GATv2Conv(
                in_channels=h, out_channels=hidden_channels,
                heads=num_heads, edge_dim=edge_dim, dropout=dropout, concat=True
            ))
            self.norms.append(nn.LayerNorm(h))
        self.dropout = nn.Dropout(dropout)
        lin_layers = [nn.Linear(h, hidden_dim), nn.ReLU(), nn.Dropout(dropout)]
        for _ in range(int(num_layers) - 1):
            lin_layers += [nn.Linear(hidden_dim, hidden_dim), nn.ReLU(), nn.Dropout(dropout)]
        lin_layers += [nn.Linear(hidden_dim, num_classes - 1)]
        self.head = nn.Sequential(*lin_layers)

    def forward(self, x, edge_index, edge_attr, bus_node_mask):
        for conv, norm in zip(self.convs, self.norms):
            x = conv(x, edge_index, edge_attr)
            x = norm(x)
            x = F.relu(x)
            x = self.dropout(x)
        return self.head(x[bus_node_mask])

def coral_transform(y, num_classes):
    thresholds = torch.arange(num_classes - 1, device=y.device, dtype=y.dtype).unsqueeze(0)
    return (y.unsqueeze(1) > thresholds).float()

def coral_loss(*, logits, y, num_classes, pos_weight,
               under_penalty_lambda, high_class_threshold):
    targets = coral_transform(y, num_classes)
    bce = F.binary_cross_entropy_with_logits(
        logits, targets, pos_weight=pos_weight, reduction="mean"
    )
    if under_penalty_lambda and high_class_threshold is not None:
        expected_class = torch.sigmoid(logits).sum(dim=1)
        under_amount = F.relu(y.float() - expected_class)
        high_mask = y >= int(high_class_threshold)
        under_penalty = (under_amount[high_mask] ** 2).mean() if high_mask.any() else 0.0
        return bce + float(under_penalty_lambda) * under_penalty
    return bce

def coral_predict(logits, threshold):
    return (torch.sigmoid(logits) > float(threshold)).sum(dim=1).long()

def run_epoch(*, model, loader, optimizer, decode_threshold, device,
```

```
            num_classes, pos_weight, under_penalty_lambda, high_class_threshold):
    train_mode = optimizer is not None
    model.train() if train_mode else model.eval()
    loss_list, correct, total = [], 0, 0
    with torch.set_grad_enabled(train_mode):
        for data in loader:
            data = data.to(device)
            logits = model(data.x, data.edge_index, data.edge_attr, data.bus_node_mask)
            y = data.y_class[data.bus_node_mask].long().to(device)
            loss = coral_loss(
                logits=logits, y=y, num_classes=num_classes, pos_weight=pos_weight,
                under_penalty_lambda=under_penalty_lambda,
                high_class_threshold=high_class_threshold,
            )
            if train_mode:
                optimizer.zero_grad()
                loss.backward()
                torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                optimizer.step()
            pred = coral_predict(logits.detach(), threshold=decode_threshold)
            loss_list.append(float(loss.detach().cpu()))
            correct += int((pred == y).sum().item())
            total += int(y.numel())
    return {"loss": float(np.mean(loss_list)), "acc": correct / max(total, 1)}
```

Hyperparameter selection and inference.

```
def composite_selection_score(metrics, *, f1_weight, loss_weight):
    return (
        float(metrics["high_recall"])
        + float(f1_weight) * float(metrics["high_f1"])
        - float(loss_weight) * float(metrics["loss"])
    )

def objective(trial):
    hp = _sample_hparams(trial, hparam_space)
model = GAT_V(
        in_channels=in_channels, edge_dim=edge_dim,
        hidden_channels=hp.hidden_channels, hidden_dim=hp.hidden_dim,
        num_layers=hp.num_layers, num_gnn_layers=hp.num_gnn_layers,
        num_classes=num_classes, num_heads=hp.num_heads, dropout=hp.dropout,
    ).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=hp.lr, weight_decay=hp.weight_decay)
    best_score = -float("inf")
    for epoch in range(epochs):
        run_epoch(
            model=model, loader=train_loader, optimizer=optimizer,
            decode_threshold=hp.coral_prediction_threshold, device=device,
            num_classes=num_classes, pos_weight=pos_weight,
            under_penalty_lambda=hp.under_penalty_lambda,
            high_class_threshold=high_class_threshold,
        )
        val_m = evaluate_detailed(
            model=model, loader=val_loader, device=device,
            num_classes=num_classes,
            decode_threshold=hp.coral_prediction_threshold,
            pos_weight=pos_weight,
            under_penalty_lambda=hp.under_penalty_lambda,
            high_class_threshold=high_class_threshold,
        )
        score = composite_selection_score(
            val_m, f1_weight=selection_f1_weight, loss_weight=selection_loss_weight
        )
        best_score = max(best_score, score)
    return float(best_score)

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=n_trials)

model.eval()
```

```
with torch.no_grad():
    logits = model(data.x, data.edge_index, data.edge_attr, data.bus_node_mask)
    y_pred = coral_predict(logits, threshold=decode_threshold)
```

A.5. Implementation of the Second Approach: Pair-Aware GINE-Based Prediction

Physically aware cut selection and label construction.

```
def assign_class(value, cuts):
    if not np.isfinite(value):
        return -1
    t0, t1, t2, t3 = cuts
    if value <= t0:
        return 0
    if value <= t1:
        return 1
    if value <= t2:
        return 2
    if value <= t3:
        return 3
    return 4

def usability_score(dynamic_shares):
    entropy = normalized_entropy(dynamic_shares)
    class_floor = min(
        clipped_ratio(dynamic_shares[1], MIN_DYNAMIC_CLASS2_SHARE),
        clipped_ratio(dynamic_shares[2], MIN_DYNAMIC_CLASS3_SHARE),
        clipped_ratio(dynamic_shares[3], MIN_DYNAMIC_CLASS4_SHARE),
    )
    class1_score = (
        1.0 if dynamic_shares[0] <= MAX_DYNAMIC_CLASS1_SHARE
        else float(np.clip(
            (1.0 - dynamic_shares[0]) / (1.0 - MAX_DYNAMIC_CLASS1_SHARE), 0.0, 1.0
        ))
    )
    return (
        USABILITY_W_ENTROPY * entropy
        + USABILITY_W_FLOOR * class_floor
        + USABILITY_W_CLASS1 * class1_score
    )

def physical_score(samples, cuts):
    kpis = np.array([s.kpi_value for s in samples], dtype=float)
    phis = np.array([s.phi for s in samples], dtype=float)
    classes = np.array([assign_class(v, cuts) for v in kpis], dtype=float)
    valid = classes >= 0
    spearman = safe_spearman(phis[valid], classes[valid])
    means = [
        float(np.mean(phis[classes == c])) if np.any(classes == c) else np.nan
        for c in range(5)
    ]
    mono_pairs = mono_ok = 0
    for a, b in zip(means[:-1], means[1:]):
        if np.isfinite(a) and np.isfinite(b):
            mono_pairs += 1
            mono_ok += int(b + 1e-9 >= a)
    mono = mono_ok / mono_pairs if mono_pairs else float("nan")
    sp = 0.0 if not np.isfinite(spearman) else max(spearman, 0.0)
    mn = 0.0 if not np.isfinite(mono) else mono
    return PHYSICAL_W_SPEARMAN * sp + PHYSICAL_W_MONO * mn

def write_class_bins_json(path, voltage_cuts, spower_cuts):
    payload = {
        "voltage": {"cuts": [float(x) for x in voltage_cuts]},
        "spower": {"cuts": [float(x) for x in spower_cuts]},
    }
    path.write_text(json.dumps(payload, indent=2) + "\n", encoding="utf-8")

```

```python
def kpi_values_to_class_array(kpi_arr, raw_cuts):
    out = np.full(kpi_arr.shape, np.nan, dtype=float)
    finite = np.isfinite(kpi_arr)
    out[finite] = np.searchsorted(raw_cuts, kpi_arr[finite], side="left")
    return out

def build_full_dataset_for_type(*, kpi_path, action_path, disconnection_path,
                            raw_cuts, output_dataset_path,
                            apply_disconnection_mask):
    kpi_df = pd.read_parquet(kpi_path)
    value_cols = [c for c in kpi_df.columns if c not in ID_COLS]
    raw_cuts = np.asarray(raw_cuts, dtype=float)
    action_disconnect_class = int(raw_cuts.size) + 1
    class_df = kpi_df[ID_COLS].copy()
    values = kpi_df[value_cols].to_numpy(dtype=float)
    for j, col in enumerate(value_cols):
        class_df[col] = pd.Series(
            kpi_values_to_class_array(values[:, j], raw_cuts), dtype="Int64"
        )
    class_df = _apply_flag_mask_to_matching_rows(
        class_df, pd.read_parquet(action_path),
        replacement_value=action_disconnect_class,
    )
    if apply_disconnection_mask:
        class_df = _apply_flag_mask_to_matching_rows(
            class_df, pd.read_parquet(disconnection_path),
            replacement_value=action_disconnect_class,
        )
    class_df.to_parquet(output_dataset_path, index=False)
    return output_dataset_path

write_class_bins_json(class_bins_path, voltage_cuts, spower_cuts)
build_full_dataset_for_type(
    kpi_path=kpi_path, action_path=action_path,
    disconnection_path=disc_path, raw_cuts=cuts,
    output_dataset_path=out_path, apply_disconnection_mask=True,
)
```

Physically aware cut selection and label construction.

```json
{
  "voltage": {
    "cuts": [1e-07, 5.000000000000001e-07, 2.5e-06, 1e-05]
  },
  "spower": {
    "cuts": [1e-07, 5.000000000000001e-07, 2.5e-06, 7.5e-06]
  }
}
```

Power-system graph representation.

```python
def _node_feature(node_type, *, v=0.0, angle=0.0, p=0.0, q=0.0, fault_on=False):
    return [
        float(node_type),
        float(v),
        float(angle),
        float(p),
        float(q),
        1.0 if fault_on else 0.0,
    ]

def _edge_feature(edge_type, *, r=0.0, x=0.0, b1=0.0, g1=0.0, b2=0.0, g2=0.0,
                  fault_on=False):
    return [
        float(edge_type),
        1.0 if fault_on else 0.0,
        float(r),
        float(x),
        float(b1),
```

```
        float(g1),
        float(b2),
        float(g2),
    ]

def process_distance_task(iidm_path, output_csv):
    iidm = Path(iidm_path)
    op_name = iidm.name if iidm.is_dir() else iidm.parent.name
    try:
        row_count = write_electrical_distance_csv_from_iidm(
            iidm, Path(output_csv), store_matrices=False,
        )
        return op_name, row_count, None
    except Exception as exc:
        return op_name, 0, str(exc)

for op_dir in operating_points:
    output_path = graph_output / f"{op_dir.name}.pt"
    distance_csv = electric_output / f"{op_dir.name}.csv"
    dyd_path = resolve_op_dyd_path(op_dir)
    data, metadata = build_graph(op_dir, dyd_path=dyd_path)
    torch.save({"data": data, "metadata": metadata}, output_path)
    process_distance_task(str(op_dir), str(distance_csv))
```

Power-system graph representation.

```
def branch_admittances(r, x):
    r = np.asarray(r, dtype=np.float64)
    x = np.asarray(x, dtype=np.float64)
    z = r + 1j * x
    y = np.zeros_like(z, dtype=np.complex128)
    nonzero = np.abs(z) > 0.0
    y[nonzero] = 1.0 / z[nonzero]
    return y

def build_y_bus_sparse(n, branch_i, branch_j, branch_y,
                       branch_y_shunt_i=None, branch_y_shunt_j=None):
    if branch_i.size == 0:
        return csc_matrix((n, n), dtype=np.complex128)
    branch_i = np.asarray(branch_i, dtype=np.intp)
    branch_j = np.asarray(branch_j, dtype=np.intp)
    branch_y = np.asarray(branch_y, dtype=np.complex128)
    if branch_y_shunt_i is None:
        branch_y_shunt_i = np.zeros(branch_y.shape, dtype=np.complex128)
    if branch_y_shunt_j is None:
        branch_y_shunt_j = np.zeros(branch_y.shape, dtype=np.complex128)
    keep = (branch_y != 0) | (branch_y_shunt_i != 0) | (branch_y_shunt_j != 0)
    branch_i, branch_j = branch_i[keep], branch_j[keep]
    branch_y = branch_y[keep]
    branch_y_shunt_i = branch_y_shunt_i[keep]
    branch_y_shunt_j = branch_y_shunt_j[keep]
    rows = np.concatenate([branch_i, branch_j, branch_i, branch_j])
    cols = np.concatenate([branch_i, branch_j, branch_j, branch_i])
    data = np.concatenate([
        branch_y + branch_y_shunt_i,
        branch_y + branch_y_shunt_j,
        -branch_y,
        -branch_y,
    ])
    y_bus = coo_matrix((data, (rows, cols)), shape=(n, n)).tocsc()
    y_bus.eliminate_zeros()
    return y_bus

def write_electrical_distance_csv_from_iidm(iidm_path, output_csv, *,
                                        mode=_MODE_VOLTAGE_LEVEL,
                                        store_matrices=False):
    del store_matrices
    iidm_path = _resolve_iidm_path(Path(iidm_path))
    network = pp.network.load(str(iidm_path))
    return write_electrical_distance_csv_from_network(
```

```
        network, output_csv, mode=mode, source_iidm_path=str(iidm_path),
    )
```

Power-system graph representation.

```
def _distances_from_precomputed(dist_by_vli, source_vls, target_vls):
    out = np.full(len(target_vls), np.inf, dtype=np.float64)
    for vli in source_vls:
        series = dist_by_vli.get(vli)
        if series is None:
            continue
        d = series.reindex(target_vls).to_numpy(dtype=np.float64, copy=False)
        out = np.minimum(out, d)
    return out

def append_electrical_distance_feature(graph_dataset, *, graph_dir,
                                       electric_distance_dir, logger):
    stats = {
        "total": len(graph_dataset), "appended": 0, "missing_event": 0,
        "empty_graph": 0, "all_disconnected": 0, "missing_op_csv": 0,
    }
    for data_i in graph_dataset:
        num_nodes = int(data_i.x.shape[0])
        if num_nodes <= 0:
            stats["empty_graph"] += 1
            continue
        loc_type = str(getattr(data_i, "event_location_type", "missing"))
        loc_idx = int(getattr(data_i, "event_location_index", -1))
        anchors = _event_anchor_nodes(data_i, loc_type, loc_idx)
        if not anchors:
            dz = np.zeros(num_nodes, dtype=np.float64)
            stats["missing_event"] += 1
        else:
            try:
                dist_by_vli, idx_to_bus_vl = _get_op_electric_context(
                    str(getattr(data_i, "op_name", "")),
                    graph_dir, electric_distance_dir,
                )
            except FileNotFoundError:
                dz = np.zeros(num_nodes, dtype=np.float64)
                stats["missing_op_csv"] += 1
            else:
                source_vls = []
                for a in anchors:
                    ai = int(a)
                    if 0 <= ai < num_nodes:
                        vl = idx_to_bus_vl[ai]
                        if vl:
                            source_vls.append(vl)
                source_vls = list(dict.fromkeys(source_vls))
                target_vls = sorted({vl for vl in idx_to_bus_vl if vl})
                if not source_vls or not target_vls:
                    dz = np.zeros(num_nodes, dtype=np.float64)
                    stats["missing_event"] += 1
                else:
                    min_dist = _distances_from_precomputed(
                        dist_by_vli, source_vls, target_vls
                    )
                    finite = np.isfinite(min_dist)
                    if finite.any():
                        max_finite = float(min_dist[finite].max())
                        min_dist[~finite] = max_finite + 1.0
                        dz_by_vl = {
                            vl: float(np.log1p(d))
                            for vl, d in zip(target_vls, min_dist)
                        }
                    else:
                        dz_by_vl = {}
                        stats["all_disconnected"] += 1
                    dz = np.array(
```

```
                    [dz_by_vl.get(idx_to_bus_vl[i], 0.0)
                     for i in range(num_nodes)],
                    dtype=np.float64,
                )
        dz_tensor = torch.tensor(dz, dtype=data_i.x.dtype, device=data_i.x.device)
        data_i.x = torch.cat([data_i.x, dz_tensor.unsqueeze(1)], dim=1)
        data_i.dz_fault = dz_tensor
        stats["appended"] += 1
    logger.info("Electrical distance appended. stats=%s", stats)
    return stats

graph_dataset, skipped = build_graph_dataset_multi(
    dataset_voltage=dataset_voltage,
    dataset_spower=dataset_spower,
    graph_dir=OP_GRAPHS_DIR,
    num_classes=num_classes,
    country_filter=country_filter,
    logger=logger,
)
append_electrical_distance_feature(
    graph_dataset,
    graph_dir=OP_GRAPHS_DIR,
    electric_distance_dir=OP_ELECTRIC_DISTANCE_DIR,
    logger=logger,
)
```

Pair-aware GINE architecture.

```
class ResidualGINEBlock(nn.Module):
    def __init__(self, hidden_dim: int, dropout: float) -> None:
        super().__init__()
        mlp = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim * 2),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim * 2, hidden_dim),
        )
        self.conv = GINEConv(mlp, train_eps=True)
        self.norm = nn.LayerNorm(hidden_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor, edge_index: torch.Tensor, edge_attr: torch.Tensor) -> torch.Tensor:
        update = self.conv(x, edge_index, edge_attr)
        return F.relu(self.norm(x + self.dropout(update)))


class PairAwareGINE(nn.Module):
    """Direct event- and target-conditioned GINE model.

    This is intentionally close to the strongest pair-aware residual encoder,
    but removes every historical KPI prior. The output heads predict from the
    graph representation itself.

    ``num_classes`` is the total number of output classes including the flag
    class. Classes ``0 .. num_classes-2`` are KPI-derived activity levels;
    class ``num_classes-1`` is the flag (disconnected/controlled) class.
    """

    def __init__(
        self,
        *,
        num_node_tokens: int,
        num_contingency_tokens: int,
        target_mask_attr: str,
        hparams: PairAwareHParams,
        num_classes: int,
        num_node_types: int = 3,
        num_edge_types: int = 3,
    ) -> None:
        super().__init__()
```

```
        h = int(hparams.hidden_dim)
        self.hparams = hparams
        self.target_mask_attr = str(target_mask_attr)
        self.num_classes = int(num_classes)

        self.node_type_embedding = nn.Embedding(num_node_types, hparams.type_dim)
        self.edge_type_embedding = nn.Embedding(num_edge_types, hparams.type_dim)
        self.node_token_embedding = nn.Embedding(num_node_tokens, hparams.node_id_dim)
        self.contingency_embedding = nn.Embedding(num_contingency_tokens, hparams.contingency_id_dim)
        self.event_type_embedding = nn.Embedding(2, hparams.type_dim)

        # Node continuous input: v, angle, p, q, fault_on, electrical distance.
        self.node_input = nn.Sequential(
            nn.Linear(hparams.type_dim + hparams.node_id_dim + 6, h),
            nn.ReLU(),
            nn.LayerNorm(h),
        )
        # Edge continuous input: fault_on, r, x, b1, g1, b2, g2.
        self.edge_input = nn.Sequential(
            nn.Linear(hparams.type_dim + 7, h),
            nn.ReLU(),
            nn.LayerNorm(h),
        )

        self.blocks = nn.ModuleList(
            [ResidualGINEBlock(h, hparams.dropout) for _ in range(hparams.num_gnn_layers)]
        )
        self.jumping_projection = nn.Sequential(
            nn.Linear(h * (hparams.num_gnn_layers + 1), h),
            nn.ReLU(),
            nn.LayerNorm(h),
        )

        self.event_encoder = nn.Sequential(
            nn.Linear(h + h + hparams.type_dim + hparams.contingency_id_dim, h),
            nn.ReLU(),
            nn.Dropout(hparams.dropout),
            nn.Linear(h, h),
            nn.ReLU(),
        )
        self.event_to_node = nn.Linear(h, h)

        self.target_pair_projection = nn.Linear(hparams.node_id_dim, hparams.pair_dim)
        self.contingency_pair_projection = nn.Linear(hparams.contingency_id_dim, hparams.pair_dim)

        # target h + event h + global(mean|max)=2h + |h-e| + h*e + dz
        # + explicit target ID + explicit contingency ID + pair interaction
        decoder_in = (
            h * 6
            + 1
            + hparams.node_id_dim
            + hparams.contingency_id_dim
            + hparams.pair_dim
        )
        self.shared_decoder = nn.Sequential(
            nn.Linear(decoder_in, hparams.decoder_hidden_dim),
            nn.ReLU(),
            nn.Dropout(hparams.dropout),
            nn.Linear(hparams.decoder_hidden_dim, hparams.decoder_hidden_dim),
            nn.ReLU(),
            nn.Dropout(hparams.dropout),
        )
        self.class_head = nn.Linear(hparams.decoder_hidden_dim, self.num_classes)
        self.inactive_head = nn.Linear(hparams.decoder_hidden_dim, 1)
        self.regression_head = nn.Linear(hparams.decoder_hidden_dim, 1)

    def forward(self, data) -> dict[str, torch.Tensor]:
        x = data.x
        edge_attr = data.edge_attr
        edge_index = data.edge_index
        batch = data.batch.to(x.device)
```

```
        num_graphs = int(data.num_graphs)

        node_type = x[:, 0].long().clamp(min=0, max=self.node_type_embedding.num_embeddings - 1)
        node_token = data.node_token.long().clamp(min=0, max=self.node_token_embedding.num_embeddings - 1)
        node_id_embedding = self.node_token_embedding(node_token)
        node_cont = torch.cat([x[:, 1:5], x[:, 5:6], x[:, 6:7]], dim=1)
        h = self.node_input(
            torch.cat(
                [self.node_type_embedding(node_type), node_id_embedding, node_cont],
                dim=1,
            )
        )

        edge_type = edge_attr[:, 0].long().clamp(min=0, max=self.edge_type_embedding.num_embeddings - 1)
        edge_cont = edge_attr[:, 1:8]
        e = self.edge_input(torch.cat([self.edge_type_embedding(edge_type), edge_cont], dim=1))

        states = [h]
        for block in self.blocks:
            h = block(h, edge_index, e)
            states.append(h)
        h = self.jumping_projection(torch.cat(states, dim=1))

        graph_mean = _safe_global_mean_pool(h, batch, size=num_graphs)
        graph_max = _safe_global_max_pool(h, batch, size=num_graphs)
        graph_context = torch.cat([graph_mean, graph_max], dim=1)

        event_node_mask = data.event_node_mask.bool()
        if not bool(event_node_mask.any()):
            raise RuntimeError("A batch contains no event anchor nodes")
        event_node_pool = _safe_global_mean_pool(
            h[event_node_mask], batch[event_node_mask], size=num_graphs
        )

        event_edge_mask = data.event_edge_mask.bool()
        if bool(event_edge_mask.any()):
            edge_batch = batch[edge_index[0]]
            event_edge_pool = _safe_global_mean_pool(
                e[event_edge_mask], edge_batch[event_edge_mask], size=num_graphs
            )
        else:
            event_edge_pool = h.new_zeros((num_graphs, h.shape[1]))

        event_type = data.event_graph_type.view(-1).long().clamp(min=0, max=1)
        contingency_token = data.contingency_token.view(-1).long().clamp(
            min=0, max=self.contingency_embedding.num_embeddings - 1
        )
        contingency_embedding = self.contingency_embedding(contingency_token)
        event_context = self.event_encoder(
            torch.cat(
                [
                    event_node_pool,
                    event_edge_pool,
                    self.event_type_embedding(event_type),
                    contingency_embedding,
                ],
                dim=1,
            )
        )

        target_mask = getattr(data, self.target_mask_attr).bool()
        target_batch = batch[target_mask]
        target_h = h[target_mask]
        target_event = event_context[target_batch]
        target_global = graph_context[target_batch]
        event_as_node = self.event_to_node(target_event)
        dz = x[target_mask, 6:7]

        target_node_id = node_id_embedding[target_mask]
        target_contingency = contingency_embedding[target_batch]
        pair_interaction = torch.tanh(
```

```
            self.target_pair_projection(target_node_id)
            * self.contingency_pair_projection(target_contingency)
        )

        parts = [
            target_h,
            target_event,
            target_global,
            torch.abs(target_h - event_as_node),
            target_h * event_as_node,
            dz,
            target_node_id,
            target_contingency,
            pair_interaction,
        ]
        shared = self.shared_decoder(torch.cat(parts, dim=1))
        return {
            "class_logits": self.class_head(shared),
            "inactive_logit": self.inactive_head(shared).squeeze(1),
            "log_kpi_std": self.regression_head(shared).squeeze(1),
        }
```

Training objective.

```
def selection_score(metrics, weights=None):
    w = weights or SelectionScoreWeights()
    return float(
        w.balanced_accuracy_weight * metrics["balanced_accuracy"]
        + w.macro_f1_weight * metrics["macro_f1"]
        + w.accuracy_weight * metrics["accuracy"]
        + w.within_one_weight * metrics["within_one_accuracy"]
    )

def _ordinal_cdf_loss(logits, y, num_classes):
    probabilities = torch.softmax(logits, dim=1)
    pred_cdf = probabilities.cumsum(dim=1)[:, :-1]
    true_cdf = F.one_hot(y, num_classes=int(num_classes)).float().cumsum(dim=1)[:, :-1]
    return torch.abs(pred_cdf - true_cdf).mean()

def _decode_gated(logits, gate_logit, threshold):
    inactive_probability = torch.sigmoid(gate_logit)
    active_pred = logits[:, 1:].argmax(dim=1) + 1
    return torch.where(
        inactive_probability >= float(threshold),
        torch.zeros_like(active_pred),
        active_pred,
    )

# Inside _run_epoch, after model(data) and target-mask filtering:
classification_loss = F.cross_entropy(logits, y, weight=class_weights)
gate_target = (y == 0).float()
gate_loss = F.binary_cross_entropy_with_logits(
    gate_logit, gate_target, pos_weight=inactive_pos_weight
)
ordinal_loss = _ordinal_cdf_loss(logits, y, num_classes)
finite_reg = torch.isfinite(log_target)
regression_loss = (
    F.smooth_l1_loss(reg_prediction[finite_reg], log_target[finite_reg])
    if bool(finite_reg.any()) else logits.new_zeros(())
)
total_loss = (
    loss_weights.classification * classification_loss
    + loss_weights.regression * regression_loss
    + loss_weights.inactive_gate * gate_loss
    + loss_weights.ordinal_cdf * ordinal_loss
)
```

Hyperparameter selection and inference.

```
def _sample_hparams(trial, space):
    return PairAwareHParams(
        hidden_dim=int(_suggest(trial, "hidden_dim", space["hidden_dim"])),
        node_id_dim=int(_suggest(trial, "node_id_dim", space["node_id_dim"])),
        contingency_id_dim=int(
            _suggest(trial, "contingency_id_dim", space["contingency_id_dim"])
        ),
        type_dim=int(_suggest(trial, "type_dim", space["type_dim"])),
        pair_dim=int(_suggest(trial, "pair_dim", space["pair_dim"])),
        num_gnn_layers=int(_suggest(trial, "num_gnn_layers", space["num_gnn_layers"])),
        decoder_hidden_dim=int(
            _suggest(trial, "decoder_hidden_dim", space["decoder_hidden_dim"])
        ),
        dropout=float(_suggest(trial, "dropout", space["dropout"])),
        lr=float(_suggest(trial, "lr", space["lr"])),
        weight_decay=float(_suggest(trial, "weight_decay", space["weight_decay"])),
    )

def _train_one(*, trial_obj, trial_root, hparams):
    return run_pair_aware_training(
        task=task,
        target_mask_attr=target_mask_attr,
        train_loader=train_loader,
        validation_loader=val_loader,
        test_loader=None,
        output_dir=trial_root,
        num_node_tokens=int(attachment["node_vocab_size"]),
        num_contingency_tokens=int(attachment["contingency_vocab_size"]),
        hparams=hparams,
        loss_weights=loss_weights,
        log_mean=log_mean,
        log_std=log_std,
        cuts=cuts_array,
        epsilon=epsilon,
        inactive_gate_threshold=inactive_gate_threshold,
        epochs=epochs,
        patience=patience,
        fixed_epochs=None,
        selection_output=selection_output_arg,
        class_weight_mode=class_weight_mode,
        inactive_gate_pos_weight_mode=inactive_gate_pos_weight_mode,
        selection_score_weights=selection_score_weights,
        num_classes=num_classes,
        logger=logger,
        trial=trial_obj,
    )

def objective(trial):
    trial_root = trials_root / f"trial_{trial.number}"
    hparams = _sample_hparams(trial, hparam_space)
    save_trial_params(
        trial_root, trial_number=int(trial.number), params=dict(trial.params)
    )
    result = _train_one(trial_obj=trial, trial_root=trial_root, hparams=hparams)
    trial.set_user_attr("selected_output", result["selected_output"])
    trial.set_user_attr("best_epoch", int(result["best_epoch"]))
    return float(result["best_validation_score"])

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=n_trials)

retrain_result = run_pair_aware_training(
    task=task,
    target_mask_attr=target_mask_attr,
    train_loader=train_val_loader,
    validation_loader=None,
    test_loader=None,
    output_dir=final_root,
    num_node_tokens=int(attachment["node_vocab_size"]),
    num_contingency_tokens=int(attachment["contingency_vocab_size"]),
```

```
    hparams=best_hparams,
    loss_weights=loss_weights,
    log_mean=log_mean,
    log_std=log_std,
    cuts=cuts_array,
    epsilon=epsilon,
    inactive_gate_threshold=inactive_gate_threshold,
    epochs=epochs,
    patience=patience,
    fixed_epochs=best_epoch,
    selection_output=selected_output,
    class_weight_mode=class_weight_mode,
    inactive_gate_pos_weight_mode=inactive_gate_pos_weight_mode,
    selection_score_weights=selection_score_weights,
    num_classes=num_classes,
    logger=logger,
    trial=None,
)

output = model(data)
logits = output["class_logits"]
gate_logit = output["inactive_logit"]
reg_prediction = output["log_kpi_std"]
if selected_output == "class":
    y_pred = logits.argmax(dim=1)
elif selected_output == "gated":
    y_pred = _decode_gated(logits, gate_logit, inactive_gate_threshold)
else:
    y_pred = _classes_from_log_prediction(
        reg_prediction.detach().cpu().numpy(),
        log_mean=log_mean, log_std=log_std, cuts=cuts, epsilon=epsilon,
)
```

A.6. Simplified cases, retention, and comparison to the benchmark

Simplified cases, retention, and comparison to the benchmark.

```
def _retained_substations_from_predictions(substation_predictions, epsilon):
    retained = set()
    threshold = float(epsilon)
    for substation_id, value in substation_predictions.items():
        sid = str(substation_id).strip()
        if not sid:
            continue
        cls = float(value)
        if cls >= threshold:
            retained.add(sid)
    return retained

def _patch_iidm_switches(root, retained_substations):
    for switch in _iter_by_local_tag(root, "switch"):
        switch.set("retained", "false")
    for substation in _iter_by_local_tag(root, "substation"):
        substation_id = substation.attrib.get("id", "").strip()
        if substation_id not in retained_substations:
            continue
        for switch in _iter_by_local_tag(substation, "switch"):
            switch.set("retained", "true")

def apply_node_breaker_simplification(
    iidm_path, epsilon, action_locations, substation_predictions
):
    tree = _load_iidm_tree(Path(iidm_path))
    root = tree.getroot()
    action_component_ids = {
        str(value).strip() for value in action_locations if str(value).strip()
    }
    from_actions, unmatched = substation_ids_for_components(root, action_component_ids)
    from_predictions = _retained_substations_from_predictions(
        substation_predictions, epsilon
```

```
    )
    retained_substations = set(from_actions)
    retained_substations.update(from_predictions)
    _patch_iidm_switches(root, retained_substations)
    _write_iidm_tree(tree, Path(iidm_path))
```

Simplified cases, retention, and comparison to the benchmark.

```
_, path = fastdtw(series1, series2, dist=lambda a, b: abs(a - b))
errors = [abs(series1[i] - series2[j]) for i, j in path]
rmse = np.sqrt(np.mean([e ** 2 for e in errors]))
```

A.7. Switch modelling and root-function bookkeeping

Switch modelling and root-function bookkeeping.

```
ModelSwitch::ModelSwitch(const std::shared_ptr<SwitchInterface>& sw) :
NetworkComponent(sw->getID()),
topologyModified_(false),
inLoop_(false),
canBeClosed_(!sw->isOpen() || sw->isRetained()) {
  if (sw->isOpen())
    connectionState_ = OPEN;
  else
    connectionState_ = CLOSED;
  irO_ = 0;
  iiO_ = 0;
  irYNum_ = 0;
  iiYNum_ = 0;
}

void ModelSwitch::initSize() {
  if (network_->isInitModel()) {
    sizeF_ = 2;
    sizeY_ = 2;
    sizeZ_ = 0;
    sizeG_ = 0;
    sizeMode_ = 0;
    sizeCalculatedVar_ = 0;
  } else {
    sizeF_ = 2;
    sizeY_ = 2;
    sizeZ_ = 1;
    sizeG_ = 0;
    sizeMode_ = 1;
    sizeCalculatedVar_ = nbCalculatedVariables_;
  }
}
```

Switch modelling and root-function bookkeeping.

```
void ModelSwitch::evalF(propertyF_t type) {
  if (type == DIFFERENTIAL_EQ)
    return;
  if (getConnectionState() == CLOSED && !modelBus1_->getSwitchOff()) {
    if (inLoop_) {
      f_[0] = y_[0] - 1.;
      f_[1] = y_[1] - 1.;
    } else {
      f_[0] = modelBus1_->ur() - modelBus2_->ur();
      f_[1] = modelBus1_->ui() - modelBus2_->ui();
    }
  } else {
    f_[0] = y_[0];
    f_[1] = y_[1];
  }
}

```

```
18 void ModelSwitch::setFequations(std::map<int, std::string>& fEquationIndex) {
19   if (getConnectionState() == CLOSED && !modelBus1_->getSwitchOff()) {
20     if (inLoop_) {
21       fEquationIndex[0] = std::string("y_[irNum_] - 1 localModel:").append(id());
22       fEquationIndex[1] = std::string("y_[iiNum_] - 1 localModel:").append(id());
23     } else {
24       fEquationIndex[0] = modelBus1_->id() + "->ur() - "
25                         + modelBus2_->id() + "->ur() localModel:" + id();
26       fEquationIndex[1] = modelBus1_->id() + "->ui() - "
27                         + modelBus2_->id() + "->ui() localModel:" + id();
28     }
29   } else {
30     fEquationIndex[0] = std::string("y_[irNum_] localModel:").append(id());
31     fEquationIndex[1] = std::string("y_[iiNum_] localModel:").append(id());
32   }
33 }
```